

Software Supply Chain Smells in React Applications

João Paulo Pinto Pires

**Master's degree in Informatics Engineering,
Specialization in
Software Engineering**

Supervisor: Isabel Azevedo

Porto, September 2026

Statement of Integrity

I hereby declare having that I have conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore, the work presented in this document is original and authored by me, having not previously been used for any other end. The exceptions are explicitly recognised in the section “Ethical considerations” of the first chapter. This section also states how AI tools were used and for what purpose.

I further declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

ISEP, Porto, September 20, 2026

Dedictory

To my family and friends, for their constant support and encouragement throughout this journey, especially during the most difficult moments.

To my supervisor, for her guidance, support, and valuable feedback throughout this dissertation.

Abstract

Modern software development relies extensively on third-party dependencies, increasing exposure to software supply chain risks that may affect security and maintainability. In the npm ecosystem, recurring dependency-related patterns, referred to as software supply chain smells, can indicate potentially undesirable conditions even when no concrete vulnerability is known. This dissertation addresses the identification, prioritisation, and visualisation of these smells in React applications that use npm as their package manager, to provide developers with more contextual and actionable information about dependency-related concerns.

The research follows a Design Science Research Methodology and is supported by a systematic literature review aligned with PRISMA, complemented by a forward snowballing process. The reviewed evidence was used to consolidate a catalogue of software supply chain smells covering dependency configuration and versioning, transparency and provenance, maintainer governance, and dependency-resolution concerns. Based on these findings, a Smell Severity Scoring System (SSSS) is proposed to prioritise detected smell instances by combining baseline smell severity with contextual factors related to dependency position, vulnerability exposure and persistence, and package responsiveness.

The proposed approach was implemented as a prototype that combines a backend analysis pipeline with an interactive web interface application. The backend combines multiple detection and enrichment mechanisms, normalises the resulting findings, applies the SSSS, and exports the analysis results as a structured JSON document. The frontend consumes this JSON output and presents the analysed dependencies through an interactive node-link graph with severity cues, filtering, and detailed smell information.

The prototype evaluation shows that the proposed approach can differentiate the priority of smell instances according to their dependency and vulnerability context rather than relying exclusively on baseline smell severity. The evaluated cases also show that contextual prioritisation and interactive exploration can support the inspection and triage of potentially large sets of dependency-related findings.

Keywords: Software Supply Chain Smells; npm; React; Severity Scoring; Dependency Visualisation.

Resumo

O desenvolvimento de software moderno depende em grande parte de dependências de terceiros, aumentando a exposição a riscos na cadeia de fornecimento de software que podem afetar a segurança e a manutenibilidade. No ecossistema npm, padrões recorrentes relacionados com dependências, designados neste trabalho como *smells* na cadeia de fornecimento de software, podem indicar condições potencialmente indesejáveis, mesmo quando não é conhecida uma vulnerabilidade concreta. Esta dissertação aborda a identificação, priorização e visualização destes *smells* em aplicações React que utilizam o npm como gestor de pacotes, com o objetivo de apoiar a análise de problemas relacionados com as dependências através de informação contextual.

A investigação segue a metodologia *Design Science Research* e é suportada por uma revisão sistemática da literatura alinhada com o PRISMA, complementada pelo processo de *forward snowballing*. As evidências analisadas permitiram definir um catálogo de *smells* na cadeia de fornecimento de software que abrangem configuração e versionamento de dependências, transparência e proveniência, aspetos relacionados com os responsáveis pela manutenção dos pacotes e problemas relacionados com a resolução de dependências. A partir destes resultados, é proposto um *Smell Severity Scoring System* (SSSS) para priorizar os *smells* detetados, tendo em conta a gravidade base e fatores contextuais relacionados com a posição da dependência, a exposição, persistência de vulnerabilidades e a capacidade de resposta do pacote.

A abordagem proposta foi implementada através de um protótipo composto por um pipeline de análise de *backend* e uma aplicação web interativa de *frontend*. O *backend* combina múltiplos mecanismos de deteção e enriquecimento, normaliza os resultados obtidos, aplica o SSSS e exporta os resultados para um documento JSON. O *frontend* consome esse documento JSON e apresenta as dependências analisadas através de um grafo interativo de nós e arestas com indicadores de gravidade, filtragem e informação detalhada sobre o *smell*.

A avaliação do protótipo demonstra que a abordagem proposta consegue diferenciar a prioridade das instâncias dos *smells* de acordo com o contexto da dependência e vulnerabilidade, em vez de se basear exclusivamente na gravidade inicial do *smell*. Os casos avaliados mostram também que a priorização contextual e a exploração interativa podem apoiar a análise e a triagem de grandes volumes de resultados relacionados com dependências.

Palavras-chave: Smells na Cadeia de Fornecimento de Software; npm; React; Pontuação de Gravidade; Visualização de Dependências.

Table of Contents

List of Figures	xv
List of Tables	xvii
List of Listings	xix
List of Abbreviations	xxi
1 Introduction	1
1.1 Context	1
1.2 Problem Description	2
1.3 Objectives, Research Questions and Methodology	2
1.4 Ethical Considerations	4
1.4.1 Use of AI tools	4
1.5 Project Planning	5
1.5.1 Scope of Work and Project Charter	5
1.5.2 WBS and Integrated Project Schedule	5
1.5.3 Risk Identification and Management	8
1.6 Skill Management	9
1.6.1 Identification of Required Skills	9
1.6.2 Assessment of Current Skills	10
1.6.3 Strategies for Skill Development	11
1.6.4 Identification of Skills to Improve	12
1.7 Document Structure	12
2 Background	15
2.1 React and npm Ecosystem Scope	15
2.2 Fundamentals of Software Quality and Security	16
2.2.1 Security as a Software Quality Characteristic (ISO/IEC 25010)	16
2.2.2 Weaknesses, Vulnerabilities, and Risk	16
2.2.3 Software Supply Chain Smells	16
2.3 Software Supply Chain Risks	17
2.3.1 OWASP Top 10:2025	17
2.3.2 Analysis and Detection Tools	17
2.3.3 Supply Chain Transparency and SBOMs (OWASP CycloneDX)	18
2.4 Vulnerabilities Prioritisation and Scoring	19
2.4.1 Scoring and Prioritisation Models (CVSS and EPSS)	19
2.4.2 OWASP Risk Rating Methodology	20
2.5 Software Dependency Visualisation and Graphs	21
2.5.1 Dependency Graphs	21
2.5.2 Security Graph Visualisation with Cytoscape.js	22

2.6	Summary	22
3	Literature Review	23
3.1	Research Design and Approach	23
3.2	Research Questions	24
3.3	Data Sources	24
3.4	Search Queries.....	25
3.5	Eligibility Criteria	25
3.5.1	Inclusion Criteria	26
3.5.2	Exclusion Criteria	26
3.6	Data Collection Process	27
3.7	Research and Results	29
3.7.1	RQ1: What types of software supply chain smells can be systematically identified and defined in React applications?.....	29
3.7.2	RQ2: How can a set of software supply chain smells, specific to React applications, be operationalised in an SSSS to effectively prioritise remediation actions?.....	34
3.7.3	RQ3: What information and contextual cues can an SSSS-based dependency graph visualisation provide for identifying and prioritising software supply chain smells, compared with existing list-based analysis tools?.....	36
3.7.4	Forward Snowballing.....	37
3.7.5	Summary.....	40
4	Analysis and Design	45
4.1	Smell Catalogue Design	45
4.1.1	Define Criteria	45
4.1.2	Classify Smells	46
4.2	Smell Severity Scoring System Design (SSSS)	51
4.2.1	Review Severity Models	52
4.2.2	SSSS Dimensions	53
4.2.3	Scoring Formula	55
4.3	Prototype Design	62
4.3.1	Requirements	63
4.3.2	Solution Architecture	68
5	Implementation	77
5.1	Implementation Overview	77
5.2	Technology Stack and Development Environment.....	78
5.3	Backend Implementation	80
5.3.1	Input Handling and Project Extraction	81
5.3.2	Dependency Graph Extraction.....	81
5.3.3	External Detector Adapter: Dirty-Waters Integration.....	82
5.3.4	Custom Smell Detectors.....	83
5.3.5	SSSS Scoring Engine	85

5.3.6	Output Generation: JSON and Markdown	87
5.4	Frontend Implementation.....	88
5.4.1	JSON Loading and Data Mapping	88
5.4.2	Dependency Graph Rendering	89
5.4.3	Filtering and Interaction Mechanisms	89
5.4.4	Smell Detail Panel and Summary View	90
5.5	Execution Workflows.....	90
5.5.1	Local Developer Workflow	90
5.5.2	Automation-Oriented Workflow	92
5.6	Implementation Verification.....	93
5.7	Implementation Evaluation	98
5.7.1	Evaluation Strategy	98
5.7.2	Evaluation Cases and Overall Results.....	99
5.7.3	SSSS Prioritisation Behaviour	100
5.7.4	Information Provided by the Prototype	101
5.7.5	SSSS Sensitivity Analysis	103
5.8	Implementation Limitations	105
5.9	Summary.....	106
6	Conclusion	109
6.1	Achievements and Contributions.....	109
6.2	Difficulties and Challenges.....	110
6.3	Limitations	111
6.4	Future Work.....	112
	Bibliography.....	115
	Appendix A - Project Charter.....	121
	Appendix B - WBS	124
	Appendix C - Backend Analysis Sequence.....	125

List of Figures

Figure 1 - PRISMA Flow diagram for the data collection process	27
Figure 2 - Use case diagram of the proposed prototype	68
Figure 3 - High-level backend analysis and visualisation sequence	69
Figure 4 – Proposed Solution Architecture	73
Figure 5 - Mockup of the Web Application Layer	74
Figure 6 - Analysis and Scoring Implementation Pipeline	78
Figure 7 - React application after the user selects a JSON file.....	88
Figure 8 - Local Workflow Diagram.....	92
Figure 9 - Successful execution of the dependency analysis workflow in GitHub Actions	93
Figure 10 - Filtered frontend view of the portfolio evaluation case	102

List of Tables

Table 1 - Project Schedule.....	6
Table 2 - Project risks identification.....	8
Table 3 - Assessment of current skills and gap prioritisation	10
Table 4 - Selected databases.....	25
Table 5 - Search queries.....	25
Table 6 - Inclusion criteria.....	26
Table 7 - Exclusion criteria	26
Table 8 - List of reports included in this review	28
Table 9 - Summary of findings from Jafari et al. [49] relevant to SSSS design.....	35
Table 10 - Study included through forward snowballing.....	38
Table 11 - Practitioner ratings of software supply chain smells. Adapted from Schmid et al. [27]	40
Table 12 - Summary of smell dimensions and representative smells/signals (RQ1)	41
Table 13 - SSSS dimensions and operational implications (RQ2).....	42
Table 14 - Classification and final designation adopted of software supply chain smells derived from the literature review	46
Table 15 - Baseline severity criteria for software supply chain smells	48
Table 16 - Baseline severity classification of operationalised software supply chain smells	49
Table 17 - Reference scoring and prioritisation models considered for the SSSS design.....	53
Table 18 - Proposed SSSS dimensions and weights	54
Table 19 - Dimension normalisation	55
Table 20 - Baseline severity normalisation	56
Table 21 - Production reachability and dependency depth normalisation	57
Table 22 - Vulnerability exposure and persistence normalisation.....	58
Table 23 - Responsiveness and update strategy normalisation	59
Table 24 - Final SSSS score to rating mapping	60
Table 25 - Dimension values for Example 1: Deprecated production dependency with a high vulnerability	60
Table 26 - Dimension values for Example 2: Pinned dependency without a known vulnerability	61
Table 27 - Dimension values for Example 3: Inaccessible release reference in a transitive dependency with unavailable vulnerability data	62
Table 28 - Architectural concerns and constraints considered for the prototype	63
Table 29 - Functional requirements and acceptance criteria	64
Table 30 - Non-functional requirements and acceptance criteria.....	66
Table 31 - Proposed JSON output structure	70
Table 32 - Detection mechanisms and data sources associated with each operationalised smell	76
Table 33 - Technology stack and role of each technology in the prototype.....	79
Table 34 - Custom smell detector families and implemented detection logic	84

Table 35 - JSON top-level sections	87
Table 36 - Test coverage of selected critical backend modules	93
Table 37 - Test coverage of principal frontend modules	94
Table 38 - Functional requirements verification	95
Table 39 - Non-functional requirements verification.....	97
Table 40 - Characterisation and overall results of the evaluation cases.....	99
Table 41 - Representative examples of SSSS prioritisation behaviour.....	100
Table 42 - Information exposed by list-based outputs and the proposed prototype.....	102
Table 43 - Sensitivity to the baseline severity weight (S).....	103
Table 44 - Sensitivity to the production relevance weight (P)	104
Table 45 - Sensitivity to the vulnerability weight (V)	104
Table 46 - Sensitivity to the responsiveness weight (R).....	104

List of Listings

Listing 1 - Example of the JSON output structure	72
Listing 2 - Example of a scored smell instance generated by the SSSS	86
Listing 3 - Backend analysis command using the npm script	91
Listing 4 - Direct backend analysis command using Node.js	91
Listing 5 - Frontend development server command	91

List of Abbreviations

2FA	Two-Factor Authentication
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
CI/CD	Continuous Integration / Continuous Delivery
CLI	Command-Line Interface
CRA	Cyber Resilience Act
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DNS	Domain Name System
DSR	Design Science Research
DSRM	Design Science Research Methodology
EPSS	Exploit Prediction Scoring System
GHSA	GitHub Security Advisory
JSON	JavaScript Object Notation
JSX	JavaScript Syntax Extension
npm	Node Package Manager
OWASP	Open Worldwide Application Security Project
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
RDAP	Registration Data Access Protocol
ROC	Receiver Operating Characteristic
RQ	Research Question
SBOM	Software Bill of Materials

SCA	Software Composition Analysis
SemVer	Semantic Versioning
SHA	Secure Hash Algorithm
SLR	Systematic Literature Review
SSSS	Smell Severity Scoring System
URL	Uniform Resource Locator
WBS	Work Breakdown Structure

1 Introduction

This chapter introduces the subject of this dissertation by presenting the context and relevance of software supply chain smells in React applications that use Node Package Manager (npm) [1] as their package manager. It begins by describing the problem addressed, the project's objectives, the research questions, and the adopted research methodology. It then presents the ethical considerations that guide the work, including the responsible use of AI tools. In addition, the chapter introduces the main project planning artefacts, including the scope of work, the project charter, the Work Breakdown Structure (WBS), the integrated project schedule, and the approach to risk identification and management. Finally, the chapter analyses the technical and research skills required to achieve the proposed objectives, assesses the current skill set, identifies relevant gaps and strategies to address them, and concludes with an overview of the document structure.

1.1 Context

Modern software development frequently relies on open-source dependencies, which involves significant trust in third-party code and raises concerns about software supply chain security [2], [3]. Liu et al. [3] define a software supply chain smell as a package that matches specific patterns indicating potential security issues, either currently or in the future.

Software maintainability and security in component-based web applications, like those built with React, can be compromised by software supply chain smells [3]. These smells, such as using unmaintained, stale, or bloated dependencies, can introduce potential risks that developers are often unaware of. While there has been some work on visually analysing the structure and code quality of these applications [4], there is a lack of a comprehensive approach that specifically targets and visualises these supply chain smells. On the other hand, developers are often overwhelmed by the volume of detected issues and lack actionable insights to triage and remediate them effectively [5]. This makes it difficult for developers to proactively address issues, with severity-based prioritisation, related to dependency hygiene and security.

The SolarWinds incident is widely referenced as a paradigmatic software supply chain compromise because malicious code was introduced into a trusted vendor software lifecycle and then propagated downstream through legitimate updates of the SolarWinds Orion platform. CISA's joint advisory describes how the adversary inserted a malicious version of SolarWinds.Orion.Core.BusinessLayer.dll into the Orion software lifecycle and had it signed with SolarWinds' legitimate code signing certificate, enabling distribution through normal update channels and subsequent victim compromise [6]. In response, CISA issued Emergency Directive 21-01, underscoring the systemic impact of the campaign and requiring rapid mitigation actions for affected environments, illustrating how compromise of upstream build/update processes can cascade across many downstream organisations [7].

In the European context, the EU Cyber Resilience Act (CRA) strengthens the regulatory imperative for proactive management of software and supply chain risk by establishing horizontal cybersecurity requirements for “products with digital elements”. The European Commission summarises that the CRA entered into force on 10 December 2024, that the main obligations apply from 11 December 2027, and that reporting obligations apply earlier from 11 September 2026, providing a clear compliance timeline that incentivises systematic vulnerability handling and risk reduction practices across the software lifecycle [8]. In addition, the Commission’s CRA reporting guidance operationalises expectations for timely disclosure by requiring manufacturers to submit an early warning within 24 hours of becoming aware of actively exploited vulnerabilities or severe incidents, followed by a full notification within 72 hours, and subsequent final reporting within defined deadlines [9]. These requirements further motivate approaches that improve visibility and prioritisation of supply chain security issues.

1.2 Problem Description

The underlying problem spans multiple, sometimes conflicting, technical dimensions involving both software maintainability and software security. Modern development practices rely heavily on open-source dependencies, which require a significant level of trust in third-party code [3]. Every dependency introduces both value and risk [5], yet developers commonly lack effective mechanisms to assess, compare, or visualise that risk within complex dependency graphs. Actions intended to improve one quality attribute, such as functionality, rapid integration, or performance, may unintentionally degrade others, such as security or maintainability.

Existing tools, such as the npm and third-party solutions, such as OWASP Dependency-Check [10] and Snyk [11], provide partial support but do not constitute an obvious or comprehensive solution. Software Composition Analysis (SCA) tools have been developed to scan dependencies and cross-reference them with known vulnerability databases, while Software Bills of Materials (SBOMs) offer formal, machine-readable inventories of components and their hierarchical relationships [3]. Although these techniques are valuable for security auditing, they primarily surface raw lists of vulnerabilities or versioning issues and rarely provide visual, contextual, or prioritisation mechanisms that help developers understand the structural implications of supply chain smells, such as unmaintained, stale, or bloated dependencies. As a result, current approaches fail to support developers in holistically identifying, interpreting, and managing supply chain smells within the specific context of React applications.

1.3 Objectives, Research Questions and Methodology

The general objective of this work is to design, implement, and evaluate a comprehensive approach for identifying, analysing, prioritising, and visualising software supply chain smells in React applications that use npm as their package manager.

For that, the next specific objectives were defined:

- Define and operationalise a set of software supply chain smells that affect React applications.
- Propose a Smell Severity Scoring System (SSSS) to prioritise corrective actions.
- Develop a visualisation tool that displays dependencies as a node-link diagram, enriched with visual cues based on the SSSS.
- Evaluate the prototype on real npm/React projects in terms of smell detection, prioritisation behaviour, and the information provided, including a comparison with existing analysis tools.

The scope of this dissertation is limited to applications that use npm as their package manager and are demonstrated in React projects. This delimitation is important because dependency metadata, lockfiles, version resolution rules, peer dependency behaviour, and available registry information differ across package managers. Therefore, the smell catalogue, SSSS, and prototype are designed and evaluated in the context of npm-based React projects, while support for other package managers is considered a possible future extension.

To address the general and specific objectives outlined above, this dissertation is guided by three research questions that structure both the theoretical grounding and the artefact-oriented contributions:

- RQ1 examines the different types of supply chain smells and the approaches for identifying them, with particular emphasis on their relevance to the React and npm ecosystem.
- RQ2 investigates how software supply chain smells relevant to React applications can be operationalised into an SSSS to prioritise remediation actions effectively.
- RQ3 examines the information and contextual cues that an SSSS-based dependency graph visualisation can provide for identifying and prioritising software supply chain smells, compared with existing list-based analysis tools.

The research follows the Design Science Research Methodology (DSRM) within a Design Science Research (DSR) approach, organised into six primary activities: (i) problem identification and motivation, (ii) definition of objectives for a solution, (iii) design and development, (iv) demonstration, (v) evaluation, and (vi) communication, with iterations driven by evidence gathered during demonstration and evaluation [12]. In this dissertation, the designed artefact comprises: (i) a consolidated catalogue of software supply chain smells relevant to npm-based React applications, with a defined subset operationalised in the prototype; (ii) an SSSS that combines baseline severity with contextual factors to prioritise detected smell instances; and (iii) an interactive dependency-graph visualisation that integrates detected smells, SSSS-based prioritisation, filtering, and structural dependency context.

To provide a solid theoretical base, a systematic literature review (SLR) was conducted in an initial phase and complemented by the forward snowballing process [13] described in Chapter

3. The SLR follows the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) [14] guidelines, ensuring transparency and reproducibility in the processes of identification, screening, eligibility assessment, and inclusion of studies. The review addresses the research questions by synthesising existing knowledge on software supply chain smells, severity assessment, and dependency visualisation, with particular emphasis on React applications.

Based on the literature review, the smell catalogue and the SSSS were defined, and a subset of the identified smells was operationalised in the prototype. Implementation verification assesses whether the resulting artefact satisfies the functional and non-functional requirements and their acceptance criteria. The prototype is subsequently evaluated on two public React/npm projects by examining the generated findings, SSSS prioritisation behaviour, the information provided by the frontend, and the sensitivity of the selected SSSS weights.

1.4 Ethical Considerations

This dissertation complies with the Code of Good Practices and Conduct of P.Porto [15], with particular attention to the duties established for students in article 6th (point n), the declaration of commitment in article 8th, and the best practices for research activities defined in article 10th.

This dissertation also complies with the ACM/IEEE Software Engineering Code of Ethics and Professional Practice [16], which articulates eight guiding principles that prioritise public welfare, high professional standards for software artefacts, and integrity and independence in technical judgment. Accordingly, the work emphasises responsible risk communication, demonstrable competence and methodological rigour in the design and evaluation of the proposed tool, and the careful handling of any potentially sensitive information in a manner consistent with the public interest.

The dissertation is aligned with the European Code of Conduct for Research Integrity (ALLEA, Revised Edition 2023) [17], which frames good research practice around four principles - reliability, honesty, respect, and accountability - and provides guidance on transparent and reproducible procedures, robust data practices and stewardship, and responsible dissemination of results. These commitments are operationalised through traceable documentation of methods and assumptions, appropriate attribution, and clear reporting of limitations, including explicit disclosure of any use of external services and AI/automated tools where applicable.

1.4.1 Use of AI tools

In line with the European Code of Conduct for Research Integrity (ALLEA, Revised Edition 2023) [17], this dissertation discloses the use of AI-based support tools to ensure transparency and accountability in the research process. Specifically, ChatGPT was used as an auxiliary aid for drafting and structuring text and for language refinement, with all outputs subject to critical review by the author, who remains fully responsible for the accuracy, originality, and integrity

of the final content and for ensuring that no results, analyses, or evidence were fabricated or substituted by the tool.

Importantly, AI tools were not used to automatically generate literature review content, to fabricate results, or to replace the author's own analysis and interpretation. The identification of software supply chain smells, the design of the SSSS, the implementation of the visualisation prototype, and the evaluation of the proposed solution were carried out by the author based on academic literature, technical documentation, and practical experimentation.

1.5 Project Planning

1.5.1 Scope of Work and Project Charter

The scope of this project is to design, implement, and evaluate a prototype tool for identifying, analysing, prioritising, and visualising software supply chain smells in React applications. The work focuses on smells related to dependencies such as unmaintained, stale, or bloated in the npm ecosystem, and on how these smells affect maintainability and security. The main outputs include a definition of a set of software supply chain smells in React applications, an SSSS, and an interactive visualisation tool that represents dependencies and components using a node-link diagram enriched with severity indicators.

The project charter defines the main elements of the dissertation, including its title, sponsor, supervisor, and student, as well as the problem statement, objectives, stakeholders, and expected deliverables. It also describes the problem addressed in this dissertation and the need for mechanisms that help developers visualise and prioritise dependency-related concerns in React applications.

The project charter serves as a reference throughout the project, helping to maintain alignment between expectations, scope, and available resources.

1.5.2 WBS and Integrated Project Schedule

The WBS decomposes the project into manageable work packages and tasks, providing a hierarchical view of all the activities required to achieve the project objectives. At the highest level, the WBS includes tasks such as project management and planning, literature review, smell catalogue design, severity scoring model design, visualisation tool design, prototype implementation and testing, project evaluation, writing and revision of the dissertation and associated deliverables.

The WBS diagram (Appendix B – WBS) illustrates these relationships and helps identify dependencies between tasks, serving as a basis for the project schedule and risk management.

The integrated project schedule is shown in a schedule that maps the main work packages and tasks from the WBS into the calendar of the dissertation phases. The schedule considers the milestones defined in the project charter, such as the delivery of the extended abstract, the final dissertation preparation (PREPD) document, the prototype baseline for evaluation, the prototype final package, and the final dissertation document. Literature review activities begin early and continue iteratively, supporting both the definition of the smells and the scoring model, while design and implementation phases are planned to start once a sufficient theoretical basis has been established.

Table 1 summarises the integrated project schedule, mapping the main WBS work packages to the project calendar and key milestones.

Table 1 - Project Schedule

Phase	Activity	Begin	End
1.1	Planning	15/10/2025	30/11/2025
1.1.1	Define Project Scope and Objectives	15/10/2025	21/10/2025
1.1.1.1	Create Project Charter	22/10/2025	03/11/2025
1.1.2	Planning (project planning package)	04/11/2025	30/11/2025
1.1.2.1	Create WBS	04/11/2025	15/11/2025
1.1.2.2	Create Gantt Chart / Integrated Schedule	16/11/2025	30/11/2025
1.1.2.3	Risk Identification and Management	16/11/2025	30/11/2025
1.2	Literature Review	15/11/2025	30/04/2026
1.2.1.1	Design Research Questions	15/11/2025	01/12/2025
1.2.1.2	Define Search Strategy	02/12/2025	07/12/2025
1.2.1.3	Paper Screening and Selection	08/12/2025	20/12/2025
M1	Extended Abstract (milestone)	15/12/2025	15/12/2025
1.2.1.4	Analysis of Papers	21/12/2025	04/01/2026
1.2.1.5	Forward Snowballing	01/01/2026	30/04/2026
1.2.1.6	Summarise Findings / PREPD consolidation	05/01/2026	07/01/2026
M2	Final PREPD Document (milestone)	07/01/2026	07/01/2026
1.3	Design	01/02/2026	15/05/2026
1.3.1	Smell Catalogue Design	01/02/2026	01/05/2026
1.3.2	Smell Severity Scoring System Design	01/03/2026	01/05/2026
1.3.3	Visualisation Design	01/05/2026	15/05/2026
1.4	Build	15/05/2026	30/07/2026

Table 1 (continued)

Phase	Activity	Begin	End
1.4.1	Prototype Implementation	15/05/2026	30/07/2026
1.4.1.1	Implement Data Extraction from React Projects	15/05/2026	25/07/2026
1.4.1.2	Implement Smell Detection Logic	15/05/2026	25/07/2026
1.4.1.3	Implement Smell Severity Scoring System	15/06/2026	25/07/2026
1.4.1.4	Implement Visualisation Application	01/06/2026	16/07/2026
1.4.1.5	Integrate Components	15/05/2026	25/07/2026
1.4.2	Testing (unit + rendering + fixes)	15/05/2026	30/07/2026
M3	Functional Prototype (milestone)	31/07/2026	31/07/2026
1.5	Evaluate	01/08/2026	31/08/2026
1.5.1	Evaluation Preparation	01/08/2026	10/08/2026
1.5.1.1	Define Evaluation Tasks	01/08/2026	10/08/2026
1.5.1.2	Prepare Sample Projects	01/08/2026	10/08/2026
1.5.2	Data Analysis	10/08/2026	25/08/2026
1.5.2.1	Quantitative Analysis	10/08/2026	25/08/2026
1.5.2.2	Qualitative Analysis	10/08/2026	28/08/2026
1.5.2.3	Compare With Existing Tools	20/08/2026	25/08/2026
1.5.2.4	Evaluation Summary and Conclusions	25/08/2026	31/08/2026
1.6	Final Dissertation Artefacts	06/01/2026	20/09/2026
1.6.1	Writing and Revision (iterative)	06/01/2026	10/09/2026
1.6.1.1	Write Different Chapters	06/01/2026	01/09/2026
1.6.1.2	Supervisor Review Cycles	01/02/2026	20/09/2026
1.6.1.3	Final Proofreading	01/09/2026	20/09/2026
1.6.3	Prototype Final Package and Submission	18/09/2026	20/09/2026
1.6.3.2	Prepare User Guide (README / quick-start)	01/09/2026	09/09/2026
1.6.3.1	Final Release (tag/version)	15/09/2026	15/09/2026
1.6.3.3	Package and Publish Artefacts	16/09/2026	20/09/2026
1.6.2	Submission and Presentation	16/09/2026	30/09/2026
M4	Final Dissertation Document (milestone)	20/09/2026	20/09/2026
1.6.2.2	Prepare Presentation	20/09/2026	30/09/2026
1.6.2.3	Oral Defence (date per institution)	-	-

1.5.3 Risk Identification and Management

Risk management is integrated into the project planning process to anticipate events that may adversely affect the schedule, the feasibility of implementation, and the quality of the final artefact. The purpose of this section is to identify the main project risks and define mitigation actions that reduce both their likelihood and potential impact. Risk monitoring was performed through regular progress reviews aligned with the project plan, enabling early detection of deviations and timely corrective action.

The main risks identified for this project are: (i) schedule delays, (ii) resource unavailability, and (iii) performance limitations. Table 2 summarises each risk, together with an assessment of probability and impact and the corresponding mitigation strategy.

Table 2 - Project risks identification

Code	Risk	Probability	Impact	Mitigation strategy
RIS-01	Schedule delays	Medium	High	Weekly plan reviews; prioritisation of critical tasks; incremental delivery with intermediate milestones and time buffers for integration and evaluation.
RIS-02	Resource unavailability	Medium	High	Initial identification of external dependencies; use of alternative sources and tools;
RIS-03	Performance limitations	Medium	High	Continuous evaluation and monitoring; incremental and modular development; optimisation techniques (e.g., filtering and progressive disclosure) and early testing with representative graph sizes.

RIS-01 Schedule delays

Schedule delays may occur from the inherent complexity of combining research activities (evidence synthesis and model definition) with iterative software development. In this project, delays can be triggered by unexpected technical challenges (e.g., extracting and modelling transitive dependency graphs, integrating smell detection, or refining the scoring logic) and by rework driven by evolving design decisions. Mitigation focuses on establishing a disciplined monitoring cycle through weekly reviews of the work plan and systematic tracking of critical tasks.

RIS-02 Resource unavailability

This risk concerns the unavailability or instability of resources required for implementation and analysis, including access to dependency data, limitations imposed by registries or APIs, platform downtime, or constraints in the availability of supporting tools. If realised, such issues

can block development activities and delay the preparation of evaluation materials. The mitigation strategy is to identify external dependencies early and reduce single points of failure by maintaining alternative data sources and tool options.

RIS-03 Performance limitations

Performance limitations may happen from large dependency graphs, the time needed to build the dependency graph and calculate the indicators used by the scoring model, and the rendering costs of interactive node-link visualisations. If the prototype becomes slow or unresponsive, it may delay exploration and triage, thereby reducing usability and weakening the internal validity of the evaluation. Mitigation is pursued through an iterative, performance-aware development approach: evaluation is conducted at regular intervals, and scalability is assessed early using representative projects with large dependency graphs.

1.6 Skill Management

Skill management is treated as a planning and control activity to ensure that the project objectives are achievable within the available time and resources. It identifies the competencies required to design and evaluate the proposed artefacts, namely the React-focused smell catalogue, the SSSS, and the visualisation prototype. It establishes a structured approach to assess current proficiency, plan targeted improvement actions, and reduce execution risk.

1.6.1 Identification of Required Skills

The project requires competencies across four complementary areas:

- Frontend and ecosystem engineering: proficiency in React and JavaScript/TypeScript, together with practical understanding of the npm ecosystem (semantic versioning, lockfiles, dependency resolution, build tooling, and testing practices).
- Supply chain and dependency risk analysis: familiarity with software supply chain risk concepts, and the ability to interpret outputs from Software Composition Analysis (SCA) tools and related practices (e.g., dependency inventory and vulnerability/maintenance signals).
- Data modelling and visual analytics: capability to model dependency structures as graphs and implement interactive node-link visualisations, including selecting suitable layout strategies and interaction patterns to support exploration and triage.
- Research and evaluation skills: ability to conduct a PRISMA-aligned SLR, translate evidence into operational definitions and scoring rules, and design an evaluation that combines artefact-based validation with developer-oriented feedback.

In addition, transversal skills such as time management, academic writing in English, and effective communication with the supervisor are required to sustain progress and ensure research quality.

1.6.2 Assessment of Current Skills

To make the assessment explicit and reproducible, proficiency is rated using a four-level scale: 1 - basic awareness, 2 - working knowledge, 3 - proficient, 4 - advanced. For each skill, the table records the required level, current level, and the priority of the gap (High/Medium/Low), based on expected impact on near-term deliverables.

Table 3 - Assessment of current skills and gap prioritisation

Skill area	Required	Current	Gap priority	Brief note
React + JavaScript/TypeScript development	3	2	Low	Adequate foundation for prototyping and iteration.
npm dependency management (lockfiles, resolution, SemVer)	3	2	Medium	Needs strengthening to model transitive graphs reliably.
Software supply chain risk concepts (npm ecosystem)	3	1-2	High	Limited baseline, critical for smell definition and interpretation.
SCA tools and workflows (e.g., Dependency-Check/Snyk)	3	1-2	High	Required to ground smells in practical signals and tool outputs.
SBOM concepts (e.g., CycloneDX / inventory mindset)	2-3	1-2	Medium	Useful to structure dependency representation and traceability.
Graph modelling + interactive visualisation (node-link)	3	2	High	Central to the visual analytics artefact.
SLR with PRISMA	3	2	Medium	Exists some experience but must be reinforced.
Severity/scoring model design (operationalisation)	3	1-2	High	New challenge; Underpins prioritisation artefact.

Table 3 (continued)

Skill area	Required	Current	Gap priority	Brief note
Evaluation design	3	1-2	High	Essential for credible validation of usefulness and communication.
Academic writing and communication	3	2	Medium	Ongoing refinement is needed to ensure clarity and rigour.
Time management and planning discipline	3	2	Medium	Required to manage iterative research + implementation workstreams.

This assessment indicates that the baseline skills for React development are sufficient for the project. However, the main skill gaps are in supply chain risk analysis, graph-based visualisation, and the design of the scoring and evaluation approach, which are critical to the success of the proposed artefacts.

1.6.3 Strategies for Skill Development

To address the prioritised gaps, skill development was conducted through targeted learning and short proof-of-concept cycles:

- Focused study blocks: allocate dedicated time early in the schedule to consolidate supply chain and dependency-risk fundamentals, using a curated set of recent academic and practitioner sources aligned with the dissertation scope.
- Tool-based experimentation: run controlled experiments with npm graph extraction and at least one SCA tool to understand detection logic, common false positives/negatives, and data that can be operationalised as smell signals.
- Prototype-driven learning: implement small, incremental prototypes for dependency graph parsing and node-link rendering to de-risk the visualisation approach and converge on a viable interaction model.
- Methodological reinforcement: follow structured guidance for PRISMA execution and data extraction to ensure the literature review remains transparent and reproducible.
- Supervision feedback loops: use regular meetings to validate decisions on smell definitions, scoring rules, and evaluation instruments, and to review writing outputs for clarity and rigor.

These strategies are designed to reduce rework by ensuring that the most uncertain competencies (visualisation, scoring, and evaluation) are strengthened before dependent deliverables are due.

1.6.4 Identification of Skills to Improve

Based on the assessment, the priority competencies to improve during PREPD and the early dissertation phase are:

- Supply chain risk and “smell” interpretation in npm/React contexts, to support defensible smell definitions and avoid superficial signals.
- Practical SCA usage and result interpretation, to ground the approach in realistic detection outputs and constraints.
- Graph-based modelling and interactive visualisation design, to ensure the prototype can effectively support localisation and triage in large dependency graphs.
- Operationalisation and scoring model design, to translate qualitative findings into consistent prioritisation rules.
- Evaluation design and execution, to validate whether the artefact improves developer understanding and decision-making compared to existing alternatives.

Improving these skills is directly tied to the project objectives and deliverables, since they underpin the smell catalogue, the SSSS, the visualisation tool, and the evaluation of their effectiveness.

1.7 Document Structure

This dissertation is organised into six chapters, each addressing a specific stage of the research process, from the problem definition and theoretical foundations to the design, implementation, and conclusion of the proposed solution.

Chapter 1 introduces the dissertation context and motivation, defines the problem, objectives, and research questions, and presents the adopted methodology. It also addresses ethical considerations, project planning, skill management, and the overall document structure.

Chapter 2 presents the conceptual and technical background required for the dissertation. It covers the React and npm ecosystem, software supply chain risks, relevant analysis and detection tools, vulnerability prioritisation approaches, and dependency graph visualisation concepts.

Chapter 3 describes the systematic literature review and the complementary forward snowballing process. It presents the research protocol, selected studies, and the findings used to identify software supply chain smells, support the design of the SSSS, and motivate the proposed visualisation approach.

Chapter 4 presents the analysis and design of the proposed solution. It defines the consolidated smell catalogue, the SSSS, the functional and non-functional requirements, the prototype architecture, and the detection strategies associated with each smell.

Chapter 5 describes the proposed solution implementation. It presents the technology stack, backend and frontend implementation, and the supported execution workflows. The chapter also distinguishes implementation verification from prototype evaluation: the verification section assesses whether the implemented solution satisfies the defined functional and non-functional requirements and their acceptance criteria, while the evaluation section examines the prototype on real npm/React projects in terms of smell detection, prioritisation behaviour, and the information provided, including a comparison with existing analysis tools. The chapter concludes by discussing the main implementation limitations.

Chapter 6 concludes the dissertation by summarising the main achievements and contributions, discussing the difficulties and limitations identified throughout the work, and presenting directions for future research and development.

2 Background

This chapter presents the conceptual and technical background required to understand the scope of this dissertation and the design decisions introduced in later chapters. It first outlines the main characteristics of the React and npm ecosystem, followed by fundamental concepts related to software quality, security, vulnerabilities, and software supply chain smells. It then introduces relevant supply chain risks, analysis and detection tools, prioritisation approaches, and dependency graph visualisation concepts that support the implementation and evaluation of the proposed prototype.

2.1 React and npm Ecosystem Scope

React is a JavaScript library for building user interfaces from reusable components [18]. This dissertation analyses software supply chain smells specifically in the context of React applications that use the npm ecosystem. npm projects describe package metadata and dependency declarations primarily through the package.json manifest. Dependencies may be declared using exact versions, Semantic Versioning (SemVer) ranges, tags, Git repositories, remote tarballs, local directories, and other supported package specifiers. npm also distinguishes different dependency roles, including regular, development, optional, and peer dependencies. In particular, peer dependencies express compatibility with another host package and, since npm 7, npm automatically considers them during installation and dependency resolution [19], [20].

npm lockfiles provide a complementary view of the dependency state. The package-lock.json file records the concrete dependency tree generated by npm, including resolved package versions and dependency relationships, allowing subsequent installations to reproduce the same dependency tree. Modern npm lockfiles also contain sufficient information to reconstruct the installed package graph without inspecting every dependency manifest individually [21]. npm additionally supports npm-shrinkwrap.json, which uses the same format and serves a similar dependency-locking purpose, but may be included when publishing a package. When both files are present in the project root, npm-shrinkwrap.json takes precedence over package-lock.json [21].

The implementation later described in Chapter 5 uses npm-compatible parsers rather than reproducing these semantics through ad-hoc string processing. npm-package-arg parses the different package-specifier forms accepted by npm, while node-semver provides operations for parsing and evaluating SemVer versions and ranges [22], [23]. These libraries therefore provide a technical basis for consistently interpreting dependency declarations according to the same concepts used by the npm ecosystem.

2.2 Fundamentals of Software Quality and Security

2.2.1 Security as a Software Quality Characteristic (ISO/IEC 25010)

ISO/IEC 25010 positions security as a core product quality characteristic and decomposes it into the sub-characteristics: confidentiality, integrity, non-repudiation, accountability, authenticity and resistance [24]. These dimensions are particularly relevant for modern web applications, where security properties depend not only on first-party code but also on third-party components and the processes used to build and distribute software artefacts.

2.2.2 Weaknesses, Vulnerabilities, and Risk

A clear separation between weakness, vulnerability, and risk is essential for consistent analysis and prioritisation. In the CWE glossary, a vulnerability is defined as a flaw in a software, firmware, hardware, or service component resulting from a weakness that can be exploited, producing negative impact on confidentiality, integrity, or availability [25]. Building on standard risk assessment guidance, risk can be framed as a function of the likelihood of a threat event, and the potential adverse impact should it occur [26]. This distinction motivates the use of complementary scoring signals (severity vs. exploitation likelihood) when remediation capacity is constrained.

2.2.3 Software Supply Chain Smells

The concept of a software supply chain smell extends the general notion of a code smell to the software supply chain context. Liu et al. [3] define a software supply chain smell as a package that matches specific patterns indicating potential security issues, either currently or in the future. In their approach, these patterns are detected using information obtained from project dependencies, package registries, and source code repositories. More recent work by Schmid et al. [27] similarly characterises software supply chain smells as patterns affecting packages in a dependency tree that may indicate present or future security concerns.

A software supply chain smell should therefore not be interpreted as equivalent to a vulnerability. A vulnerability represents an exploitable flaw, whereas a software supply chain smell acts primarily as an indicator of a potentially undesirable security condition that warrants further analysis. Consequently, the presence of a smell does not necessarily imply that the affected package contains an exploitable vulnerability. Examples identified in the literature include inadequate source traceability, missing provenance information, weak dependency constraints, deprecated packages, and governance-related characteristics. The literature review presented in Chapter 3 systematically analyses these patterns and provides the evidence used to define the consolidated smell catalogue adopted in this dissertation.

2.3 Software Supply Chain Risks

2.3.1 OWASP Top 10:2025

Software supply chain risk is explicitly captured in the OWASP Top 10:2025 as A03:2025 Software Supply Chain Failures, defined as breakdowns or compromises in the processes of building, distributing, or updating software, frequently driven by vulnerabilities or malicious modifications in third-party code, tools, or dependencies [28].

OWASP emphasises that this category has expanded beyond “known vulnerable components” to encompass failures across the broader ecosystem (including transitive dependencies, developer tooling, and CI/CD) and notes its high perceived criticality (top-ranked in the community survey) despite the practical difficulty of detection and limited mapping to public CVEs. Concretely, OWASP highlights that organisations are likely vulnerable when they do not systematically track component versions (including nested dependencies), do not perform continuous vulnerability monitoring, lack robust change management across the supply chain (e.g., IDEs/extensions, repositories, artefact stores), fail to harden the pipeline with least privilege and separation of duties, rely on untrusted sources, and operate CI/CD pipelines with weaker security than production systems [28].

To mitigate these failures, OWASP recommends establishing rigorous patch and dependency management grounded in centrally managed SBOMs, continuous inventory and vulnerability monitoring (e.g., CVE/NVD/OSV), preferential use of trusted and signed components, timely risk-based upgrading, staged rollouts, and systematic hardening of repositories, developer workstations, build systems, and artefacts (including MFA, IAM lockdown, signed builds, provenance, and immutable builds) [28].

2.3.2 Analysis and Detection Tools

Software Composition Analysis (SCA) tools support the identification of third-party components and known vulnerabilities in software dependencies. Within the npm ecosystem, npm audit provides a native vulnerability-analysis mechanism by submitting information about the dependency tree to the configured registry and returning known vulnerabilities together with severity and remediation information [29]. The audit mechanism requires dependency information derived from the project and its lockfile and can produce machine-readable JSON output, making it suitable for integration into automated analysis pipelines. However, its scope is primarily known-vulnerability detection rather than the broader identification of supply chain smells considered in this dissertation.

Dirty-Waters is a command-line analysis tool proposed by Liu et al. specifically for detecting software supply chain smells [3]. Rather than focusing exclusively on known CVEs, it analyses package and repository information to identify patterns related to source-code accessibility, release traceability, package deprecation and forks, code signatures, provenance, and

dependency aliases. This makes it particularly relevant to the present work because it provides an existing implementation for a subset of the smells identified in the literature, while leaving other dependency-configuration, source-usage, governance, and resolution-related smells to complementary detection mechanisms. In the proposed prototype, Dirty-Waters is therefore treated as one analysis source rather than as the complete detection solution. Its results are normalised together with findings obtained from custom detectors before contextual enrichment and SSSS scoring. The detailed integration mechanism is presented later in Chapters 4 and 5.

Source-level dependency usage requires a different form of analysis. Knip is a static-analysis tool for JavaScript and TypeScript projects that builds an internal representation from application entry points and source imports to identify unused files, exports, and dependencies. It also compares observed usage against package declarations and can identify dependencies that are referenced by the project but not declared in package.json [30]. Consequently, Knip provides information relevant to the Unused Dependency and Missing Dependency smells identified in the literature.

Known vulnerability information can also be enriched through security advisory databases. GitHub provides a Global Security Advisory REST API through which public advisories can be queried using GHSA identifiers and filtered by ecosystem, package, vulnerability identifiers, or severity. Advisory records expose information such as publication dates, affected package/version ranges, severity, CVSS information, and associated CVE identifiers [31]. Such metadata is useful when vulnerability presence alone is insufficient and temporal information, such as how long an advisory has been publicly known, is required for contextual prioritisation. In this dissertation, advisory metadata complements the results returned by npm audit. The implementation uses it to obtain publication information for identified GHSA, which contributes to the persistence component of the vulnerability dimension of the SSSS. The concrete enrichment and failure-handling mechanisms are described in Chapter 5.

The npm Registry also exposes package metadata required for analyses that cannot be derived exclusively from a repository snapshot, including published versions, maintainers, contributors, deprecation information, and package manifests [32]. In the prototype, registry metadata is therefore used as complementary evidence for governance-related smells, PeerSpin verification, and package responsiveness. The specific queries and decision rules remain implementation concerns and are described in Chapter 5.

2.3.3 Supply Chain Transparency and SBOMs (OWASP CycloneDX)

Managing supply chain risk also requires visibility into “what is in the software”, commonly operationalised through Software Bills of Materials (SBOMs).

OWASP CycloneDX is a full-stack Bill of Materials standard designed to support supply chain capabilities for cyber risk reduction and is published as an Ecma International standard (ECMA-424) [33], [34]. CycloneDX, standardised as ECMA-424, defines the syntax and semantics of a

machine-readable (JSON) Bill of Materials that records a detailed inventory of software and hardware components, services, dependencies, vulnerabilities, and compositions. It supports the inclusion of metadata, annotations, external references, and lifecycle/formulation context to enable exchange across tools and stakeholders for improved supply-chain transparency and cybersecurity assurance [33].

2.4 Vulnerabilities Prioritisation and Scoring

Vulnerability prioritisation requires more than listing detected issues. In practice, development teams often have limited remediation capacity and must decide which findings to address first. For this reason, prioritisation models are relevant to this dissertation because they provide conceptual references for distinguishing severity, exploitation likelihood, contextual impact, and remediation urgency. Although this work does not aim to score vulnerabilities directly, these models provide a useful foundation for the later design of the SSSS, where software supply chain smell instances are ranked according to both their intrinsic severity and their surrounding dependency context.

2.4.1 Scoring and Prioritisation Models (CVSS and EPSS)

The Common Vulnerability Scoring System (CVSS) is widely used to communicate vulnerability severity through a numerical score and an associated qualitative rating. In CVSS v4.0, the scoring model is organised into four metric groups: Base, Threat, Environmental, and Supplemental [35], [36]. The Base metric group represents the intrinsic characteristics of a vulnerability that are constant over time and across user environments. The Threat metric group captures threat-related characteristics that may change over time, while the Environmental metric group represents characteristics that are specific to a particular consumer environment, such as the presence of mitigations or the criticality of the affected system. Finally, the Supplemental metric group provides additional contextual information that may support interpretation but does not directly replace the main severity score [36].

CVSS is relevant to this dissertation for two main reasons. First, it establishes a clear distinction between intrinsic characteristics and contextual modifiers. This distinction is useful for the proposed SSSS because a software supply chain smell can have a baseline severity based on the smell type, while its final priority can be adjusted based on contextual factors such as production reachability, dependency depth, known vulnerabilities, and package responsiveness. Second, CVSS provides a qualitative vocabulary that is already familiar in vulnerability management. FIRST.org defines the CVSS qualitative scale as None = 0.0, Low = 0.1 - 3.9, Medium = 4.0 - 6.9, High = 7.0 - 8.9, and Critical = 9.0 - 10.0 [36]. Although the SSSS does not reuse the CVSS formula, this qualitative vocabulary is useful for communicating prioritisation results to developers in an interpretable way.

However, CVSS should not be interpreted as a complete risk score. NIST highlights that CVSS is designed to provide a qualitative measure of vulnerability severity, rather than a full measure of risk [35]. This limitation is important because remediation priority depends not only on how severe a vulnerability is in technical terms, but also on how likely it is to be exploited, where it appears in the dependency graph, whether it affects production code, and whether the affected dependency is actively maintained.

The Exploit Prediction Scoring System (EPSS) complements CVSS by focusing on exploitation likelihood rather than technical severity. EPSS is a data-driven model that estimates the probability that a published CVE will be exploited in the wild within the next 30 days, expressing this likelihood as a value between 0 and 1 [37]. Therefore, EPSS is useful alongside severity-based approaches because it helps distinguish vulnerabilities that are technically severe from vulnerabilities that are more likely to be exploited soon.

In the context of the dissertation, CVSS and EPSS are considered conceptual references for vulnerability prioritisation rather than direct inputs to the implemented SSSS. CVSS informs the distinction between intrinsic severity and contextual factors and provides a familiar qualitative severity vocabulary, while EPSS illustrates how exploitation likelihood can complement technical severity. The current prototype does not directly incorporate CVSS numerical scores or EPSS probabilities into the SSSS calculation. Exploitation-likelihood information is therefore considered a possible extension for future work.

2.4.2 OWASP Risk Rating Methodology

The OWASP Risk Rating Methodology provides an additional reference for prioritisation because it frames risk as a combination of likelihood and impact [38]. Unlike CVSS, which is focused on standardised vulnerability severity, the OWASP methodology is explicitly intended to support contextual risk estimation and adaptation to a specific organisation, application, or system environment. This makes it especially relevant to this dissertation, since software supply chain smells are not equally urgent in every context. Their practical importance depends on factors such as whether the affected dependency is used in production, whether it is direct or transitive, whether it is associated with known vulnerabilities, and whether it weakens source traceability, artefact integrity, or build provenance.

OWASP structures risk estimation into a sequence of steps: identifying the risk, estimating likelihood, estimating impact, determining severity, deciding what to fix, and customising the model according to the organisation or application context. The general model can be expressed as: **Risk = Likelihood × Impact** [38].

In this methodology, likelihood is estimated through two groups of factors: threat agent factors and vulnerability factors. Threat agent factors describe the characteristics of the potential attacker or attacker group, such as skill level, motive, opportunity, and size. Vulnerability factors describe how likely it is that the vulnerability can be discovered and exploited, including ease of discovery, ease of exploit, awareness, and intrusion detection [38]. These factors are relevant

because the probability of exploitation depends not only on the existence of a weakness, but also on how accessible, known, and exploitable it is.

Impact is also decomposed into technical impact and business impact. Technical impact captures the direct effect on security properties, such as loss of confidentiality, integrity, availability, and accountability. Business impact captures consequences for the organisation or stakeholders, such as financial damage, reputation damage, non-compliance, and privacy violations [38]. This separation is important because two technically similar issues may have different business consequences depending on the affected system, data, users, or regulatory context.

For the proposed SSSS, the OWASP Risk Rating Methodology is not adopted as a formula to be reused directly. Instead, it informs the design principle that prioritisation should combine intrinsic severity with contextual modifiers. A software supply chain smell may be intrinsically relevant, but its remediation priority should increase when it affects production-reachable dependencies, co-occurs with known vulnerabilities, weakens artefact authenticity or integrity, or appears in a dependency that is difficult to update. On the contrary, a smell may receive a lower priority when its impact is indirect, limited to development-only dependencies, or strongly dependent on additional contextual evidence.

Therefore, CVSS, EPSS, and OWASP Risk Rating play complementary roles in this dissertation. CVSS contributes a standardised severity vocabulary, EPSS contributes exploitation-likelihood information when known vulnerabilities are present, and OWASP Risk Rating contributes the broader principle that prioritisation should be context-sensitive and based on both likelihood and impact.

2.5 Software Dependency Visualisation and Graphs

2.5.1 Dependency Graphs

Software dependencies can naturally be represented as a directed graph in which nodes represent packages or components and edges represent dependency relationships. In the npm ecosystem, this representation is particularly useful because applications commonly contain both direct dependencies declared by the root project and transitive dependencies introduced through other packages. Modern npm lockfiles contain sufficient information about the resolved package tree to reconstruct these relationships without relying exclusively on the high-level declarations present in package.json [21].

Graph representations preserve structural context that is lost when findings are shown only as independent lists. For software supply chain analysis, the position of an affected package in the graph can indicate whether it is direct or transitive, identify the packages through which it is introduced, and support the contextualisation of multiple findings affecting related

dependencies. This structural information is also relevant to the SSSS proposed in this dissertation, where dependency depth and production reachability contribute to prioritisation.

2.5.2 Security Graph Visualisation with Cytoscape.js

Cytoscape.js is an open-source JavaScript graph-theory library designed for graph analysis and interactive network visualisation. It provides a graph model together with browser-based rendering, layouts, styling, viewport manipulation, zooming, panning, selection, and event handling, allowing graph data to be incorporated into interactive web applications [39]. The library supports directed and undirected graphs, compound graphs, and several graph traversal and manipulation operations. Franz et al. describe Cytoscape.js as a general-purpose library for the analysis and visualisation of network data [39].

These capabilities are compatible with the requirements of the visualisation proposed in this dissertation. A dependency package can be represented as a node, dependency relationships as edges, and analysis attributes such as smell presence and severity can be encoded through node styling and interaction. Cytoscape.js therefore provides the rendering and interaction layer used by the prototype, while the decision to adopt a node-link representation and the specific severity-oriented visual encodings are justified by the literature findings and design decisions presented in Chapters 3 and 4.

2.6 Summary

This chapter established the conceptual and technical background required for the remainder of the dissertation. It characterised the React and npm ecosystem, distinguished software supply chain smells from vulnerabilities and broader risk concepts and introduced the analysis sources and prioritisation models relevant to the proposed solution. It also presented software supply chain transparency concepts and graph-based dependency visualisation, providing the foundations for the following chapters.

3 Literature Review

This chapter presents the literature review that supports this dissertation. The review was designed to identify and synthesise existing research on software supply chain smells, dependency management issues, severity assessment, and dependency visualisation in the context of JavaScript, npm, and React applications.

The main literature review was conducted as a SLR, following a predefined protocol aligned with the PRISMA framework. This protocol defined the research questions, data sources, search queries, eligibility criteria, and data collection process used to identify and select the initial set of studies. The systematic search covered publications available up to the end of December 2025.

Given the rapid evolution of research on software supply chain security and dependency management, the review was complemented with a forward snowballing follow-up conducted between January and the end of April 2026. For this follow-up, Google Scholar was used to identify newer studies citing the articles included in the original review. Its purpose was not to replace the systematic process, but to update and strengthen the evidence base by identifying recent studies that could refine, extend, or challenge the original findings.

The chapter first describes the research design and approach, research questions, selected data sources, the search queries, and the eligibility criteria. It then presents the data collection process and the initial set of studies included in the SLR. The results are subsequently synthesised according to each research question, followed by a dedicated subsection describing the forward snowballing process and its implications for the smell catalogue, the severity scoring rationale, and the visualisation requirements. The chapter concludes with a summary of the main findings and their implications for the proposed solution.

3.1 Research Design and Approach

As introduced in Section 1.3, the literature review was conducted as an SLR aligned with the PRISMA framework. The review followed a predefined protocol specifying the research questions, search strings, data sources, inclusion and exclusion criteria, and data extraction procedures. PRISMA was used to support transparent and reproducible reporting of the study identification, screening, eligibility assessment, and inclusion process [14].

The review focuses on three main concepts: (i) software supply chain smells and risks related to dependencies, (ii) methods for assessing the severity or impact of such smells and (iii) visualisation techniques for software dependencies and quality issues. The research questions (RQ1–RQ3) are designed to capture these concepts and to connect them to the specific context of React applications. For each concept, relevant data were extracted, including the types of

smells or associated risks, the metrics or models used to assess severity, and the visualisation approaches applied.

In addition to the initial SLR, a complementary forward snowballing [13] follow-up was performed after the original review's due date. Forward snowballing was used to identify newer publications that cited one or more of the studies included in the systematic review. This step was conducted through Google Scholar between January and April 2026 and was intended to update the review with recent evidence relevant to the research questions.

3.2 Research Questions

The research questions are central to structuring both the literature review and the overall dissertation. They translate the general aim of improving developers' ability to understand and manage software supply chain smells in React applications. For that, the research questions and their goals below were defined:

RQ1: What types of software supply chain smells can be systematically identified and defined in React applications?

- **Goal:** To identify the different types of supply chain smells and the approaches for identifying them, with particular emphasis on their potential problems in the React ecosystem.

RQ2: How can a set of software supply chain smells, specific to React applications, be operationalised in a 'Smell Severity Scoring' system to effectively prioritise remediation actions?

- **Goal:** To analyse existing methods, metrics, and models used to assess the severity and impact of supply chain smells, and to use these insights to develop an SSSS tailored to React applications.

RQ3: What information and contextual cues can an SSSS-based dependency graph visualisation provide for identifying and prioritising software supply chain smells, compared with existing list-based analysis tools?

- **Goal:** To examine the information and contextual cues provided by an SSSS-based dependency graph visualisation for identifying and prioritising software supply chain smells, compared with existing list-based analysis tools.

3.3 Data Sources

To identify relevant data, two digital literature sources were used, as shown in Table 4. IEEE Xplore was selected as a specialised source for peer-reviewed research in software engineering and computer science, including conference proceedings and journal articles. B-On was used as an aggregated search platform providing access to multiple scientific databases and publishers through a single interface.

Table 4 - Selected databases

Database	Uniform Resource Locator (URL)
IEEE Xplore	https://ieeexplore.ieee.org/
B-On	https://research.ebsco.com/

3.4 Search Queries

The search queries were constructed to reflect the three main concepts of the research: (i) software supply chain smells and risks related to dependencies, (ii) severity, impact, or risk scoring and (iii) visualisation of software dependencies and quality issues.

For each concept, combinations of keywords and synonyms were defined, using logical operators (AND, OR) and wildcards (e.g., "smell*", "vulnerabilit*") to capture variations in terminology. Table 5 summarises the search queries used for each concept.

Table 5 - Search queries

Concept	Search Query
Software supply chain smells and risks related to dependencies	("software supply chain" OR "dependency management" OR "third-party libraries" OR "open-source libraries") AND ("smell*" OR "vulnerabilit*" OR "attack*" OR "dependenc*") AND ("React" OR "JavaScript" OR "npm" OR "component-based")
Severity, impact, and risk scoring of smells and dependency issues	("software supply chain" OR "dependency management" OR "open-source libraries") AND ("smell*" OR "vulnerabilit*" OR "dependenc*") AND ("severity" OR "impact" OR "risk scor*" OR "risk assessment" OR "prioritization" OR "prioritisation" OR "ranking") AND ("React" OR "JavaScript" OR "npm" OR "component-based")
Visualisation of software dependencies and quality issues	("software visualization" OR "software visualisation" OR "code quality" OR "code structure") AND ("dependency graph" OR "node-link diagram" OR "visual analytics" OR "graph visualization" OR "graph visualisation" OR "visually analyzing" OR "visually analysing")

3.5 Eligibility Criteria

Clear inclusion and exclusion criteria are essential to ensure that the selected studies are relevant, recent, and aligned with the objectives of the project. The criteria guide the screening and eligibility phases of the PRISMA process, helping to filter out work that is outside the scope of software supply chain smells, dependency management, severity assessment, or visualisation in the context of software systems.

3.5.1 Inclusion Criteria

Table 6 presents the inclusion criteria, which focus on studies that address supply chain risks, software maintainability or security issues, severity or risk assessment methods, and visualisation techniques, restricted to publications written in English and published in the last five years.

Table 6 - Inclusion criteria

ID	Criteria
CI1	Studies that discuss software supply chain smells, dependency management issues, or software supply chain risks.
CI2	Studies that address software maintainability or security aspects related to dependencies or component-based web applications.
CI3	Studies that propose or evaluate metrics, models, or methods to assess the severity, impact, or risk of software smells or dependency issues.
CI4	Studies that explore or apply software visualisation techniques (e.g., dependency graphs, visual analytics, node-link diagrams) for understanding software dependencies or quality issues.
CI5	Journal articles, conference papers, systematic literature reviews, relevant master's or doctoral theses, and scholarly preprints published within the last 5 years.
CI6	Studies written in English.

3.5.2 Exclusion Criteria

Table 7 lists the exclusion criteria, which remove studies unrelated to software supply chains, focused solely on code-level smells without dependency aspects, lacking methodological or empirical contributions, or for which the full text is not available.

Table 7 - Exclusion criteria

ID	Criteria
CE1	Studies unrelated to software supply chains or dependency management in npm ecosystem (e.g., manufacturing supply chains).
CE2	Studies focusing exclusively on code-level smells (e.g., long method) without addressing dependency or supply chain aspects.
CE3	Studies not providing empirical, conceptual, or methodological contributions (e.g., tutorials, blogs, presentations).
CE4	Studies for which full text is not available.

3.6 Data Collection Process

Figure 1 presents the PRISMA flow diagram summarising the identification and selection process for this SLR. A total of 168 records were retrieved from B-On (n = 132) and IEEE Xplore (n = 36). Prior to screening, duplicates were removed (n = 51), and a further set of records was excluded based on title relevance ("other reasons", n = 47), resulting in 70 records screened. During full-text retrieval, 6 reports could not be obtained, and the remaining 64 reports were assessed for eligibility. Of these, 46 were excluded for not meeting the predefined inclusion and exclusion criteria, leading to a final set of 18 studies included in the review.

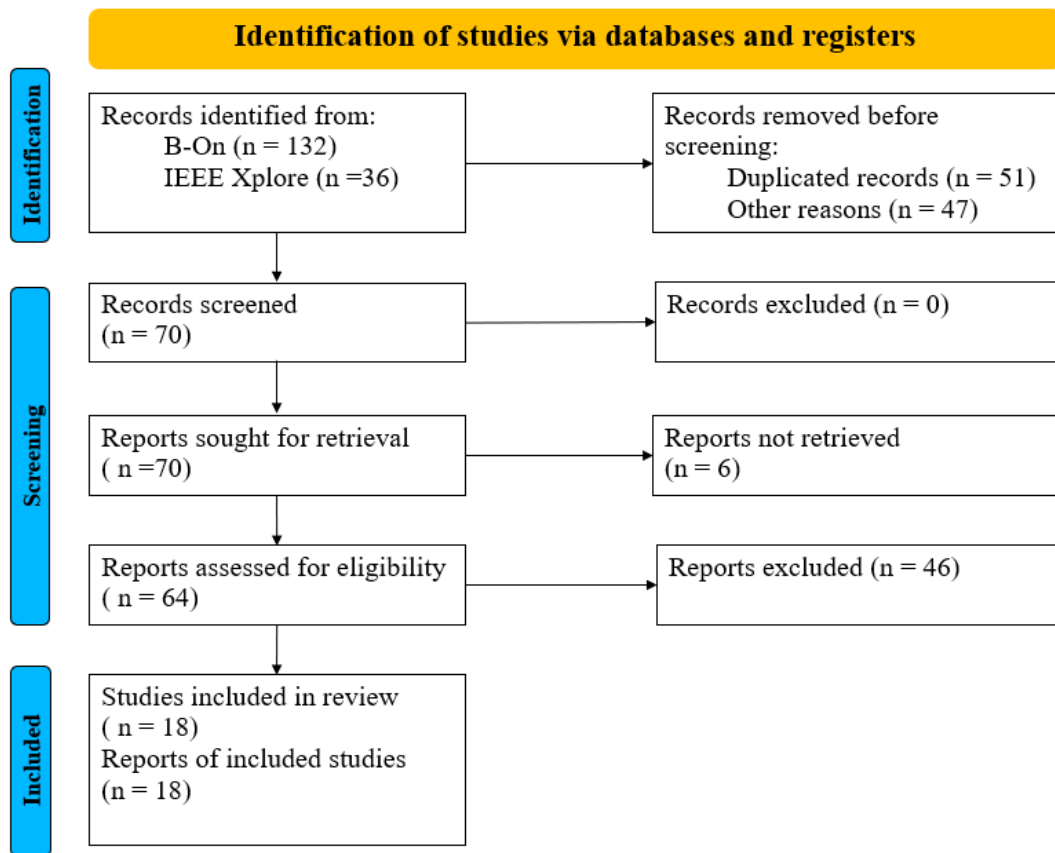


Figure 1 - PRISMA Flow diagram for the data collection process

After the collection process, the reports included in the review are presented in Table 8.

Table 8 - List of reports included in this review

ID	Title	Publication type	RQ	Reference
1	What are Weak Links in the npm Supply Chain?	Conference paper	1	[2]
2	Dirty-Waters: Detecting Software Supply Chain Smells	Conference paper	1	[3]
3	Visually Analyzing the Structure and Code Quality of Component-based Web Applications	Conference paper	3	[4]
4	Dependency Smells in JavaScript Projects	Journal article	1	[40]
5	A Comprehensive Study on the Impact of Vulnerable Dependencies on Open-Source Software	Conference paper	2	[41]
6	How Do Developers Follow Security-Relevant Best Practices When Using NPM Packages?	Conference paper	1	[42]
7	Supply Chain Risk Analysis Via SBOM Data Enrichment	Conference paper	1	[43]
8	Understanding and Detecting Peer Dependency Resolving Loop in npm Ecosystem	Conference paper	1	[44]
9	Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem	Conference paper	1	[45]
10	Investigating the Resolution of Vulnerable Dependencies with Dependabot Security Updates	Conference paper	1	[46]
11	Characterising Contributions that Coincide with Vulnerability Mitigation in NPM Libraries	Conference paper	1,2	[47]
12	Pinning Is Futile: You Need More Than Local Dependency Versioning to Defend against Supply Chain Attacks	Journal article / FSE paper	2	[48]
13	Dependency Practices for Vulnerability Mitigation	Scholarly preprint	2	[49]
14	Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM	Conference paper	2	[50]

Table 8 (continued)

ID	Title	Publication type	RQ	Reference
15	What's in a URL? An Analysis of Hardcoded URLs in npm Packages	Workshop paper	1	[51]
16	Dependency Management Practices for the npm Software Ecosystem	Doctoral thesis	2	[52]
17	Jack-in-the-box: An Empirical Study of JavaScript Bundling on the Web and its Security Implications	Conference paper	2	[53]
18	Towards Visual Analytics Dashboards for Provenance-driven Static Application Security Testing	Conference paper	3	[54]

3.7 Research and Results

3.7.1 RQ1: What types of software supply chain smells can be systematically identified and defined in React applications?

Existing work on JavaScript and npm ecosystems has identified several categories of software supply chain-related issues and how they can be detected, which are directly relevant to React applications.

At the dependency-configuration level, versioning and constraint choices are typically interpreted through Semantic Versioning (SemVer), which is widely treated as a de-facto standard in the npm ecosystem, although it is not consistently followed in practice [40]. In this context, Jafari et al. [40] define a catalogue of dependency smells that affect how third-party packages are fetched and updated, and that are associated with security vulnerabilities, breaking changes, and maintenance problems. The identified smells are:

- **Pinned Dependency:** Using a fixed version (e.g.: “jquery”: “3.4.1”) of a dependency prevents the package from automatically receiving bug fixes or security patches and requires manual updates.
- **URL Dependency:** Fetching a dependency from a specific URL (e.g., a Git repository) instead of the package registry increases security risks and the probability of link inaccessibility.
- **Restrictive Constraint:** Only updating dependency to patch releases (e.g.: “jest”: “<=25.0.0”) will result in not receiving compatible features and some features.
- **Permissive Constraint:** Allowing a dependency to update to new major versions (e.g.: “jest”: “>=25.0.0”) introduces a significantly higher risk of breaking changes.

- **No Package-Lock:** Not including a package-lock file in the source repository removes any guarantee of consistent installations across environments.
- **Unused Dependency:** Installing unused dependency packages will lead to an unnecessarily bloated dependency folder.
- **Missing Dependency:** Omitting a needed dependency from package.json can lead to code breakage or ambiguous dependencies.

Peer dependencies introduce a distinct, React-relevant failure mode during dependency installation and resolution, because packages that act as ecosystem “plugins” (e.g., react-dom with respect to react) assume that compatible peer versions are available elsewhere in the dependency tree [44]. Wang et al. [44] have recently shown that peer dependency conflicts can trigger a peer dependency resolving loop (PeerSpin), in which npm repeatedly replaces nodes while attempting to satisfy incompatible peer constraints, potentially resulting in non-termination, resource exhaustion, and installation failure. The same study operationalises detection through pattern-based analysis grounded in npm’s node-replacement semantics and, at ecosystem scale, reports thousands of affected package versions while providing tooling to automatically flag these problematic configurations.

Beyond registry-versus-URL sourcing decisions, supply-chain fragility may also arise from URLs embedded directly in published package artefacts (i.e., hardcoded URL strings in source files) [51]. In a large-scale measurement of npm packages, Wyss et al. [51] find that URL usage in code is widespread and is frequently associated with risky practices, including substantial use of unencrypted HTTP and occasional plaintext credentials. They further report evidence of link decay and identify packages referencing domains that appear past their expiration date, indicating that hardcoded URLs can degrade both security and functionality over time and may expose packages to domain takeover scenarios, where re-registration enables an attacker to control the content served by referenced URLs and thereby influence package behaviour.

Complementing these dependency-configuration smells, Liu et al. [3] introduce the notion of software supply chain smells as recurring “specific patterns” in package registries and source repositories that expose projects to underlying supply chain risks. In their Dirty-Waters tool, they define a set of smells centred on transparency and provenance of third-party packages:

- **Inaccessible Source Code Link:** The package metadata does not provide a valid link to the source code repository, making it difficult to audit changes or verify the integrity of the dependency.
- **Inaccessible Release Tag:** It is not possible to trace the exact source code corresponding to a released version.
- **Using Deprecated Package:** The package is marked as deprecated or the repository is archived, signalling that it is no longer actively maintained and may have unresolved vulnerabilities.

- **Using Forked Package:** The published package is provided by a forked repository rather than the original project, increasing the risk of consuming code that diverges from the upstream repository.
- **Missing Provenance Information:** The package lacks signed provenance or build attestations, reducing confidence that the artefact has not been tampered with.

At the governance and ecosystem level, Zahan et al. [2] identify a complementary set of weak link signals in the npm supply chain, which can be interpreted as maintainer and project governance-related supply chain smells. They identified:

- **Expired Maintainer Domain:** Maintainer's domain is expired and does not have 2FA authentication set up on their account, enabling attackers to re-register the domain and hijack those accounts.
- **Packages with Install Scripts:** Dependencies that execute installation scripts increase the risk of arbitrary code execution during package installation.
- **Unmaintained Packages:** Attackers can target packages that are more likely to be compromised and sneak in malware due to a lack of maintenance.
- **Too many Maintainers:** A package with too many maintainers will provide an attacker many targets to exploit account takeover and social engineering attacks. Zahan et al. [2], drawing on previous work, note that having more than 20 maintainers in a npm package is questionable. Their empirical analysis further showed that the top 1% of packages ranked by number of maintainers had an average of 32.4 maintainers, compared with an ecosystem-wide average of 1.7 maintainers. Based on this evidence, this dissertation operationalises the smell as a package having more than 20 maintainers.
- **Too many Contributors:** An attacker can sneak in malicious code, bypassing the maintainer's radar when a maintainer is responsible for many contributors. Rather than defining an absolute number of contributors, Zahan et al. [2] analyse the maintainer-to-contributor ratio. In the top 1% of packages presenting the most extreme ratios, the authors observed an average ratio of approximately 1 maintainer to 40 contributors, compared with an ecosystem-wide average ratio of 3 maintainers to 2 contributors. Based on this empirical observation, this dissertation operationalises the smell using a threshold of at least 40 contributors per maintainer.
- **Overloaded Maintainers:** Individual maintainers responsible for many packages may struggle to monitor and secure all of them effectively, potentially increasing the impact of a compromised maintainer account. Zahan et al. [2] analyse this signal using maintainer reach and other characteristics such as the number of owned packages, downstream dependents, dependency chains, and inactive packages. However, rather than establishing a validated absolute threshold for identifying an overloaded maintainer, the authors focus on extreme cases, including the top 1% of maintainers ranked by maintainer reach.

Together, these patterns indicate that software supply chain smells span at least three dimensions: dependency configuration and versioning, transparency and provenance of released artefacts, and maintainer and project governance.

Research on vulnerability propagation in npm shows that analysing only direct dependencies, or approximating transitive impact via simple reachability, can be misleading because it neglects npm-specific dependency resolution rules, resulting in wrongly resolved dependencies [45]. To address this, Liu et al. [45] construct a dependency-vulnerability knowledge graph (DVGraph) at ecosystem scale and propose an algorithm (DTRresolver) to statically resolve dependency trees and transitive propagation paths while respecting official npm resolution semantics. For RQ1, this work contributes an approach that operationalises fine-grained analysis of transitive vulnerability exposure under npm’s dependency resolution semantics, and it provides evidence that remediation strategies derived from correctly resolved dependency trees can be more effective than baseline tooling such as npm audit fix in reducing vulnerable dependencies.

Kabir et al. [42] examine how developers follow security-relevant best practices when consuming npm packages through a large-scale empirical study of 841 popular open-source JavaScript applications, assessing three commonly recommended practices: using npm audit/npm audit fix to identify and remediate known vulnerabilities, removing unused and duplicated dependencies (e.g., via depcheck and npm dedupe), and committing lockfiles to achieve deterministic installations. Their results indicate substantial gaps between recommendations and practice: only 32% of the analysed applications comprehensively lock dependency versions, and the examined tools report vulnerable, unused, and duplicated dependencies in a large fraction of projects. The authors further note that, even when such issues are detected, existing tools do not consistently resolve them automatically, and developers may discount tool outputs, partly due to perceived false positives. For RQ1, this highlights a key limitation of current approaches: detection mechanisms may be available, but their practical impact depends on tool effectiveness and sustained developer adoption in real projects.

Mohayeji et al. [46] studied developers’ receptivity to GitHub Dependabot security updates in engineered and actively maintained JavaScript projects, analysing 4,195 security updates across 978 repositories. They report that 57% of these security updates are merged and that, when developers do not merge a security update, they often address the associated vulnerability manually. Moreover, their analyses indicate that developers prioritise fixes by vulnerability severity and that bot fixes tend to occur substantially faster than manual fixes. The authors also discuss practical limitations observed in their dataset and replication, including noise/overload concerns and cases where a security alert may persist even after merging a Dependabot update, suggesting that automation improves responsiveness but does not remove the need for mechanisms that support risk understanding and prioritisation.

Rojpaisarnkit et al. [47] conducted a mixed-method empirical study to analyse npm GitHub projects affected by 554 vulnerability advisories, mining 4,699 “coinciding” contributions (2,159 pull requests and 2,547 issues) that are closed during the mitigation period. Their findings indicate that mitigation commonly overlaps with substantial concurrent maintenance work:

developers spend, on average, around 50% to 60% of the mitigation period attending to coinciding PRs and issues, and a majority of sampled coinciding contributions are not directly related to the vulnerability (often reflecting bugs, features, or other maintenance activities). For RQ1, this provides evidence that the practical effectiveness of detection and remediation approaches is shaped by competing workload and prioritisation pressures, which should be treated as a material constraint when designing developer-facing mechanisms for managing dependency risks in npm ecosystems.

Lemay and Katiyar [43] propose an SBOM-driven approach to supply-chain risk prioritisation in which an organisation's internal SBOM inventory (generated with Black Duck) is treated as a source of truth and subsequently enriched with maintenance-related metadata from upstream repositories - primarily GitHub, often identified via repository links associated with npm packages - to support organisational understanding of supply-chain exposure and potential susceptibility to XZ-type infiltration scenarios. They hypothesise that libraries exhibiting maintenance work that is disproportionate to the number of maintainers may be more amenable to infiltration, define corresponding maintenance-oriented features, and compute a risk metric to surface extreme cases for further inspection. Although no confirmed supply-chain attacks are found in their dataset, they report that the metric highlights libraries that could plausibly be susceptible under this hypothesis. The authors [43] also emphasise practical constraints that affect SBOM analytics in organisational settings, including the need to consolidate per-scan SBOM archives, confidentiality and regulatory restrictions on sharing dependency inventories, and incomplete maintenance metadata in SBOM tooling. In addition, they explicitly characterise their initial metric as naive and note that validation against ground truth remains challenging.

Despite the breadth of detectable signals, the literature remains comparatively thin on developer-facing visualisation strategies that help engineers interpret and act on risks in large dependency graphs. For example, Dirty-Waters delivers results via command-line execution and report artefacts (e.g., Markdown) and explicitly notes that the absence of a GUI may hinder adoption, while certain smells may require contextual triage due to false alarms. In parallel, practitioner feedback on weak-link signals indicates a preference for surfacing risk information within dependency graphs, rather than as detached lists, to enable rapid localisation of problematic packages in their transitive context. Empirical evidence on tool usage further suggests that, even when detection and automation mechanisms exist (e.g., npm audit/npm audit fix and Dependabot), their practical impact is mediated by usability, noise, and developer uptake, reinforcing the need for representations that make risk signals explainable and actionable in situ [2], [3], [42], [46].

Overall, existing research shows that software supply chain smells relevant to React span configuration-level practices (version constraints, lockfiles, and dependency declarations), registry/repository transparency and provenance, and maintainer/governance signals; more recent work extends this space to include resolution level pathologies (e.g., peer-dependency loops), code-level external references (e.g., hardcoded URLs), transitive vulnerability propagation under npm resolution semantics, bot-mediated remediation workflows,

maintainer workload constraints during mitigation, and SBOM-enriched risk analytics. However, these contributions are typically delivered as separate detectors, reports, or automation mechanisms, providing limited support for (i) comparing heterogeneous smell types on a common severity scale, (ii) prioritising remediation under realistic workload and adoption constraints, and (iii) communicating dependency risk in a way aligned with how React developers reason about dependency subgraphs and component structure. This gap motivates an integrated approach that operationalises React-oriented smell definitions, couples them with a principled severity scoring scheme, and exposes the resulting prioritisation through an explorable dependency and component visualisation [2], [3], [40], [42], [43], [44], [45], [46], [47], [51].

3.7.2 RQ2: How can a set of software supply chain smells, specific to React applications, be operationalised in an SSSS to effectively prioritise remediation actions?

Recent empirical studies on dependency-related vulnerabilities provide concrete guidance on which factors should drive the severity of software supply chain smells [41], [49], [52].

Kumar et al. [41] built the VODA (an SCA tool), and they analysed more than 50k releases from over 1k open-source projects across several languages, showing that vulnerable dependencies are transitive, and a critical vulnerability persists on average for over a year before being fixed and these results emphasise the importance of managing dependencies. For those that are eventually fixed, the average time to removal is around four to five months, and the number of direct and transitive dependencies is strongly correlated with the number of vulnerabilities.

Focusing on the npm ecosystem, Jafari et al. [49] study 458 vulnerabilities and 201,027 affected packages, and measure how long dependent packages take to adopt the fixed versions of their vulnerable dependencies. They find that, while vulnerable packages typically release a fix within a little over one year, dependents take on average more than six months to install that fix, and that restrictive update strategies (e.g., relying only on patch updates) cause developers to miss on average 30% of available vulnerability fixes. Their random forest model, trained on nine package attributes such as release frequency, age, number of dependencies and update strategy, achieves a ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) of 0.85 when classifying “fast responders” versus “slow responders” to vulnerability fixes, indicating that these attributes are good predictors of how quickly a package adopts the fixed version. This study analyses how a change in the top 5 dependency practices - namely, the use of a restrictive update strategy, package age, the use of a balanced update strategy, package release frequency, and the number of dependencies - increases or decreases the likelihood of a package being classified as a fast responder to vulnerabilities, and outlines a set of findings, summarised in Table 9, that are relevant to deriving heuristics for the SSSS.

Table 9 - Summary of findings from Jafari et al. [49] relevant to SSSS design

ID	Finding [49]
F1	Restrictive update strategies are associated with delayed vulnerability remediation.
F2	Non-restrictive (balanced or permissive) update strategies lead to faster adoption of fixes.
F3	Older packages tend to be slower in adopting security fixes.
F4	Packages with higher release frequency tend to respond more quickly to vulnerabilities.
F5	Packages with fewer dependencies tend to have shorter exposure to security vulnerabilities.

In his thesis, Jafari [52] shows that dependency smells such as pinned versions, restrictive and overly permissive constraints, missing or unused dependencies, and missing lock files are highly prevalent (80% of projects contain at least two distinct smells) and are perceived by developers as clearly harmful for security and maintainability.

Together, these results [41], [49], [52] imply that the severity of a supply chain smell should be weighted by the responsiveness of the affected package (e.g., release frequency, age, update strategy) and by whether the smell corresponds to a pattern that developers consistently associate with negative consequences.

Additional studies refine how to prioritise remediation by looking at which dependencies affect production behaviour and how update strategies influence the attack surface. Latendresse et al. [50] conducted an empirical study on 100 JavaScript projects and showed that production dependencies form only a small subset of all installed packages: many runtime dependencies are never shipped to production, while about 28.2 % of production dependencies are declared as development dependencies, often due to bundler behaviour or attempts to silence security warnings. They also report that 51.8 % of production dependencies are transitive, and that the majority of security alerts concern dependencies that never reach production, reinforcing the need to distinguish between production or non-production dependencies when evaluating software supply chain smells and prioritising remediation actions.

Rack et al. [53] show that bundling fundamentally alters how npm dependencies manifest in production, as third-party code is frequently merged into JavaScript bundles and delivered from the application's own servers, making it appear as first-party code. In their large-scale study, bundles are present in roughly 40% of websites, and the authors reidentify 1,051 vulnerabilities originating from 33 vulnerable npm packages within bundled code, including 17 critical and 59 high-severity issues. For a React-oriented SSSS, this evidence further motivates treating deployment reachability (e.g., whether a dependency is included in production bundles or build artefacts) as a first-class factor when prioritising remediation actions.

He et al. [48] quantify, via counterfactual simulations over historical npm resolutions, the effect of pinning direct dependencies and find that pinning increases the maintenance cost of outdated and vulnerable dependencies and, in large dependency graphs, can even increase the risk of exposure to malicious package updates. They argue that pinning direct dependencies should not be used as a positive security practice and recommend instead freezing entire dependency graphs with lock files or adopting coordinated, ecosystem level pinning strategies.

In combination, these findings [41], [42], [47], [48], [49], [50], [52] suggest that the SSSS for React applications should operationalise severity along at least four dimensions: (i) whether the smelled dependency is reachable in production and at what depth in the graph; (ii) the presence and persistence of known vulnerabilities; (iii) the responsiveness and update strategy of the affected package; and (iv) the empirical impact of specific dependency smells (e.g., pinned dependencies, missing lock files, bloated or unused dependencies), thereby enabling a principled prioritisation of remediation actions.

Operationally, the SSSS can compute a normalised sub-score per dimension and combine the resulting values through weighted aggregation to rank smell instances. The literature supports considering vulnerability severity and persistence as contextual prioritisation signals, while missing metadata should be handled explicitly to preserve reproducibility and explainability.

3.7.3 RQ3: What information and contextual cues can an SSSS-based dependency graph visualisation provide for identifying and prioritising software supply chain smells, compared with existing list-based analysis tools?

A representative example of a visualisation approach tailored to React is the work of Tamer et al. [4], who propose an interactive tool for analysing the structure and code quality of component-based JavaScript front-end applications. Their prototype focuses on React and its template language JavaScript Syntax Extension (JSX), and models the application as a component dependency graph, visualised as a node-link diagram with two analysis levels: an overview of dependencies between components and a detail level perspective of the dependencies of functions of a single component. The tool integrates this graph with a code-quality panel that displays static metrics such as lines of code, cyclomatic complexity, Halstead measures, and JSX-specific metrics (e.g., lines of JSX and maximum JSX nesting), using colour-coded thresholds and warning icons to highlight potential quality problems and refactoring opportunities. In a small user study with four participants, the authors report that the approach increases program comprehension (4.75 of 5), evaluation of code quality (4.75 of 5), and identification of refactoring opportunities (4.5 of 5). These findings suggest that a node-link visualisation for React applications, enriched with metric-based cues, is a promising way to communicate software quality issues, an idea that this dissertation extends by encoding software supply chain smells and their severity rather than only general code metrics.

Beyond React-specific visualisation, Schreiber et al. [54] provide complementary evidence from the security analysis domain by proposing a web-based visual analytics dashboard that unifies results from multiple static analysis tools, such as code smells, security warnings, and

dependency scanning (via OWASP Dependency Check), that can help developers obtain an overview and prioritise work. To make severity actionable at scale, they visualise hierarchies of projects/warnings/files using an Icicle chart and explicitly map warning severity to a colour scale, allowing teams to quickly identify where critical issues concentrate and potentially distribute remediation effort accordingly. Importantly, the paper also acknowledges practical obstacles common to static analysis adoption, such as false positives and the need for understandable, actionable results, and positions interactive dashboards to make results more usable.

Taken together, these findings suggest that interactive visualisation can support the identification and prioritisation of software-related issues by combining structural context with visual severity cues. Node-link representations preserve structural relationships between components and support localising affected elements within a wider dependency context, while severity-oriented visual cues can make priority differences more immediately visible. Overview and detail mechanisms can further provide additional information about individual findings without losing their structural context.

In the context of software supply chain smells, these characteristics support the use of a dependency graph enriched with SSSS-based severity information. However, the available literature does not establish that graph-based representations are inherently more effective than existing list-based analysis tools. The React-specific study by Tamer et al. [4] is based on a small participant sample, while Schreiber et al. [54] identifies further user-oriented evaluation as future work. Therefore, the literature primarily supports the selection of structural context, severity cues, and interactive exploration as relevant design mechanisms. The comparison between the information provided by the proposed prototype and existing analysis tools is subsequently examined through the artefact evaluation conducted in this dissertation.

3.7.4 Forward Snowballing

To complement the SLR, forward snowballing was conducted between January and the end of April 2026. The purpose of this step was to identify recent studies citing the papers included in the original review and to assess whether they introduced additional evidence relevant to the dissertation's research questions.

The 18 studies included in the SLR were used as the seed set. For each study, Google Scholar was used to inspect citing publications. Candidate studies were screened by title, abstract, and, when necessary, full text, using the same inclusion and exclusion criteria defined for the original review. Only studies that cited at least one paper from the seed set and contributed to RQ1, RQ2, or RQ3 were considered for inclusion.

This process resulted in the inclusion of one additional study, presented in Table 10.

Table 10 - Study included through forward snowballing

ID	Study identified through forward snowballing	Publication type	Studies cited	RQ relevance	Inclusion rationale
FS1	Schmid et al. [27], "Software Supply Chain Smells: Lightweight Analysis for Secure Dependency Management"	Scholarly preprint	[2], [3], [40]	RQ1, RQ2	Extends the concept of software supply chain smells, provides a practitioner-informed taxonomy, evaluates Dirty-Waters, and contributes evidence on perceived severity and ecosystem-level prevalence.

The included study was analysed according to its contribution to the three research questions, with particular emphasis on its implications for the smell catalogue and the later design of the SSSS.

For RQ1, Schmid et al. [27] extend the evidence base of the original review by presenting a practitioner-informed taxonomy of software supply chain smells. The authors define a software supply chain smell as a package in a dependency tree that matches specific patterns indicating potential security issues, either current or future. Their taxonomy includes nine smells:

- **No Source Code URL:** The package's metadata does not include a URL to the associated source code repository. This prevents consumers from inspecting the code that is included and executed, making it difficult to audit the package for vulnerabilities or malicious code.
- **Invalid Source Code URL:** The package's metadata provides a source code URL that is invalid or inaccessible, which similarly prevents source-level inspection and may indicate poor maintenance or, in some cases, an attempt to create a false sense of transparency.
- **Inaccessible Commit SHA/Release Tag:** The source repository of the package does not provide the commit SHA or release tag corresponding to the published package version. This weakens traceability, prevents reproducible analysis of the exact artefact, and complicates post-incident investigation.
- **Deprecated:** The package is marked as deprecated, either because it is no longer maintained or because known vulnerabilities exist, increasing the likelihood of unresolved security issues in dependent projects.
- **Fork:** The source repository associated with a package is a fork of another upstream repository. While forks may be legitimate, they introduce uncertainty about whether

the published package corresponds to the trusted upstream source or includes unauthorised changes.

- **No Code Signature:** The package release is not cryptographically signed, reducing assurance about the authenticity and integrity of the artefact.
- **Invalid Code Signature:** The package release has an invalid signature status, meaning that its authenticity cannot be reliably verified and that the artefact may have been modified, signed with an unexpected key, or affected by signing workflow problems.
- **Aliased:** The package installs one or more dependencies under alternative names, which can reduce transparency in dependency resolution and obscure the actual identity or source of the installed dependency.
- **No Provenance:** The package lacks provenance or build attestation metadata, preventing consumers from verifying how and where the artefact was built and whether it was produced from the claimed source code.

Together, these smells reinforce the relevance of traceability, authenticity, integrity, and provenance as key dimensions of software supply chain trust, complementing the dependency configuration, maintainer governance, and vulnerability-related smells identified in the original review.

These findings are particularly relevant for the smell catalogue because they extend several smell categories already identified in the original SLR with a more fine-grained taxonomy. For example, the original smell **Inaccessible Source Code Link** can be extended by distinguishing between **No Source Code URL**, where no repository link is provided in the package metadata, and **Invalid Source Code URL**, where the provided link exists but is invalid or inaccessible. Similarly, **Inaccessible Release Tag** can be extended through **Inaccessible Commit SHA/Release Tag**, which more explicitly captures the inability to trace a published package version back to the exact source code state from which it was produced. The original smells, **Using Deprecated Package** and **Using Forked Package**, are also reinforced and refined by the corresponding **Deprecated** and **Fork** smells. Finally, **Missing Provenance Information** can be extended with three more specific signals: **No Code Signature**, **Invalid Code Signature**, and **No Provenance**, which respectively capture the absence of cryptographic signing, the presence of an unverifiable or invalid signature, and the lack of build provenance or attestation metadata. In addition, the **Aliased** smell introduces a new dependency naming and resolution-related signal that was not explicitly captured in the original review and may therefore be considered as an additional catalogue extension.

For RQ2, Schmid et al. [27] provide relevant evidence because they evaluate the perceived severity of the proposed software supply chain smells through semi-structured interviews with eleven practitioners. According to the authors, eight participants had more than twenty years of professional experience, two had eighteen years, and one had around ten years. Nine worked in security, while one focused on quality assurance, and one was a development manager. Participants were asked to rate each smell according to its security impact, considering both maliciousness and vulnerability. Table 11 summarises the ratings reported by the authors. The

results show that six out of the nine smells received at least one critical rating, with **Invalid Code Signature** receiving the highest number of critical ratings. The authors also report high or critical ratings for **Invalid Source Code URL**, **No Source Code URL**, and **Inaccessible Release Tag**, while **Fork** was mostly rated as medium, **Aliased** received the highest number of no ratings, and **No Provenance** was mostly rated as low. These findings do not define a scoring model directly but provide practitioner-informed evidence that can be considered when defining baseline severity levels in the proposed SSSS.

Table 11 - Practitioner ratings of software supply chain smells. Adapted from Schmid et al. [27]

ID	Smell	L	M	H	C	N
1	No Source Code URL	1	2	5	3	0
2	Invalid Source Code URL	1	3	3	4	0
3	Inaccessible Release Tag	1	3	5	2	0
4	Deprecated	1	4	4	2	0
5	Fork	3	7	1	0	0
6	No Code Signature	2	2	4	2	1
7	Invalid Code Signature	1	0	1	8	1
8	Aliased	3	2	1	0	5
9	No Provenance	7	2	1	0	1

Note: **L** = low severity, **M** = medium, **H** = high, **C** = critical, **N** = no rating. The most frequent rating for each smell is shown in bold. Adapted from Schmid et al. [27].

For RQ3, the contribution is indirect, since Schmid et al. [27] do not propose or evaluate a dependency visualisation technique. Nevertheless, their emphasis on using software supply chain smells to support informed security decisions provides complementary motivation for presenting smell information in a clear, developer-facing form. Therefore, the direct evidence for visualisation remains primarily grounded in the studies discussed in RQ3.

3.7.5 Summary

The SLR, complemented by the forward snowballing process, shows that software supply chain smells relevant to React applications span at least three core dimensions: (i) dependency configuration and versioning practices, (ii) transparency, traceability, integrity, and provenance of released artefacts, and (iii) maintainer and project governance signals. In addition, the review identifies complementary ecosystem-level concerns, including resolution-level pathologies such as peer-dependency loops, code-level external references such as hardcoded URLs, transitive vulnerability propagation under npm resolution semantics, bot-mediated remediation workflows, maintainer workload constraints during vulnerability mitigation, and SBOM-enriched risk analytics. The forward snowballing study does not substantially change this

structure but extends the transparency and provenance dimension with more fine-grained smells related to source accessibility, release traceability, code signing, dependency aliasing, and build provenance. Table 12 summarises the smell dimensions and representative smells/signals identified across the systematic review and the forward snowballing process.

Table 12 - Summary of smell dimensions and representative smells/signals (RQ1)

Dimension	Representative smells / signals included in the review	Primary source(s) in the review
Dependency configuration and versioning	Pinned Dependency; URL Dependency; Restrictive Constraint; Permissive Constraint; No Package-Lock; Unused Dependency; Missing Dependency.	Jafari et al. [40]
Resolution-level pathologies (extension)	Peer dependency resolving loop (PeerSpin).	Wang et al. [44]
Code-level external references (extension)	Hardcoded URLs; risky practices (e.g., unencrypted HTTP, occasional plaintext credentials); link decay / expired domains as takeover exposure.	Wyss et al. [51]
Transparency, traceability, integrity, and provenance of released artefacts	Inaccessible Source Code Link; No Source Code URL; Invalid Source Code URL; Inaccessible Release Tag; Inaccessible Commit SHA/Release Tag; Using Deprecated Package; Deprecated; Using Forked Package; Fork; No Code Signature; Invalid Code Signature; Missing Provenance Information; No Provenance.	Liu et al. [3]; Schmid et al. [27]
Maintainer and project governance	Expired Maintainer Domain; Packages with Install Scripts; Unmaintained Packages; Too many Maintainers; Too many Contributors; Overloaded Maintainers.	Zahan et al. [2]
Ecosystem-scale risk analysis (extension)	Transitive vulnerability propagation under npm resolution semantics (DVGraph/DTRresolver); bot-mediated remediation workflows; maintainer workload constraints during mitigation; SBOM-enriched risk analytics.	RQ1 synthesis + cited studies in the review
Dependency identity and naming transparency	Aliased package/dependency.	Schmid et al. [27]

Empirical evidence suggests that a React-oriented SSSS should operationalise severity along at least four dimensions: (i) whether a smelled dependency is reachable in production and at what depth in the graph, (ii) the presence and persistence of known vulnerabilities, (iii) the responsiveness and update strategy of the affected package, and (iv) the empirical impact of specific dependency smells. Operationally, this can be implemented via normalised sub-scores per dimension combined through weighted aggregation, grounding vulnerability signals in established metrics (e.g., CVSS and, where available, EPSS), while handling missing metadata explicitly to keep prioritisation reproducible and explainable.

The forward snowballing study adds practitioner-informed evidence to this severity rationale. In particular, the ratings reported by Schmid et al. [27] suggest that smells related to invalid code signatures, invalid or missing source code URLs, and inaccessible release references are perceived as more severe than smells whose interpretation is more dependent on ecosystem practices, such as forks, aliases, or missing provenance metadata. These results should not be treated as a complete scoring model, but as supporting evidence for defining baseline severity levels and contextual modifiers in the proposed SSSS. Table 13 consolidates the SSSS dimensions and operational implications.

Table 13 - SSSS dimensions and operational implications (RQ2)

SSSS dimension	What it captures (operational interpretation)	Implementation notes stated in the review
Production reachability + depth	Whether the smelled dependency is reachable in production, and its depth in the dependency graph.	Treat reachability (e.g., inclusion in production bundles/build artefacts) as a first-class factor.
Presence + persistence of vulnerabilities	Whether known vulnerabilities affect the dependency, and how persistent they are over time.	Consider vulnerability severity and persistence as contextual prioritisation signals.
Responsiveness + update strategy	Package responsiveness (e.g., release frequency, age) and update strategy as a weight on severity.	Weight smell severity by responsiveness. Reflect that remediation overlaps with competing maintenance work.
Empirical impact and practitioner-perceived severity of specific smells	Severity informed by empirical evidence on negative consequences and practitioner ratings of smell criticality.	Use empirical studies on dependency practices and practitioner-informed ratings to define baseline severity levels.

Existing evidence supports the premise that interactive visualisation can communicate smells and severity and can provide additional structural and contextual information for their by embedding prioritisation cues into explorable representations (e.g., node-link dependency graphs and linked dashboard views). React-oriented visualisation work shows the value of an overview and detail interaction model with metric-based visual cues, while security-analysis dashboards show how integrating heterogeneous tool outputs and mapping severity into

hierarchical views (e.g., Icicle charts with colour-encoded severity) can help teams localise and prioritise issues at scale. At the same time, the literature highlights practical adoption obstacles (e.g., false positives and the need for actionable results) and remains limited in large-sample empirical validation, motivating user-study-based evaluation as future work.

4 Analysis and Design

This chapter presents the analysis and design of the proposed solution. It translates the concepts introduced in the background and the evidence gathered through the literature review into three main design artefacts: a software supply chain smell catalogue for React applications, an SSSS, and a prototype architecture for detecting, scoring, and visualising smell instances. The chapter, therefore, establishes the rationale, structure, and design decisions that support the implementation and evaluation of the proposed approach.

4.1 Smell Catalogue Design

This section presents the design rationale for the software supply chain smell catalogue adopted in this dissertation. The catalogue serves as the conceptual basis for the proposed solution, as it defines which dependency-related patterns are considered relevant, how overlapping terminology in the literature is consolidated, and how each smell can later be operationalised by the detection and scoring mechanisms. Accordingly, the smells identified in the literature were classified and consolidated into a consistent set of designations to be used throughout the remainder of the dissertation.

4.1.1 Define Criteria

The smell catalogue proposed in this dissertation is defined based on the evidence gathered from the systematic literature review and complemented by the forward snowballing process described in Chapter 3. The review showed that software supply chain smells relevant to React applications span multiple dimensions, including dependency configuration and versioning practices, transparency, traceability, integrity and provenance of released artefacts, and maintainer and project governance signals. In addition, the forward snowballing study introduced more fine-grained smells related to source code accessibility, release traceability, code signing, dependency aliasing, and build provenance, which are used to extend the original smell categories rather than replace them.

Accordingly, two related but distinct decisions are considered in the catalogue design. First, a candidate smell is retained in the literature-derived catalogue when it is explicitly reported in the reviewed literature, or clearly supported by the review synthesis, is relevant to the npm/React ecosystem, and is associated with negative security, maintainability, or software supply chain implications. Second, prototype operationalisation requires an additional feasibility criterion: the smell must be detectable through available artefacts or metadata using a sufficiently explicit and reproducible detection rule. Smells for which the available evidence does not provide adequate empirical support or a defensible detection criterion may therefore remain documented in the catalogue while being excluded from the subset implemented in the prototype.

4.1.2 Classify Smells

Following the criteria above, the smells identified in the literature were organised into a catalogue that combines consolidation and refinement. In particular, the smells initially introduced by Liu et al. [3] through Dirty-Waters are treated as the original formulation of the tool-supported smell taxonomy. However, since Schmid et al. [27] later refine and extend the same tool and taxonomy, this dissertation adopts the more recent and fine-grained designations proposed by Schmid et al. [27] as the final catalogue entries for transparency, traceability, code signing, and provenance-related smells.

Table 14 summarises the smell terms identified in the review, the article(s) from which they were drawn, the final designation adopted in this dissertation, and relevant observations about refinement or alignment between earlier and later formulations.

Table 14 - Classification and final designation adopted of software supply chain smells derived from the literature review

Smell mentioned in the review	Article(s)	Final designation adopted	Obs.
Pinned Dependency	Jafari et al. [40]	Pinned Dependency	-
URL Dependency	Jafari et al. [40]	URL Dependency	-
Restrictive Constraint	Jafari et al. [40]	Restrictive Constraint	-
Permissive Constraint	Jafari et al. [40]	Permissive Constraint	-
No Package-Lock	Jafari et al. [40]	No Package-Lock	-
Unused/Bloated Dependency	Jafari et al. [40]	Unused Dependency	-
Missing Dependency	Jafari et al. [40]	Missing Dependency	-
Peer Dependency Resolving Loop (PeerSpin)	Wang et al. [44]	Peer Dependency Resolving Loop (PeerSpin)	-
Hardcoded URLs	Wyss et al. [51]	Hardcoded URLs	Retained as a literature-derived smell but excluded from prototype operationalisation.
Inaccessible Source Code Link	Liu et al. [3]; Schmid et al. [27]	No Source Code URL; Invalid Source Code URL	Original Dirty-Waters smell refined into two more specific smells by Schmid et al. [27]

Table 14 (continued)

Smell mentioned in the review	Article(s)	Final designation adopted	Obs.
Inaccessible Release Tag	Liu et al. [3]; Schmid et al. [27]	Inaccessible Commit SHA/Release Tag	Original Dirty-Waters smell refined by Schmid et al. [27]
Using Deprecated Package	Liu et al. [3]; Schmid et al. [27]	Deprecated	Later designation adopted from Schmid et al. [27]
Using Forked Package	Liu et al. [3]; Schmid et al. [27]	Fork	Later designation adopted from Schmid et al. [27]
Missing Provenance Information	Liu et al. [3]; Schmid et al. [27]	No Code Signature; Invalid Code Signature; No Provenance	Original Dirty-Waters smell refined/extended into three more specific smells by Schmid et al. [27]
Aliased	Schmid et al. [27]	Aliased	-
Expired Maintainer Domain	Zahan et al. [2]	Expired Maintainer Domain	-
Packages with Install Scripts	Zahan et al. [2]	Install Script Execution	-
Unmaintained Packages	Zahan et al. [2]	Unmaintained Packages	Retained as a literature-derived smell but excluded from prototype operationalisation.
Too many Maintainers	Zahan et al. [2]	Too many Maintainers	-
Too many Contributors	Zahan et al. [2]	Too many Contributors	-
Overloaded Maintainers	Zahan et al. [2]	Overloaded Maintainer	Retained as a literature-derived smell but excluded from prototype operationalisation.

Three smells derived from the literature are retained in the catalogue but are not operationalised in the current prototype. Hardcoded URLs, as described by Wyss et al. [51], are distinct from URL Dependency because they refer to URLs embedded directly in package source code rather than dependencies declared through URL-based specifiers. Similarly, Unmaintained Package are distinct from Deprecated, since lack of maintenance and explicit package deprecation represent different conditions. Both smells are therefore retained for traceability to the literature but excluded from the implemented subset.

Overloaded Maintainer is also retained in the catalogue but excluded from prototype operationalisation for a different reason. Zahan et al. [2] characterise overloaded maintainers through several ecosystem-level factors, including maintainer reach, number of owned packages, downstream dependents, dependency chains, and inactive packages, but do not establish a validated threshold that clearly determines when a maintainer should be considered overloaded. Consequently, implementing an automatic detection rule would require assumptions that are not sufficiently supported by the available evidence. This smell is therefore excluded from the implemented subset and considered a candidate for future work.

Before assigning a baseline severity to each smell, the severity levels used in this dissertation are defined according to two main criteria: (i) the potential for direct exploitation or abuse, and (ii) the expected impact on security, maintainability, traceability, or dependency resolution. These levels represent the intrinsic severity of each smell type before applying contextual modifiers such as production reachability, dependency depth, known vulnerabilities, and package responsiveness. Table 15 presents the criteria used to interpret the baseline severity levels.

Table 15 - Baseline severity criteria for software supply chain smells

Severity level	Criteria
Critical	Smells with direct potential for package takeover, account compromise, or code execution. These smells represent the highest baseline severity because they can enable direct abuse of the software supply chain or compromise the integrity of the installation/build process.
High	Smells strongly associated with delayed remediation, loss of source or artefact integrity, weak traceability, severe dependency resolution failures, or reduced ability to verify the origin of a package. These smells may not always enable direct exploitation, but they significantly weaken supply chain security or maintainability.
Medium	Smells in this category do not have the same impact in every project. Their importance depends on how the affected dependency is used and on its role in the application. For example, more attention may be required when the dependency is used in production or when replacing or updating it is difficult.

Table 15 (continued)

Severity level	Criteria
Low	Smells with mostly indirect, cumulative, or limited impact. These smells may still reduce transparency, increase maintenance effort, or contribute to dependency hygiene problems, but they do not usually imply direct exploitation potential or an immediate integrity risk by themselves.

Based on these criteria, each smell selected for prototype operationalisation was assigned a baseline severity level. Smells retained in the literature-derived catalogue but excluded from the implemented subset, such as Overloaded Maintainer, are not assigned a baseline severity because they do not participate in the SSSS computation. The resulting classification is presented in Table 16.

Table 16 - Baseline severity classification of operationalised software supply chain smells

Final designation adopted	Proposed baseline severity	Description and baseline severity rationale
Pinned Dependency	Medium	A fixed dependency version can prevent automatic adoption of bug fixes or security patches, requiring manual updates and potentially delaying remediation [40], [48], [52].
URL Dependency	High	Direct URL dependencies can reduce the benefits provided by the npm ecosystem, complicate SemVer-based dependency management, and increase exposure to instability, security issues, and link inaccessibility [40].
Restrictive Constraint	High	Restrictive update strategies are associated with delayed vulnerability remediation and can prevent dependent projects from adopting available fixes in a timely manner [40], [49], [52].
Permissive Constraint	Medium	Permissive constraints increase the risk of breaking changes by allowing major version updates [40], [52].
No Package-Lock	Medium	Not including a package-lock file in the source repository removes any guarantee of consistent installations across environments [40], [52].
Unused Dependency	Low	The presence of installed dependency packages that are not used, leading to an unnecessarily bloated dependency folder [40], [52].

Table 16 (continued)

Final designation adopted	Proposed baseline severity	Description and baseline severity rationale
Missing Dependency	Medium	Omitting a required dependency from package.json can lead to code breakage or ambiguous dependencies [40], [52].
Peer Dependency Resolving Loop (PeerSpin)	High	Peer dependency conflicts can trigger repeated node replacement, non-termination, resource exhaustion, and installation failure during npm dependency resolution [44].
No Source Code URL	High	The absence of a source code URL prevents consumers from inspecting the code associated with the package and was frequently rated as high or critical by practitioners [27].
Invalid Source Code URL	High	An invalid or inaccessible source code URL prevents source-level inspection and may indicate poor maintenance or misleading transparency; it also received frequent high or critical ratings by practitioners [27].
Inaccessible Commit SHA/Release Tag	High	The inability to trace a released package to the exact source state weakens reproducibility, release traceability, and post-incident investigation; practitioner ratings were mostly high [27].
Deprecated	High	A package explicitly marked as deprecated may no longer receive fixes or may have been superseded, increasing the likelihood that known issues remain unresolved or that continued use requires additional maintenance attention [3], [27].
Fork	Medium	Forks may be legitimate, but they introduce uncertainty about divergence from the original upstream source; practitioner ratings were mostly medium [3], [27].
No Code Signature	High	The absence of a code signature reduces assurance about artefact authenticity and integrity; practitioner ratings were mixed, but the most frequent rating was high [27].
Invalid Code Signature	Critical	An invalid signature means artefact authenticity or integrity cannot be reliably verified; this smell received the highest number of critical ratings among the practitioners-assessed smells [27].

Table 16 (continued)

Final designation adopted	Proposed baseline severity	Description and baseline severity rationale
No Provenance	Low	Missing provenance limits verification of how and where the artefact was built, but practitioner ratings were mostly low, suggesting that its baseline severity is lower and more dependent on ecosystem adoption [27].
Aliased	Low	Dependency aliasing can obscure the real identity or source of a dependency, but the study reported several no-ratings and lower severity ratings [27].
Expired Maintainer Domain	Critical	An expired maintainer domain can enable account hijacking or package takeover, particularly when combined with weak account protection [2].
Install Script Execution	Critical	Install scripts allow code execution during dependency installation, creating direct abuse potential in the supply chain [2].
Too many Maintainers	Medium	A high number of maintainers expands the attack surface for account compromise and social engineering, but the risk depends on governance and account security practices [2].
Too many Contributors	Low	A high number of contributors can increase review and oversight difficulty, but this is a more indirect governance signal than maintainer-related smells [2].

4.2 Smell Severity Scoring System Design (SSSS)

This section presents the design of the SSSS, which is used to prioritise the software supply chain smell instances identified in React applications. While the smell catalogue defines the patterns identified as relevant from the literature, the SSSS is applied to the subset selected for prototype operationalisation and defines how detected smell instances are assessed and ranked according to their baseline severity and contextual factors. The section first summarises the severity and prioritisation models considered as design references, then defines the scoring dimensions used by the proposed system and finally presents the scoring formula and examples of its application.

4.2.1 Review Severity Models

The severity and prioritisation models used as conceptual references for the SSSS were introduced in Section 2.4. CVSS provides a widely adopted vocabulary for communicating vulnerability severity, EPSS complements severity with exploitation likelihood, and the OWASP Risk Rating Methodology frames risk as the combination of likelihood and impact in a specific application or organisational context.

In the SSSS, these models are not reused directly as vulnerability scoring mechanisms. Their role is instead conceptual. CVSS informs the distinction between intrinsic severity and contextual modifiers, EPSS informs the use of exploitation likelihood when a smell affects a dependency with known vulnerabilities, and OWASP Risk Rating supports the idea that prioritisation should be adapted to the context in which the issue appears. This is important because software supply chain smells are not necessarily vulnerabilities by themselves. Rather, they are recurring dependency-related patterns that may indicate current or future security, maintainability, traceability, or provenance risks.

Accordingly, the proposed SSSS is designed around three elements: (i) the smell catalogue and its baseline severities, (ii) contextual dimensions that adjust the urgency of each detected smell instance, and (iii) a scoring formula that converts these inputs into a numerical score and qualitative rating for developer-facing prioritisation.

Table 17 summarises the external scoring and prioritisation models considered as design references for the proposed SSSS, highlighting their main focus and how each model informs the definition of baseline severity, contextual modifiers, and developer-facing prioritisation.

Table 17 - Reference scoring and prioritisation models considered for the SSSS design

Model/ methodology	Main focus	Role in the proposed SSSS
CVSS	Communicates characteristics and severity of software vulnerabilities through metric groups such as Base, Threat, Environmental, and Supplemental [36].	Provides a conceptual distinction between intrinsic severity and contextual factors and informs the Low/Medium/High/Critical vocabulary. CVSS numerical scores are not used directly by the implemented SSSS.
EPSS	Estimates the probability that a published CVE will be exploited in the wild in the next 30 days [37].	Provides a conceptual reference for exploitation-likelihood-based prioritisation. EPSS probabilities are not operationalised in the current prototype and are considered a possible future extension.
OWASP Risk Rating Methodology	Estimates contextual risk by combining likelihood and impact. Likelihood is informed by threat agent and vulnerability factors, while impact is informed by technical and business impact factors [38].	Supports combining smell severity with contextual dimensions, reflecting that remediation priority depends on both the probability of exploitation or materialisation and the potential impact in the specific application context.

4.2.2 SSSS Dimensions

After defining the baseline severity of each smell type, the SSSS uses a set of scoring dimensions to calculate the prioritisation score of each detected smell instance. These dimensions combine the intrinsic severity of the smell with contextual factors that influence remediation urgency. This design follows the rationale established in the literature review: not all dependency smells have the same practical impact, and their priority depends on the type of smell, whether the affected dependency is reachable in production, whether it is associated with known vulnerabilities, and whether the affected package or dependency configuration may delay remediation. Therefore, the proposed SSSS combines four dimensions: empirical impact and practitioner-perceived severity of the smell, production reachability and dependency depth, vulnerability exposure and persistence, and responsiveness and update strategy. Table 18 presents the proposed dimensions and their weights.

Table 18 - Proposed SSSS dimensions and weights

SSSS dimension	Weight	What it captures	Literature-based rationale
Empirical impact and practitioner-perceived severity of specific smells	0.30	The intrinsic severity of the detected smell type, based on the baseline severity categories defined in Table 16.	Dependency smells differ in their expected consequences, and practitioner ratings provide additional evidence that some smells, such as invalid code signatures and invalid or missing source code URLs, are perceived as more severe than others [27], [40], [52].
Production reachability + dependency depth	0.25	Whether the smelled dependency is reachable in production and how far it is from the root project in the dependency graph.	Not all installed dependencies affect production behaviour, and npm dependencies may be bundled into production JavaScript code; therefore, production reachability and dependency position are relevant for prioritisation [50], [53].
Vulnerability exposure + persistence	0.30	Whether the affected dependency is associated with known vulnerabilities, and whether vulnerability severity or persistence increases remediation urgency.	Known vulnerabilities may persist for long periods and propagate through dependency trees, supporting the use of vulnerability severity and persistence as contextual prioritisation factors [41], [45].
Responsiveness + update strategy	0.15	Whether the affected package or dependency configuration is likely to delay remediation, considering factors such as restrictive update strategy, package age, release frequency, and number of dependencies.	Dependency update strategy, package age, release frequency, and number of dependencies influence whether packages are fast or slow responders to vulnerability fixes [49], [52].

The numerical weights assigned to the four SSSS dimensions are design choices and are not directly derived from the literature. The literature supports the relevance of the selected

dimensions and provides evidence about the factors that may influence remediation priority, but it does not prescribe a validated weighting scheme for combining them.

The weights were therefore defined heuristically according to the intended role of each dimension in the prioritisation process. Smell baseline severity (S) and vulnerability exposure and persistence (V) receive the highest weights (0.30 each), as they represent, respectively, the intrinsic relevance of the detected smell and additional evidence of known vulnerability-related exposure. Production reachability and dependency depth (P) receive a substantial but lower weight (0.25), reflecting the importance of the dependency's position and expected production relevance. Responsiveness and update strategy (R) receive the lowest weight (0.15), as this dimension primarily modifies urgency according to expected remediation delay rather than representing an immediate issue by itself. These weights constitute the baseline configuration of the SSSS, and their robustness is examined through sensitivity analysis during prototype evaluation.

4.2.3 Scoring Formula

The SSSS computes a prioritisation score for each detected smell instance by combining the four dimensions defined in the previous subsection. Each dimension is normalised to a value between 0 and 1, where higher values indicate higher remediation priority. The final score is then calculated as a weighted sum and converted to a 0-100 scale. This approach allows the SSSS to rank smell instances according to both the intrinsic severity of the smell and the contextual conditions of the affected dependency.

Table 19 summarises the symbols used in the scoring formula, the corresponding dimensions, and their value ranges.

Table 19 - Dimension normalisation

Symbol	Dimension	Value range
S	Empirical impact and practitioner-perceived severity of the smell	0-1
P	Production reachability and dependency depth	0-1
V	Vulnerability exposure and persistence	0-1
R	Responsiveness and update strategy	0-1

The smell severity dimension **S** represents the intrinsic severity of the detected smell type. It is derived from the baseline severity classification defined in the smell catalogue and normalised according to Table 20.

Table 20 - Baseline severity normalisation

Baseline severity	Normalised value
Low	0.25
Medium	0.50
High	0.75
Critical	1.00

The production reachability and dependency depth dimension **P** captures the expected production relevance of the affected dependency together with its structural proximity to the root React application. The literature indicates that actual production reachability is more informative than dependency declaration type alone, since runtime dependencies may not be included in production while development dependencies may become part of production bundles [50], [53].

The current prototype does not perform bundle or build-artefact analysis and therefore cannot determine actual production reachability in all cases. Instead, P uses dependency declaration type, graph depth, and explicit source-usage evidence when available as practical proxies. These signals provide reproducible structural context but should not be interpreted as equivalent to confirmed production inclusion. Consequently, the P dimension represents an approximation of production reachability, and this limitation is considered when interpreting the resulting SSSS scores. Table 21 presents the normalisation criteria for this dimension.

Table 21 - Production reachability and dependency depth normalisation

Condition	P value	Interpretation
Direct dependency classified as production with depth 1.	1.00	Highest priority because the dependency is directly declared for production use.
Transitive dependency classified as production with depth 2.	0.85	High priority because the dependency is close to the root through a production dependency path.
Transitive dependency classified as production with depth 3 or higher.	0.70	Relevant priority, but slightly lower because the dependency is further from the root project.
Production context cannot be determined, including a direct or transitive dependency whose dependency type is unresolved.	0.50	Moderate default used when the available graph information does not provide sufficient evidence to classify production relevance.
Development dependency.	0.30	Lower priority because it is declared for development use, although this does not guarantee absence from production.
Unused or explicitly non-reachable dependency according to available source analysis.	0.10	Lowest priority because available analysis provides evidence that the dependency is not used.

The vulnerability exposure and persistence dimension **V** captures whether the affected dependency is associated with known vulnerabilities and whether those vulnerabilities have remained unresolved for a relevant period. In the current prototype scope, this dimension is primarily based on the severity level of known vulnerabilities identified through the available vulnerability lookup sources. When temporal information is available, persistence is used as an additional prioritisation signal, since vulnerabilities that remain unresolved for longer periods increase exposure and remediation urgency. When multiple vulnerabilities affect the same dependency, or when both severity and persistence conditions apply, the highest applicable value should be used. Table 22 presents the normalisation criteria for this dimension.

Table 22 - Vulnerability exposure and persistence normalisation

Condition	V value	Interpretation
Known critical vulnerability affecting the dependency, or a known vulnerability unresolved for more than 180 days.	1.00	Highest vulnerability-related priority, because the dependency is affected by a critical vulnerability or by long-term unresolved exposure.
Known high-severity vulnerability affecting the dependency, or a known vulnerability unresolved between 90 and 180 days.	0.80	Strong vulnerability-related priority due to high severity or significant persistence.
Known medium-severity vulnerability affecting the dependency, or a known vulnerability unresolved between 30 and 89 days.	0.60	Relevant vulnerability exposure, but less urgent than high or critical cases.
Known low-severity vulnerability affecting the dependency, or a known vulnerability unresolved for less than 30 days.	0.30	Limited vulnerability-related priority.
No known vulnerability was found after a successful vulnerability lookup.	0.00	No vulnerability amplification is applied.
Vulnerability lookup unavailable, incomplete, or not conclusive.	0.25	A conservative uncertainty value is applied, since the absence of available data cannot be interpreted as evidence that the dependency is safe.

The responsiveness and update strategy dimension **R** captures whether the affected dependency or dependency configuration is likely to delay remediation. This dimension is distinct from the baseline severity dimension **S**: while **S** represents the intrinsic severity of the smell type, **R** captures contextual evidence that fixes may be unavailable, delayed, or difficult to adopt. This is particularly relevant for dependency-related smells such as Pinned Dependency, Restrictive Constraint, and Deprecated, where the dependency configuration or package status can directly affect the ability to adopt security fixes. Higher values indicate a greater likelihood of delayed or difficult remediation. Table 23 presents the normalisation criteria for this dimension.

Table 23 - Responsiveness and update strategy normalisation

Condition	R value	Interpretation
The installed package version is explicitly deprecated, or its source repository is archived.	1.00	Highest remediation concern because authoritative metadata indicates that the package version or repository is no longer supported through its normal maintenance lifecycle.
A direct pinned or restrictive constraint excludes an available fixed version, or complete activity metadata shows that the latest package release or repository activity occurred more than 730 days ago.	0.75	High remediation concern because adoption of a known fix is blocked or the available metadata indicates prolonged inactivity.
Pinned or restrictive update strategy is present, but there is no evidence that it currently blocks a known fix, or maintenance/update metadata is mixed or incomplete.	0.50	Moderate concern because the configuration may delay future remediation, but the immediate impact is not confirmed.
The package is actively maintained, has recent releases, and uses a balanced update strategy.	0.25	Lower concern, because the dependency is more likely to adopt fixes in a reasonable time.
The package shows frequent releases, active maintenance, and non-restrictive update practices.	0.10	Lowest concern in this dimension, because remediation delay is less likely.

After normalising the four dimensions, the final SSSS score is calculated using the weighted aggregation shown in Equation (1), based on the dimension weights defined in Table 18.

$$SSSS\ score = 100 \times ((0.30 \times S) + (0.25 \times P) + (0.30 \times V) + (0.15 \times R))$$

$$SSSS\ score = 100 \times ((0.30 \times S) + (0.25 \times P) + (0.30 \times V) + (0.15 \times R)) \quad (1)$$

The result is a numerical score between 0 and 100. A higher score indicates a higher remediation priority. This score can then be mapped to a qualitative rating for developer-facing communication. Table 24 presents the mapping between the final numerical score and the corresponding qualitative rating.

Table 24 - Final SSSS score to rating mapping

Final SSSS score	Final rating
$0 \leq \text{score} < 40$	Low
$40 \leq \text{score} < 70$	Medium
$70 \leq \text{score} < 90$	High
$90 \leq \text{score} \leq 100$	Critical

Scores below 40 are classified as Low, scores from 40 to below 70 as Medium, scores from 70 to below 90 as High, and scores from 90 to 100 as Critical. The numerical score supports sorting and filtering of smell instances, while the qualitative rating provides a simpler interpretation for developers. In the prototype, the rating can be represented through labels, colour encoding, or visual severity indicators in the dependency graph.

4.2.3.1 Examples

To illustrate the application of the proposed scoring formula, this subsection presents three hypothetical examples of detected software supply chain smell instances. The examples are intended to show how the same scoring mechanism combines the intrinsic severity of a smell with contextual information about production reachability, vulnerability exposure and persistence, and remediation responsiveness. The final score is calculated using the formula defined in the previous subsection and then mapped to the corresponding qualitative rating.

Example 1 - Deprecated production dependency with a high vulnerability

In this example, the detected smell is a Deprecated package affecting a direct production dependency. According to the baseline severity classification, Deprecated is classified as High, resulting in $S = 0.75$. Since the dependency is directly declared in the application dependencies and is expected to affect runtime behaviour, $P = 1.00$. The package is also associated with a known high-severity vulnerability, so $V = 0.80$. Finally, because the package is deprecated and may not receive future fixes, the remediation responsiveness value is $R = 1.00$. Table 25 summarises the dimension values assigned in this example.

Table 25 - Dimension values for Example 1: Deprecated production dependency with a high vulnerability

Dimension	Assigned value	Rationale
S	0.75	Deprecated is classified as a high baseline severity smell.
P	1.00	The affected package is a direct production dependency.
V	0.80	The dependency is affected by a known high-severity vulnerability.
R	1.00	The package is deprecated, making remediation more difficult.

Substituting the dimension values from Table 25 into Equation (1) gives:

$$SSSS\ score = 100 \times ((0.30 \times 0.75) + (0.25 \times 1.00) + (0.30 \times 0.80) + (0.15 \times 1.00))$$

$$SSSS\ score = 100 \times (0.225 + 0.250 + 0.240 + 0.150) = 86.50$$

According to the rating thresholds defined in Table 24, the resulting score of 86.50 corresponds to a High rating. This result indicates that the smell should receive high remediation priority because the affected dependency is production-reachable, vulnerable, and potentially difficult to update or replace.

Example 2 - Pinned dependency without a known vulnerability

In this example, the detected smell is a Pinned Dependency affecting a direct production dependency. According to the baseline severity classification, Pinned Dependency is classified as Medium, resulting in **S = 0.50**. Since the dependency is directly used by the application, **P = 1.00**. No known vulnerability is found after a successful vulnerability lookup, so **V = 0.00**. However, the dependency is pinned, which may delay future remediation if a fix becomes necessary, although there is no evidence that it currently blocks a known security update. Therefore, **R = 0.50**. Table 26 summarises the dimension values assigned in this example.

Table 26 - Dimension values for Example 2: Pinned dependency without a known vulnerability

Dimension	Assigned value	Rationale
S	0.50	Pinned Dependency is classified as a medium baseline severity smell.
P	1.00	The affected package is a direct production dependency.
V	0.00	No known vulnerability was found after a successful lookup.
R	0.50	The pinned version may delay future remediation, but no current fix is known to be blocked.

Substituting the dimension values from Table 26 into Equation (1) gives:

$$SSSS\ score = 100 \times ((0.30 \times 0.50) + (0.25 \times 1.00) + (0.30 \times 0.00) + (0.15 \times 0.50))$$

$$SSSS\ score = 100 \times (0.150 + 0.250 + 0.000 + 0.075) = 47.50$$

According to the rating thresholds defined in Table 24, the resulting score of 47.50 corresponds to a Medium rating. This result reflects a moderate remediation priority. The smell is relevant because it affects a production dependency, but the absence of a known vulnerability prevents it from being ranked as a high priority. This example is particularly important in the React context because pinned versions may sometimes be intentionally used to avoid breaking

changes, meaning that the smell should be interpreted together with contextual evidence rather than automatically treated as critical.

Example 3 - Inaccessible release reference in a transitive dependency with unavailable vulnerability data

In this example, the detected smell is Inaccessible Commit SHA/Release Tag affecting a transitive dependency at depth 2. According to the baseline severity classification, this smell is classified as High, resulting in **S = 0.75**, because it weakens traceability between the published artefact and its corresponding source code state. Since the dependency is transitive but production-reachable at depth 2, **P = 0.85**. The vulnerability lookup is unavailable or inconclusive, so **V = 0.25**. Finally, maintenance and update information is incomplete, so **R = 0.50**. Table 27 summarises the dimension values assigned in this example.

Table 27 - Dimension values for Example 3: Inaccessible release reference in a transitive dependency with unavailable vulnerability data

Dimension	Assigned value	Rationale
S	0.75	Inaccessible Commit SHA/Release Tag is classified as a High baseline severity smell.
P	0.85	The affected package is a transitive production-reachable dependency at depth 2.
V	0.25	Vulnerability information is unavailable or inconclusive.
R	0.50	Maintenance and update evidence is incomplete.

Substituting the dimension values from Table 27 into Equation (1) gives:

$$SSSS\ score = 100 \times ((0.30 \times 0.75) + (0.25 \times 0.85) + (0.30 \times 0.25) + (0.15 \times 0.50))$$

$$SSSS\ score = 100 \times (0.225 + 0.2125 + 0.075 + 0.075) = 58.75$$

According to the rating thresholds defined in Table 24, the resulting score of 58.75 corresponds to a Medium rating. This result shows that a smell with high baseline severity may still receive a medium final priority when vulnerability evidence is unavailable and the dependency is transitive rather than direct. Nevertheless, the score remains above the lower-priority range because the smell affects traceability and occurs in a production-reachable dependency.

4.3 Prototype Design

This section presents the design of the prototype developed to operationalise the proposed approach for detecting, scoring, and visualising software supply chain smells in React applications. The prototype is structured as two complementary applications. The backend is responsible for smell detection, data collection, and score computation, while the frontend

provides an interactive visualisation of the analysed dependency graph and the detected smell instances. This separation of concerns supports modularity, facilitates incremental evolution of detection logic, and allows the analysis results to be reused both in developer-facing visualisation and in automation contexts such as CI/CD workflows.

4.3.1 Requirements

The prototype requirements were defined to ensure that the proposed solution remains feasible within the scope of this dissertation while supporting the artefact's main objectives: detecting software supply chain smells, applying the proposed SSSS, and presenting the results in a developer-facing format. To make the requirements verifiable, each requirement has explicit acceptance criteria. These criteria provide the basis for implementation verification, while the broader evaluation of whether the artefact addresses the research problem is treated separately in Section 5.7.

The requirements were derived from three main concerns: analysis capability, prioritisation capability, and result communication. In addition, a set of architectural constraints was considered to keep the solution modular, reproducible, and compatible with both interactive and automation-oriented usage scenarios. Table 28 summarises these concerns and constraints.

Table 28 - Architectural concerns and constraints considered for the prototype

Type	Description	Design implication
Analysis concern	The prototype must analyse a React/npm project and identify instances of dependency-related smells.	The backend must support repository analysis, dependency metadata extraction, and smell detection modules.
Prioritisation concern	Detected smells must be ranked according to the proposed SSSS.	The backend must compute a score and a qualitative rating for each detected smell instance.
Visualisation concern	Developers must be able to inspect the detected smells in the context of the dependency graph.	The frontend must render graph-based results and expose severity information.
Interoperability constraint	Analysis results should be reusable by different consumers.	The backend must export results in structured JSON and Markdown formats.
Modularity constraint	The detection logic should be extensible.	Smell detectors should be separated from scoring and presentation logic.
Automation constraint	The analysis should be usable in CI/CD contexts.	The backend must support non-interactive execution and report generation.

Functional Requirements

The current prototype scope is limited to npm-based React projects. Consequently, the requirements and acceptance criteria refer to npm artefacts such as package.json and package-lock.json. The architecture, however, is designed so that additional package manager adapters could be introduced in future work without changing the scoring or visualisation layers.

Table 29 presents the functional requirements of the prototype. These requirements describe the main capabilities expected from the system and include acceptance criteria to make each requirement verifiable.

Table 29 - Functional requirements and acceptance criteria

ID	Functional requirement	Acceptance criteria
FR1	The backend shall accept a target repository identifier as input.	The backend can be executed with a valid repository identifier, starting the analysis without requiring manual code changes.
FR2	The backend shall identify the package manager and dependency artefacts used by the analysed project.	The backend detects npm-based artefacts such as package.json and, when available, package-lock.json, and reports whether the required artefacts were found.
FR3	The backend shall extract the project dependency structure required for smell detection and scoring.	The backend produces a dependency representation containing package names, versions, dependency types where available, and dependency relationships.
FR4	The backend shall detect a defined subset of software supply chain smells from the smell catalogue.	For each implemented smell, the backend reports the smell type, the affected package, and the evidence used to support the detection.
FR5	The backend shall support integration with external or reusable smell-detection sources through a modular detector interface.	At least one detector can be added or replaced without changing the scoring module or the frontend data-loading logic.
FR6	The backend shall compute an SSSS score and qualitative rating for each detected smell instance.	Each detected smell instance includes the normalised S, P, V, and R values, the final numerical score, and the final qualitative rating.
FR7	The backend shall export the analysis results in JSON format for visualisation.	The generated JSON file includes the analysed project metadata, dependency graph data, detected smell instances, and severity scoring information.

Table 29 (continued)

ID	Functional requirement	Acceptance criteria
FR8	The backend shall export a Markdown report for textual inspection and CI/CD usage.	The generated Markdown report lists the detected smells, affected packages, and final severity ratings in a readable summary.
FR9	The frontend shall load the JSON output generated by the backend.	The frontend successfully parses a valid JSON output file and displays the corresponding analysis data without requiring backend execution at runtime.
FR10	The frontend shall visualise the dependency graph and highlight smell instances using severity-related visual cues.	The frontend renders nodes and edges from the JSON file and visually distinguishes packages affected by detected smells.
FR11	The frontend shall allow the user to inspect details of a detected smell instance.	Selecting or inspecting an affected package displays at least the smell type, affected dependency, final score, qualitative rating, and available evidence.
FR12	The prototype shall support non-interactive execution for automation scenarios.	The backend can be executed from the command line and produces JSON and Markdown outputs without requiring frontend interaction.

The use of specific external tools is not defined as a functional requirement. Instead, such tools are treated as implementation decisions within the Analysis and Scoring Layer, selected to reuse existing detection capabilities for part of the smell catalogue. This distinction is important because the requirement concerns the capability to detect and integrate smell evidence, while the specific tools used to implement that capability may change.

Non-Functional Requirements

Table 30 presents the non-functional requirements of the prototype. These requirements define expected quality attributes and constraints that guide the implementation and evaluation of the artefact.

Table 30 - Non-functional requirements and acceptance criteria

ID	Non-functional requirement	Acceptance criteria
NFR1	The prototype shall adopt a modular architecture separating detection, scoring, export, and visualisation responsibilities.	The backend contains separate modules or components for smell detection, SSSS scoring, and output generation, and the frontend does not implement smell detection logic.
NFR2	The detection mechanism shall be extensible.	Adding a new smell detector requires implementing or registering a detector component, without modifying the scoring formula or frontend rendering logic.
NFR3	The backend shall produce reproducible outputs for the same input and equivalent metadata sources.	Re-running the analysis on the same project version and with the same available metadata produces equivalent detected smell instances and scores.
NFR4	The JSON output shall follow a consistent structure suitable for frontend consumption.	The frontend can load the generated JSON without manual transformation, and malformed or incomplete JSON files are rejected with an error message.
NFR5	The Markdown report shall provide a concise summary of the analysis results.	The report includes the analysed project, detected smells, affected packages, final ratings, and score values.
NFR6	The frontend shall remain usable for representative dependency graphs within the prototype scope.	The frontend can render and support graph fitting, zooming, panning, filtering, and node inspection for a projected graph containing at least 200 nodes.
NFR7	The prototype shall be executable in a typical development environment.	The backend and frontend can be installed and executed using documented commands in the README.
NFR8	The backend shall be compatible with automation workflows.	The backend can be executed in a command-line environment and returns generated artefacts that can be stored as CI/CD outputs.
NFR9	The prototype shall avoid exposing unnecessary sensitive data in generated outputs.	The generated JSON and Markdown outputs contain dependency and repository analysis information, but do not intentionally include credentials, tokens, or private environment variables.

Use cases

Given the intended scope of the prototype, two main use cases represent the expected interaction scenarios: one centred on developer-driven exploratory analysis and another focused on automated execution in a CI/CD context.

Use Case 1 – Analyse a Repository and Visualise Detected Smells

- **Primary actor:** Developer
- **Goal:** Analyse a target React repository, detect smell instances, compute severity scores, and inspect the results through the web application.
- **Preconditions:**
The repository identifier and package manager are known, and the backend analysis tool is available.
- **Main success scenario:**
 1. The developer provides the target repository.
 2. The backend identifies the package manager and starts the analysis process.
 3. The backend extracts the dependency structure and available package metadata.
 4. The backend executes the configured smell detectors and records the evidence for each detected smell instance.
 5. The backend computes the severity score of each detected smell instance.
 6. The backend exports the results in .md and .json formats.
 7. The developer opens the frontend and loads the generated JSON file.
 8. The frontend renders the dependency graph and highlights the detected smells and their severity.
 9. The developer explores the graph and inspects the detected smell instances.
- **Postconditions:**
The repository is analysed, and the developer can inspect the smells and their prioritisation through the generated outputs.

Use Case 2 - Run Automated Smell Analysis in CI/CD

- **Primary actor:** CI/CD pipeline
- **Goal:** Execute smell detection automatically and generate a report that can be inspected as part of a continuous integration workflow.
- **Preconditions:**
The repository is defined in the pipeline configuration, and the backend analysis tool is available in the execution environment.
- **Main success scenario:**
 1. The CI/CD workflow triggers the backend analysis tool.
 2. The backend analyses the target repository.

3. The backend executes the configured smell detection modules and generates the corresponding evidence, scores, and report artefacts.
 4. The backend computes severity scores for the detected smell instances.
 5. The backend generates a Markdown report and a JSON output.
 6. The Markdown report is exposed as a workflow artefact for later inspection.
- **Postconditions:**
The analysis results are generated automatically and can be reviewed without opening the frontend.

Figure 2 shows the two main interaction scenarios supported by the prototype: developer-driven exploratory analysis and automated execution in CI/CD environments.

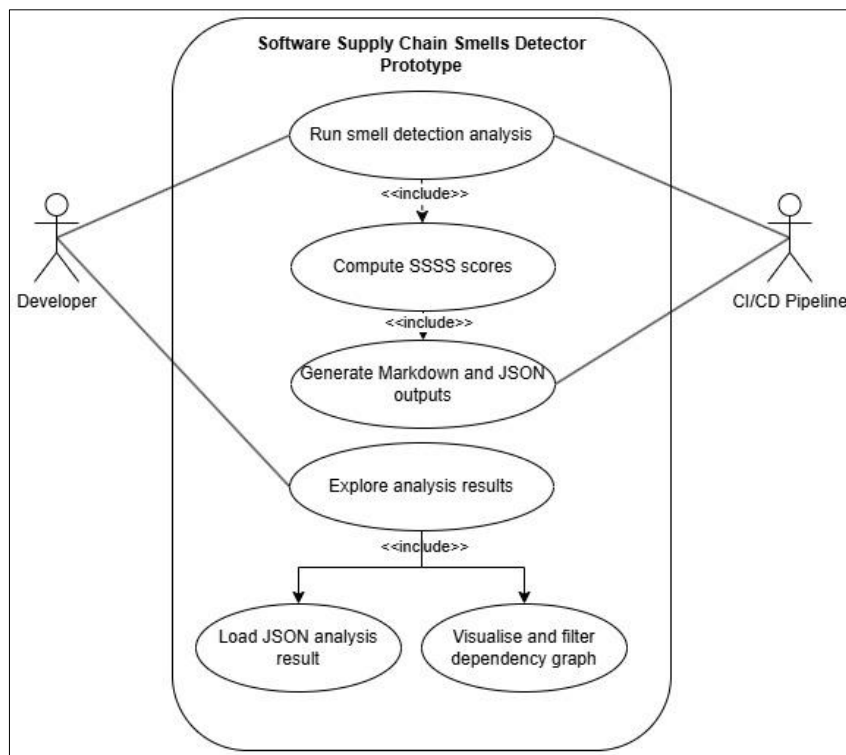


Figure 2 - Use case diagram of the proposed prototype

4.3.2 Solution Architecture

The prototype adopts a two-application architecture composed of a backend analysis application and a frontend visualisation application. This architectural decision reflects the need to separate computational concerns from presentation concerns. The backend is responsible for repository-oriented analysis, smell detection, metadata collection, and score calculation, while the frontend focuses on visual communication of the results.

At a high level, the workflow is as follows:

- First, the backend receives the target repository identifier and identifies the artefacts associated with the project under analysis.
- It then extracts the dependency structure required for smell detection and scoring. The smell detection process is performed through two complementary mechanisms:
 - One module provides an external detector adapter, which allows the prototype to reuse existing smell-detection tools when appropriate. In the current implementation, this adapter integrates Dirty-Waters for the subset of smells it supports.
 - Another module(s) performs additional analysis for smells that require custom detection logic.
- The outputs of these modules are then processed by a scoring module, which applies the proposed SSSS and produces structured results.
- Finally, the backend exports two artefacts: a Markdown summary for reporting and CI/CD usage, and a JSON file for visualisation. The frontend consumes the generated JSON file and renders the smell-enriched dependency graph.

Figure 3 presents the high-level workflow of the proposed solution architecture. The diagram shows how an analysis request is processed by the backend, how smell detection combines an external detector adapter with custom detection modules, how the SSSS scoring module assigns prioritisation scores, and how the resulting Markdown and JSON artefacts are generated. The JSON output is then consumed by the Web Application Layer to render the smell-focused dependency graph.

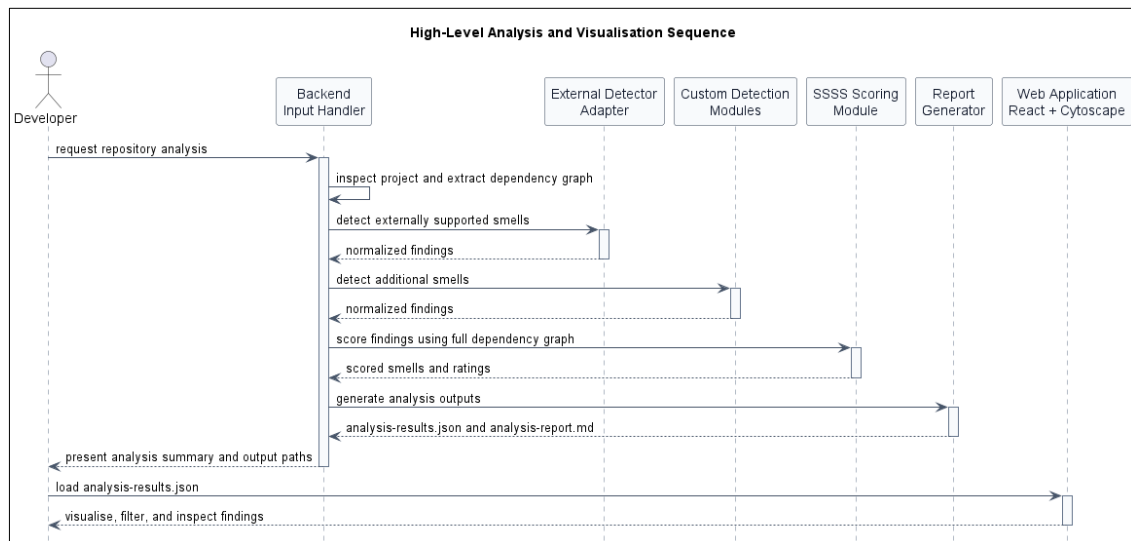


Figure 3 - High-level backend analysis and visualisation sequence

To make the interaction between the backend and frontend explicit, the prototype defines a structured JSON output format. This file acts as the data contract between the Analysis and Scoring Layer and the Web Application Layer. The backend generates this file after completing dependency extraction, smell detection, and SSSS scoring, while the frontend loads it and

renders the corresponding dependency graph and smell information. Defining this structure supports interoperability, reproducibility, and separation of concerns, since the frontend does not execute detection logic or recompute severity scores.

To improve the usability and scalability of the visualisation layer, the dependency graph exported by the backend is not intended to represent the full dependency tree. Instead, the JSON output contains a projected graph focused on detected software supply chain smells. For each dependency affected by one or more smells, the backend includes the affected dependency node, its immediate parent dependency nodes, and the edges connecting each parent to the affected dependency.

Dependencies without detected smells are included only when required to provide immediate context for a smelled dependency. This design avoids rendering very large dependency graphs containing hundreds or thousands of nodes that do not contribute directly to smell interpretation. As a result, the frontend can focus on the dependencies that require attention and on the closest dependency relationship responsible for introducing them.

The full dependency structure is still used internally by the backend for analysis and scoring purposes, including dimensions such as dependency depth and production or development reachability. However, the exported visual graph is intentionally reduced to support readability, performance, and developer-oriented inspection.

Table 31 presents the main elements expected in the JSON output.

Table 31 - Proposed JSON output structure

Field	Description
metadata	General information about the analysis execution, including tool version, generation date, analysed repository, and package manager.
project	Information about the analysed React/npm project, such as repository identifier, project name, and analysed branch or commit when available.
graph.nodes	List of nodes included in the projected graph. Each node represents either a dependency affected by at least one smell or a contextual parent dependency required to explain how the smelled dependency is introduced.
graph.edges	List of dependency relationships included in the projected graph, mainly connecting immediate parent dependencies to affected dependencies.
smells	List of detected smell instances, including smell type, affected package, detection source, supporting evidence, and associated scoring information.
summary	Aggregated information about the analysis results, such as the number of dependencies analysed, the number of smells detected, and the count of smells by severity level.

Listing 1 illustrates the main structure of the JSON output generated by the backend.

```
{
  "metadata": {
    "schemaVersion": "1.0",
    "generatedAt": "2026-06-01T10:30:00Z",
    "toolVersion": "0.1.0",
    "warnings": [
      "Problems detected during the analysis."
    ]
  },
  "project": {
    "name": "example-react-app",
    "repository": "owner/example-react-app",
    "packageManager": "npm",
    "analysedRef": "main"
  },
  "graph": {
    "nodes": [
      {
        "id": "react@18.2.0",
        "name": "react",
        "version": "18.2.0",
        "dependencyType": "production",
        "depth": 1,
        "hasSmells": false,
        "highestSeverity": null
      },
      {
        "id": "example-package@1.0.0",
        "name": "example-package",
        "version": "1.0.0",
        "dependencyType": "production",
        "depth": 2,
        "hasSmells": true,
        "highestSeverity": "Medium"
      }
    ],
    "edges": [
      {
        "source": "react@18.2.0",
        "target": "example-package@1.0.0",
        "relationship": "transitive",
        "smellIds": [
          "SMELL-001"
        ]
      }
    ]
  },
  "smells": [
    {
      "id": "SMELL-001",
      "type": "Deprecated",
      "affectedPackage": "example-package",
      "affectedVersion": "1.0.0",
      "detectionSource": "Dirty-Waters",
    }
  ]
}
```

```

        "evidence": "Package version is marked as deprecated in registry
metadata.",
        "score": {
            "S": 0.75,
            "P": 0.85,
            "V": 0.00,
            "R": 1.00,
            "finalScore": 58.75,
            "finalRating": "Medium",
            "baselineSeverity": "High"
        }
    },
    "summary": {
        "dependenciesAnalysed": 42,
        "smellsDetected": 1,
        "severityCounts": {
            "Low": 0,
            "Medium": 1,
            "High": 0,
            "Critical": 0
        }
    }
}

```

Listing 1 - Example of the JSON output structure

This structure is intentionally simple and aligned with the prototype scope. It contains enough information for the frontend to render the graph, highlight affected dependencies, and display the score and evidence associated with each smell instance. At the same time, it remains extensible, as additional fields can be added later to support new detectors, richer evidence, or integration with CI/CD and SBOM-based workflows.

Figure 4 presents the proposed solution architecture at a conceptual level. It identifies the main architectural responsibilities, interactions, external tools and data sources, and generated artefacts. The concrete implementation and runtime organisation of these responsibilities are described in Chapter 5.

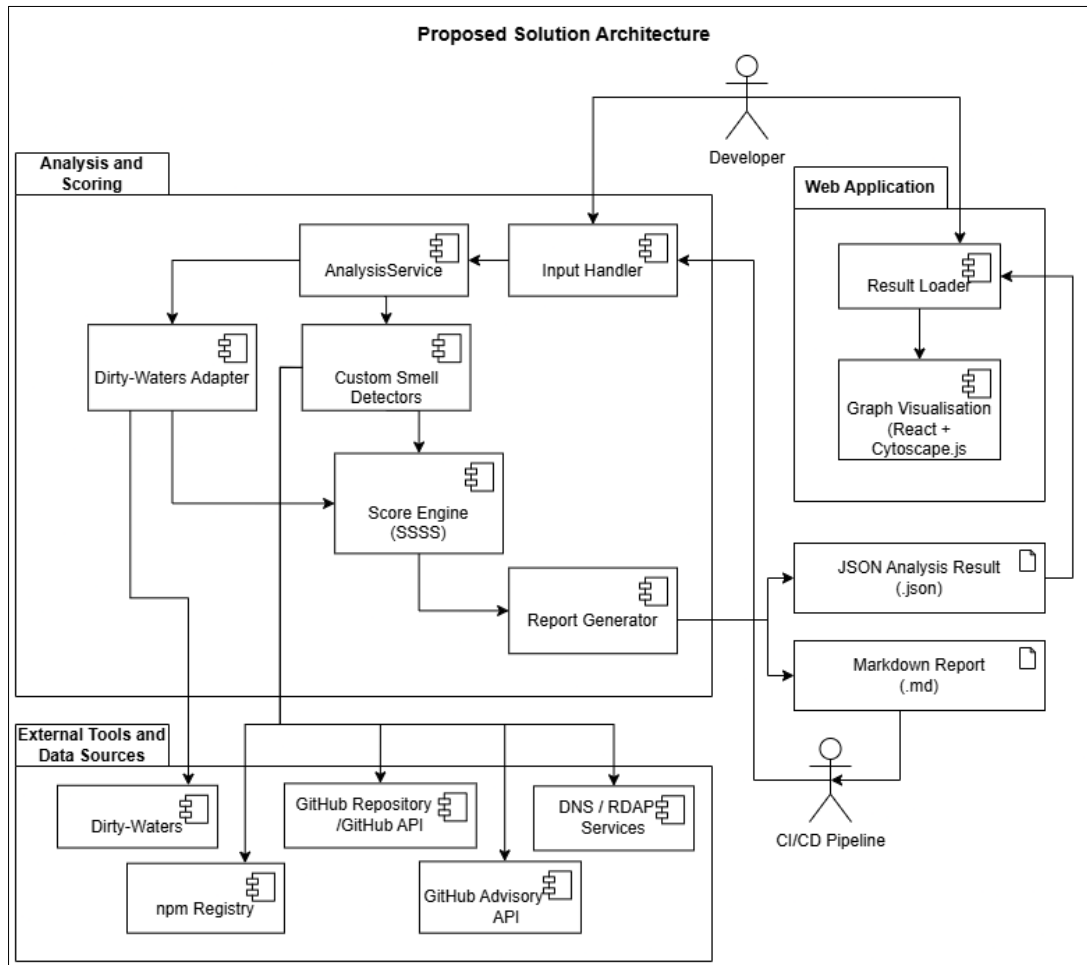


Figure 4 – Proposed Solution Architecture

The proposed architecture follows three main principles. First, it preserves modularity by decoupling smell detection strategies from scoring and presentation. Second, it supports extensibility, since new detectors can be introduced without redesigning the overall solution. Third, it supports reuse of the analysis outputs across two different contexts: developer-facing visual exploration and automated reporting in CI/CD pipelines.

4.3.2.1 Web Application Layer

The Web Application Layer is implemented as a React-based frontend whose primary responsibility is to present the backend results in a format that is easier for developers to inspect and interpret. In this prototype, the frontend does not execute smell detection logic directly. Instead, it consumes the JSON file generated by the backend and uses it to render an interactive dependency graph.

To support graph visualisation, the frontend uses the Cytoscape.js library. Cytoscape.js was selected because it is well-suited to node-link graph representations and provides the interaction capabilities needed for dependency exploration, such as node inspection,

highlighting, filtering, and layout management. This is consistent with the objective of making software supply chain smells visible within the structural context of the dependency graph, rather than presenting them only as isolated textual findings.

Figure 5 illustrates a mockup of the Web Application Layer. The mockup summarises the main interface elements expected in the prototype, including the JSON input area, the analysis summary, filtering controls, the smell-focused dependency graph, and the detail panel for the selected dependency.

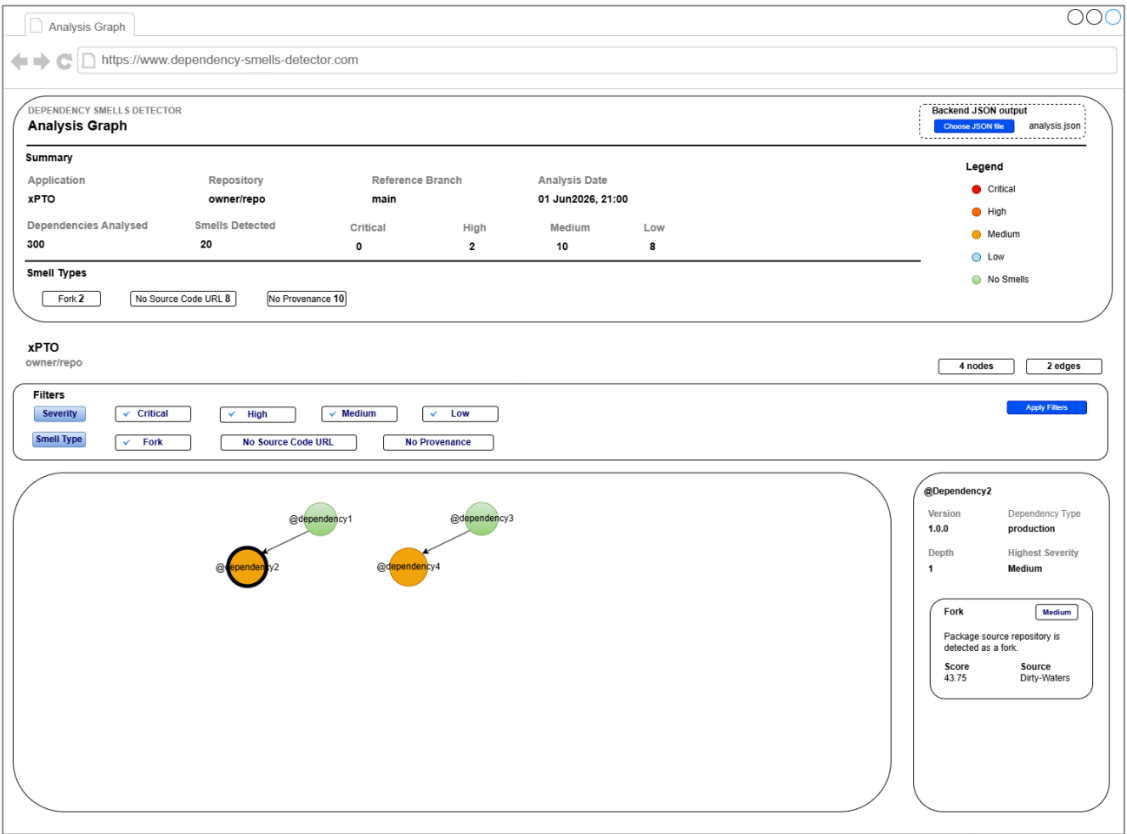


Figure 5 - Mockup of the Web Application Layer

As shown in Figure 5, the interface is organised into four main areas. The top section provides general information about the analysis, including project metadata, summary statistics, and the JSON input control. A second area provides interactive filters that allow users to refine the visualisation by severity level and smell type. The central area displays the smell-focused dependency graph, while the right-side panel presents detailed information about the currently selected dependency and its associated smell instances, including severity, score, and detection source. This organisation is intended to support fast inspection and prioritisation, while keeping the interface readable for dependency graphs within the scope of the prototype.

The main responsibilities of the Web Application Layer are therefore: loading the analysis result file, rendering the dependency graph, visually distinguishing smell instances and severity levels, and enabling the user to inspect contextual information associated with each detected smell. Although the initial mode of operation is based on loading a pre-generated JSON file, the

architecture does not prevent future evolution towards a more direct integration in which the frontend could also provide input controls for selecting the repository and package manager to be analysed.

4.3.2.2 Analysis and Scoring Layer

The Analysis and Scoring Layer is implemented in Node.js and constitutes the analytical core of the prototype. Its responsibility is to analyse the target repository, detect software supply chain smells, gather the contextual information required for scoring, and generate the final outputs used by both developers and automation workflows.

This layer is structured in a modular manner. One module is responsible for invoking Dirty-Waters and parsing its outputs in order to reuse an existing detector for a subset of smells already identified in the literature, especially those related to source code accessibility, release traceability, code signing, package aliasing, and provenance metadata. A second module is responsible for executing additional custom detection logic for other smells not directly covered by Dirty-Waters. This custom logic may inspect repository contents, dependency declarations, lockfiles, package registry information, and other metadata sources, depending on the smell being operationalised.

A scoring module complements these detection modules by applying the proposed SSSS to each detected smell instance. This module combines the baseline severity of the smell type with contextual dimensions such as production reachability, vulnerability exposure, and package responsiveness, producing a prioritisation score suitable for later visualisation and triage support.

Finally, the Analysis and Scoring Layer has two complementary outputs. The first is a Markdown report intended for textual inspection and CI/CD export scenarios, such as GitHub Actions artefacts. The second output is a JSON file containing the dependency graph together with the smells identified and their respective scores. This file is used by the Web Application Layer to present the analysis results.

The Analysis and Scoring Layer combines different detection sources according to the nature of each smell. Some smells can be obtained directly from the Dirty-Waters tool, particularly those related to source traceability, provenance, and package metadata, while others require custom analysis over repository contents or package registry metadata. Table 32 summarises the detection mechanisms and data sources associated with each software supply chain smell operationalised in the prototype.

Table 32 - Detection mechanisms and data sources associated with each operationalised smell

Smell	Detection mechanism and data sources
Pinned Dependency	GitHub Repository (package.json)
URL Dependency	GitHub Repository (package.json)
Restrictive Constraint	GitHub Repository (package.json)
Permissive Constraint	GitHub Repository (package.json)
No Package-Lock	GitHub Repository (package-lock.json / npm-shrinkwrap.json presence)
Unused Dependency	GitHub source snapshot and Knip
Missing Dependency	GitHub source snapshot and Knip
Peer Dependency Resolving Loop (PeerSpin)	GitHub Repository (package.json and lockfile) and npm Registry
No Source Code URL	Dirty-Waters analysis output
Invalid Source Code URL	Dirty-Waters analysis output
Inaccessible Commit SHA/Release Tag	Dirty-Waters analysis output
Deprecated	Dirty-Waters analysis output
Fork	Dirty-Waters analysis output
No Code Signature	Dirty-Waters analysis output
Invalid Code Signature	Dirty-Waters analysis output
No Provenance	Dirty-Waters analysis output
Aliased	Dirty-Waters analysis output
Expired Maintainer Domain	npm Registry, DNS, and RDAP services
Install Script Execution	GitHub Repository (package.json and lockfile) and npm Registry
Too Many Maintainers	npm Registry
Too Many Contributors	npm Registry

5 Implementation

This chapter presents the implementation of the solution designed in Chapter 4. It describes the technology stack and development environment, the backend analysis and scoring pipeline, the frontend visualisation layer, and the supported execution workflows. The chapter then verifies the implemented artefact against the functional and non-functional requirements defined during the design phase and evaluates its behaviour on real React/npm projects. Finally, the chapter discusses the main implementation limitations.

5.1 Implementation Overview

The implementation follows the proposed solution architecture defined in Chapter 4 through two independently executable applications. The backend implements the Analysis and Scoring Layer, while the frontend implements the Web Application Layer. This separation keeps repository inspection, smell detection, evidence enrichment, and severity calculation outside the user interface. The frontend is consequently a consumer of analysis artefacts rather than a second analysis engine.

The backend is a non-interactive command-line application designed to analyse npm-based React projects hosted on GitHub. It resolves the repository reference to be analysed, retrieves the npm manifests, builds an internal dependency graph, coordinates multiple smell detectors, enriches each finding with vulnerability and responsiveness evidence, applies the SSSS, and generates JSON and Markdown reports. Its main implementation artefacts are:

- npm project and lockfile inspection.
- Dependency graph extraction from package-lock.json or npm-shrinkwrap.json.
- An external detector adapter implemented through Dirty-Waters.
- Internal manifest, source-usage, package-governance, and peer-resolution detectors.
- Vulnerability analysis through npm audit and GitHub advisory metadata.
- SSSS based on the S, P, V, and R dimensions.
- A projected graph and structured JSON contract for the frontend.
- A concise Markdown report suitable for developer inspection or automation artefacts.

The frontend is a React application that loads an analysis-results.json file from the user's machine and maps it into a visualisation model. *Cytoscape.js* renders the projected dependency graph, while React manages file selection, summaries, staged filters, node selection, and smell details.

The implementation pipeline is shown in Figure 6. Smell detection and vulnerability analysis operate on the inspected project context, after which findings are enriched, scored, and exported.

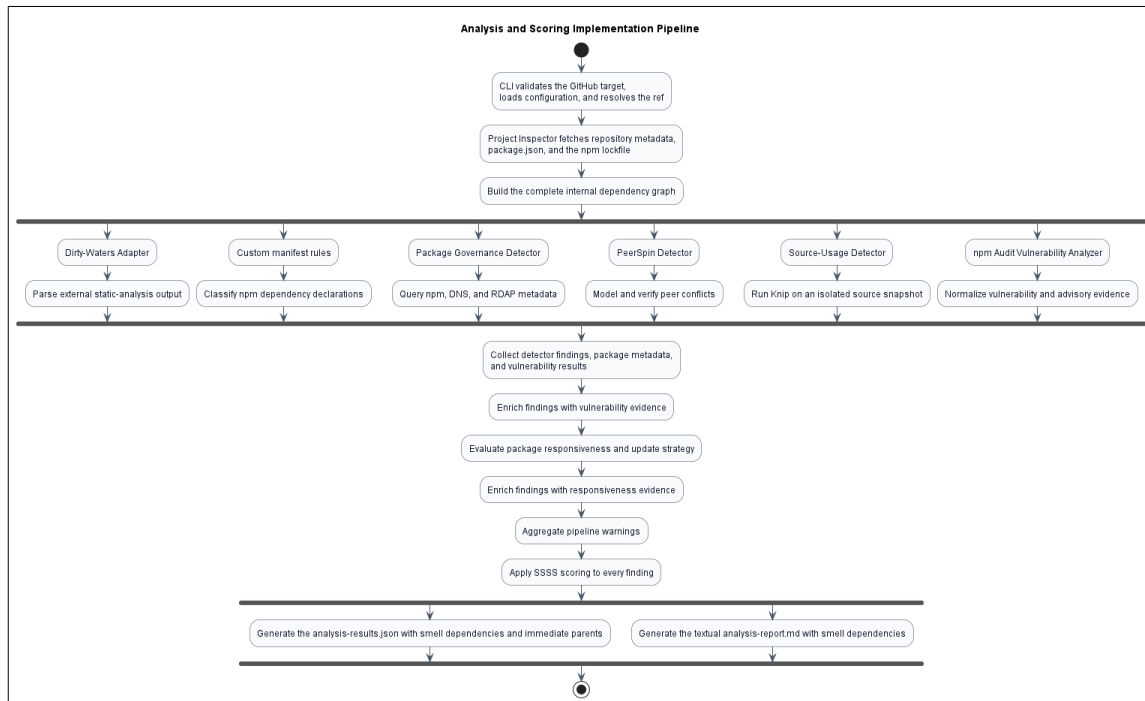


Figure 6 - Analysis and Scoring Implementation Pipeline

5.2 Technology Stack and Development Environment

The prototype uses technologies that align with the npm ecosystem. Both applications are implemented in JavaScript, which avoids a language boundary between the npm analysis domain and the React visualisation layer. ES modules are used throughout the backend and frontend.

The technologies used in the prototype were selected primarily according to their compatibility with the npm and React ecosystem, their ability to support the required analysis and visualisation capabilities, and their suitability for implementing a modular research prototype within the scope of this dissertation. Table 33 summarises the main technologies and their role in the resulting implementation.

Table 33 - Technology stack and role of each technology in the prototype

Area	Technology	Role in the prototype
Backend	Node.js and JavaScript ES modules	Provides direct access to npm tooling, package metadata formats, filesystem operations, and subprocess execution. The backend requires Node.js ^20.19.0 or >=22.12.0.
Package ecosystem	npm	Defines the prototype's package-manager scope and supplies package.json, npm lockfiles, registry metadata, and vulnerability audit data.
External detector	Dirty-Waters	Fork of Dirty-Waters [3], [27] , pinned to a specific Git revision. Implements the concrete external detector adapter for package metadata, source traceability, signature, provenance, fork, deprecation, and alias smells.
Source-usage analysis	Knip	Performs static analysis of unused and unlisted dependencies without executing the target project.
Dependency specification analysis	npm-package-arg and node-semver	Parse npm dependency specifiers and evaluate version ranges using npm-compatible semantics instead of ad hoc string rules.
Frontend	React 19	Provides component-based state management for JSON loading, summaries, filters, and detail views.
Development and build tool	Vite 7	Supplies the frontend development server and production bundling workflow.
Graph visualisation	Cytoscape.js 3	Provides graph layout, zooming, panning, styling, and node interaction for dependency data.
Data exchange	JSON	Defines a machine-readable boundary between backend analysis and frontend visualisation.
Report format	Markdown	Produces a human-readable report that can also be retained as a CI/CD artefact.
Backend testing	Node.js test runner	Verifies backend orchestration, detectors, scoring, exporters, external-client behaviour, and failure policies
Frontend testing	Vitest, React Testing Library, and jsdom	Supports automated testing of frontend data transformation, React component behaviour, user interactions, and the Cytoscape.js integration boundary.
Version control and repository access	Git and GitHub	Host the source project and provide repository contents, metadata, archives, branches, tags, and commit references.

Dirty-Waters is a Python command-line tool and is installed through pip when it is not already available. This runtime dependency is isolated behind the adapter, and the remaining backend is a Node.js application. On Windows, the adapter's package-manager preflight creates a temporary command shim when required so that Dirty-Waters can invoke npm consistently. Together with the reliance on external services and authenticated GitHub access, these runtime requirements introduce operational constraints that are discussed in the implementation limitations section.

Runtime behaviour is centralised in `backend/config/default.json`. Optional configuration files, environment variables, and CLI arguments override these defaults in the following order: defaults, configuration file, environment, and CLI. Credentials are kept outside configuration documents and are injected through environment variables. This design provides repeatable defaults while retaining operational control over timeouts, retries, concurrency, detector enablement, and external endpoints.

5.3 Backend Implementation

The backend is organised into a composition root and an orchestration service. The *createDefaultAnalysisService.js* constructs the production pipeline and injects inspectors, detector registries, vulnerability and responsiveness analysers, the scorer, and both exporters into *AnalysisService.js*. As a result, orchestration depends on small behavioural contracts rather than on concrete detector implementations.

The *AnalysisService.js* defines the execution order. Project inspection runs first because every subsequent stage requires a consistent repository identity, manifest set, and graph. Smell detection and vulnerability analysis then run concurrently. Vulnerability evidence is attached to normalised findings before responsiveness is evaluated, because the responsiveness policy can use the availability of a fixed version when assessing whether a declared constraint delays remediation. Scoring starts only after both enrichment stages have completed.

Detector failures follow an explicit optional-versus-required policy. Optional external failures become warnings and allow the remaining pipeline to continue, and a detector configured as required aborts the analysis. This is particularly important for network-dependent analysis, where a transient metadata failure should not silently be interpreted as the absence of a smell.

The backend sequence diagram (Appendix C – Backend Analysis Sequence) details the runtime orchestration of the implemented backend. After configuration is validated and the composition root creates the analysis service, the target repository is inspected to obtain the project manifests and complete dependency graph. Smell detection and vulnerability analysis are then executed over the resulting project context. Detector findings are normalised and subsequently enriched with vulnerability and responsiveness evidence before the SSSS scorer calculates the final score and qualitative rating. Finally, the scored findings are transformed into the frontend-oriented graph and exported as JSON and Markdown artefacts.

5.3.1 Input Handling and Project Extraction

The CLI accepts an `analyse` command with a required repository target in *owner/repository* format. An optional `--ref` argument identifies a branch, tag, or commit SHA. When `--ref` is present, the same value is propagated to repository-content retrieval, source snapshot materialisation, and Dirty-Waters. When it is omitted, *ProjectInspector.js* queries the GitHub repository metadata endpoint and uses the repository's `default_branch`. The effective value is stored as `project.analysedRef` in the generated JSON. If default branch resolution is unavailable, the pipeline emits a warning and permits tools to use their own defaults.

The prototype scope is React projects that use npm. The current implementation identifies the npm project through its manifests and does not independently classify or reject a repository based on a React-specific source-code signature. npm is the configured default package manager and is not exposed as a user-input CLI argument.

ProjectInspector.js retrieves the following artefacts through the GitHub API:

- Repository metadata, including the default branch, archived state, and most recent push time.
- The root `package.json`.
- `Package-lock.json`, with `npm-shrinkwrap.json` used as a supported fallback.

The inspector differentiates an absent artefact from an unavailable lookup. For example, No Package-Lock is only emitted when GitHub conclusively returns both supported lockfile paths as missing. Authentication, network, or API failures produce an unavailable status and a warning instead of a false finding. If a manifest exists but no lockfile is present, the inspector constructs a direct, unresolved graph from `package.json`. This preserves repository-level smell detection while explicitly limiting analyses that require exact installed versions.

5.3.2 Dependency Graph Extraction

PackageLockGraphExtractor.js converts npm lockfile data into a normalised graph. Modern npm lockfiles are read from the packages map, while a legacy dependency-tree path remains available for older lockfile structures. The root project uses the reserved node identifier `root`, and dependency nodes use an exact *name@version* identity. Scoped package names are preserved when deriving identities from lockfile installation paths.

Each internal node contains its identifier, package name, resolved version, dependency type, depth, smell flag, and highest final severity. Edges point from a dependent package to the dependency it declares. Root edges are classified as direct, while package-to-package edges are classified as transitive.

Dependency type is derived by graph reachability rather than solely from the lockfile entry's dev flag. Entry points from dependencies, optionalDependencies, and root peerDependencies

establish production reachability, and devDependencies establish development reachability. When the same package version is reachable through both scopes, production takes precedence. Its depth is then the shortest path within the production-reachable subgraph, even if a shorter development-only path exists. This rule keeps the P dimension aligned with runtime exposure.

The complete graph is retained inside the backend. It supports exact correlation finding, parent lookup, production reachability, depth calculation, vulnerability matching, and SSSS scoring. Exporting that entire graph would, however, make large analyses difficult to interpret. *GraphSmellProjection.js* therefore builds a separate frontend graph containing:

- Every node affected by at least one smell.
- Each affected node's immediate parent or parents.
- The root node when it is itself affected or is an immediate parent.
- Only edges that preserve those immediate parent-to-smelled-node relationships.

Projected nodes retain `dependencyType`, `depth`, `hasSmells`, and `highestSeverity`. A projected edge contains `source`, `target`, `relationship`, and the identifiers of the smells that caused it to be retained. Relationship remains direct for a root-to-dependency relation and transitive for deeper relations. The summary's `dependenciesAnalysed` value is calculated from the complete internal graph, not from the smaller projected graph.

This dual representation is intentional. The backend preserves the context required for correct scoring, while the frontend receives a bounded explanation graph focused on actionable findings.

5.3.3 External Detector Adapter: Dirty-Waters Integration

Dirty-Waters is the current implementation of the external detector adapter defined in the solution architecture. The prototype uses a fork of the original Dirty-Waters implementation [3], [27], pinned to commit `1d131e3633c0b702c169f2f5eba2dc82d58c34ce` to support reproducibility. The fork introduces operational compatibility adjustments required during prototype integration, including Windows-compatible subprocess execution and improved handling of GitHub authentication for API access and repository cloning. These changes do not modify the smell definitions or detection rules provided by Dirty-Waters.

DirtyWatersAdapter.js encapsulates installation, execution, process compatibility, output discovery, parsing, and cleanup behind the same detector contract used by internal detectors. At startup, *DirtyWatersInstaller.js* checks whether the configured executable is available. If it is absent and automatic installation is enabled, the configured Git source is installed through pip. The installation source can identify a branch or revision and is controlled through central configuration. The adapter then prepares the package-manager environment and invokes Dirty-Waters with the repository identifier, npm as the package manager, the effective Git ref when available, and the static checks enabled for the prototype.

Dirty-Waters generates a static-results artefact in its own result directory. The adapter only accepts a result whose modification time belongs to the current execution, preventing stale data from an earlier analysis from being associated with the current repository. `DirtyWatersOutputParser.js` maps tool-specific fields into the common finding model and translates Dirty-Waters dependency path markup into graph context. Once parsing is complete, the transient Dirty-Waters result directory is removed.

The adapter currently maps the following smells:

- No Source Code URL.
- Invalid Source Code URL.
- Inaccessible Commit SHA/Release Tag.
- Deprecated.
- Fork.
- No Code Signature.
- Invalid Code Signature.
- No Provenance.
- Aliased.

The normalised finding exposes a smell type, affected package and version, a detector source, and concise human-readable evidence. Tool-specific graph context and detailed scoring evidence remain internal to the pipeline. Temporary absolute paths, raw tool output, and credentials are not exported in the public JSON contract

5.3.4 Custom Smell Detectors

Internal detection is divided into focused detector families rather than implemented as one large rule set. All families return the same normalised finding structure and are registered through *DetectorRegistry.js*. A new detector can therefore be added at the composition boundary without modifying the scorer or either exporter.

Table 34 summarises the custom detector families implemented in the backend, identifying the artefacts analysed by each detector, the main detection logic applied, and the software supply chain smells produced.

Table 34 - Custom smell detector families and implemented detection logic

Detector family	Analysed artefacts	Detection logic	Produced smells
Manifest constraint detector	Root package.json and lockfile presence status	Parses npm specifiers and compares valid ranges with the compatible caret range. It evaluates dependencies, devDependencies, and optionalDependencies.	Pinned Dependency, URL Dependency, Restrictive Constraint, Permissive Constraint, No Package-Lock
Source-usage detector	Exact GitHub source snapshot and root package.json	Runs Knip in dependency-only mode and maps unused declarations and unlisted imports. Repository configuration code and package scripts are not executed.	Unused Dependency, Missing Dependency
Peer-resolution detector	npm lockfile v2/v3, root manifest, and exact registry manifests	Builds a path-sensitive model, finds Peer-to-Regular and Peer-to-Peer node-replacement conflicts, and confirms candidate constraints against exact package manifests.	Peer Dependency Resolving Loop (PeerSpin)
Package-governance detector	npm Registry manifests, lockfile install-script hints, root manifest, DNS, and RDAP	Applies conservative metadata rules and configured thresholds. External uncertainty produces warnings rather than positive findings.	Expired Maintainer Domain, Install Script Execution, Too Many Maintainers, Too Many Contributors

For dependency constraints, npm-package-arg distinguishes registry versions and tags from Git, remote URL, local file, workspace, and alias specifiers. node-semver then normalises registry ranges. Exact versions with a major version of 1 or greater are classified as pinned. A range narrower than the compatible caret interval, including an exclusively upper-bounded range, is restrictive. Tags, wildcards, open ranges, and registry ranges whose minimum version has major zero or that otherwise admit incompatible updates are permissive. Git and remote tarball specifiers produce URL Dependency findings, while local, workspace, and alias declarations are not incorrectly treated as URLs.

The source-usage detector downloads a bounded archive for the effective Git ref into a temporary directory. Knip is executed using the version installed with the backend and an analyser-owned static configuration. It does not run npm install, project scripts, or source files. Knip dependency and development-dependency issues become Unused Dependency findings, while unlisted imports become Missing Dependency findings. Missing packages have no

resolved version and receive explicit synthetic graph context rather than being matched to an unrelated transitive package with the same name.

PeerSpin detection implements the two minimal node-replacement-conflict patterns. Candidate generation is bounded by configurable traversal and conflict limits. A candidate only becomes a finding after the exact package manifests confirm the constraints inferred from the lockfile. This is a conservative static approximation: it identifies confirmed conflict patterns but does not claim to reproduce every state of npm's resolver or execute an installation simulation.

Package-governance rules are evaluated independently. An expired maintainer domain requires agreement between DNS and authoritative RDAP evidence that the registrable domain is unregistered. Install Script Execution is pre-filtered with the lockfile's `hasInstallScript` field and confirmed through `preinstall`, `install`, or `postinstall` in the exact installed-version manifest, and the project root is checked directly from its repository manifest. Too Many Maintainers uses an exclusive threshold of 20 maintainers. Too Many Contributors requires at least 40 explicitly declared contributors per unique maintainer. Missing contributor metadata remains unknown and is not interpreted as zero.

5.3.5 SSSS Scoring Engine

SsssScorer.js converts normalised findings into prioritised smell instances. Before scoring, the pipeline enriches findings with two forms of contextual evidence: vulnerability exposure and package responsiveness. Package identity resolution first uses an explicit graph node identifier and otherwise requires an unambiguous exact name and version match. This prevents evidence from being assigned to another installed version of the same package.

Each dimension is normalised to the interval from 0 to 1:

- **S - Baseline severity:** obtained from the smell catalogue. Low, Medium, High, and Critical map to 0.25, 0.50, 0.75, and 1.00 respectively.
- **P - Production reachability:** obtained from dependency type and scoped graph depth. Direct production dependencies receive 1.00, production dependencies at depth 2 receive 0.85, and production dependencies at depth 3 or greater receive 0.70. Development-only dependencies receive 0.30. Explicit source-analysis evidence can assign 0.10 to unused or unreachable cases. Unresolved context uses 0.50. The implementation does not inspect generated production bundles or build artefacts. Consequently, dependency type and graph depth are used as reproducible proxies for production relevance rather than as evidence of confirmed production reachability.
- **V - Vulnerability exposure and persistence:** obtained by running `npm audit --package-lock-only --json --ignore-scripts` in an isolated temporary directory containing only the fetched manifests. Audit entries are matched to exact installed package versions. Low, medium, high, and critical vulnerabilities map to 0.30, 0.60, 0.80, and 1.00. GHSA publication dates are retrieved through the GitHub Advisory API, and the maximum complete age in days is retained when several advisories affect a package. The stronger

value derived from severity or persistence is used. A successful clean audit produces 0.00. Unavailable or inconclusive vulnerability data produces the conservative value 0.25.

- **R - Responsiveness and update strategy:** obtained from one central policy using Dirty-Waters observations, npm release history, root GitHub activity, direct dependency constraints, and available-fix evidence. Deprecated versions or archived repositories receive 1.00. A blocked available fix or activity older than 730 days receives 0.75. Restrictive strategies without a currently known blocked fix, mixed activity, or unavailable activity receive 0.50. Recent activity receives 0.25, reduced to 0.10 when at least four releases occurred in the preceding year.

The final SSSS score is calculated using the weighted aggregation defined in Equation (1), based on the normalised S, P, V, and R dimension values.

Scores below 40 are Low, scores from 40 to 69 are Medium, scores from 70 to 89 are High, and scores from 90 to 100 are Critical. Dimension values and the final score are rounded to two decimal places. **baselineSeverity** remains available alongside **finalRating**: the former represents the catalogue classification of the smell type, whereas the latter represents the context-sensitive result after combining all four dimensions.

Listing 2 presents an example of a High-baseline smell affecting a direct production dependency for which no vulnerability was found, and recent package activity was observed.

```
{
  "id": "SMELL-001",
  "type": "No Source Code URL",
  "affectedPackage": "example-package",
  "affectedVersion": "1.0.0",
  "detectionSource": "Dirty-Waters",
  "evidence": "Package metadata does not expose a source code repository
URL.",
  "score": {
    "S": 0.75,
    "P": 1.00,
    "V": 0.00,
    "R": 0.25,
    "finalScore": 51.25,
    "finalRating": "Medium",
    "baselineSeverity": "High"
  }
}
```

Listing 2 - Example of a scored smell instance generated by the SSSS

This example also illustrates why the final rating is not required to be equal to the baseline severity. Context can either increase or reduce the priority of an individual smell instance.

5.3.6 Output Generation: JSON and Markdown

The backend generates both artefacts from the same scored finding list. *JsonAnalysisExporter.js* first creates the projected graph and summary, writes *analysis-results.json*, and returns the in-memory document. *AnalysisService.js* then passes the same generation timestamp and summary to *MarkdownReportExporter.js*, preventing the two reports from presenting different totals or analysis times.

The JSON document contains five top-level sections, as shown in Table 35.

Table 35 - JSON top-level sections

Section	Content
metadata	Schema version, UTC generation timestamp, tool version, and non-fatal warnings.
project	Application name, GitHub repository, package manager, and effective analysed ref.
graph	Projected nodes and immediate parent-to-smelled-node edges for visualisation.
smells	Public smell records containing identity, type, affected package/version, detection source, evidence, and complete SSSS score.
summary	Number of dependencies analysed in the full graph, smell count, and totals by final severity.

Detailed *evidenceData* and *graphContext* objects are required while correlating, enriching, scoring, and projecting findings, but are not exported. This keeps the public contract focused on information used by the frontend and avoids exposing internal metadata, temporary paths, or detector-specific structures.

The Markdown report is intended for direct reading and automation artefacts. It contains the analysis date in the backend host's local time, repository, package manager, analysed ref, full dependency count, smell count, and warnings. The Detected Smells section includes a summary by severity, a summary by smell type, and a detail table containing each smell identifier, package, score, final rating, detection source, and evidence. Markdown cell content is normalised to prevent evidence text from corrupting table structure.

Both exporters create the requested output directory when necessary and use stable default filenames. The JSON is the authoritative machine-readable contract. The Markdown report is a derived human-readable view and is not subsequently parsed by the frontend.

5.4 Frontend Implementation

The Web Application Layer is implemented as a separate React application. Its first screen is the analysis workspace, not a landing page. The application has no backend connection at runtime: the user explicitly selects a JSON file produced by the analysis CLI, and processing remains in the browser.

Figure 7 shows the React application after the user selects a JSON file produced by the analysis CLI.

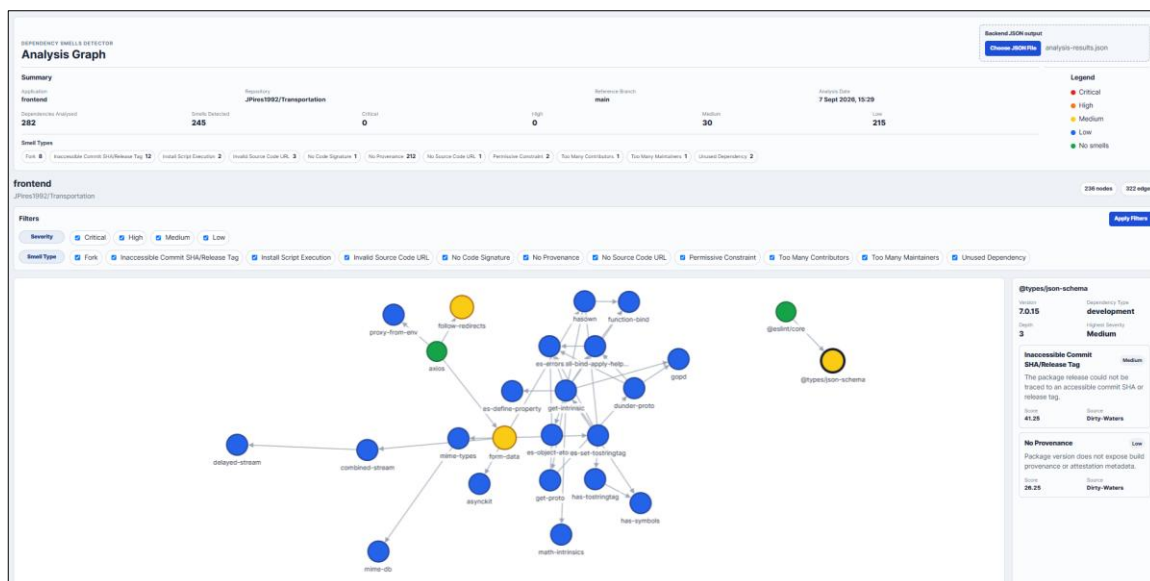


Figure 7 - React application after the user selects a JSON file

5.4.1 JSON Loading and Data Mapping

The file loader accepts JSON files through the browser File API, reads the selected file as text, parses it, and passes the resulting object to the `normalizeAnalysisResult` function in `analysisResult.js`. The normalisation boundary checks that `graph.nodes`, `graph.edges`, and `smells` are arrays before the data reaches Cytoscape. Invalid JSON or an incompatible structure produces a visible error instead of a partially rendered graph.

The frontend maps smells to nodes using the backend's canonical package identity. It enriches each node with its associated smell list and derives a highest severity only when an older artefact does not already provide one. Compatibility handling can reconstruct a root node for earlier JSON outputs that contained root findings but omitted the node itself. A fallback summary can also be calculated for older artefacts, while current reports use the backend provided summary as authoritative.

The frontend deliberately does not:

- Inspect a repository or invoke npm.

- Call Dirty-Waters, Knip, npm audit, GitHub, or the npm Registry.
- Detect new smells.
- Recompute SSSS dimensions, final scores, or ratings.
- Modify the loaded report.

This read-only boundary ensures that a result shown in the browser is the same result generated by the backend.

5.4.2 Dependency Graph Rendering

CytoscapeGraph.jsx converts normalised nodes and edges into Cytoscape elements. Node identifiers and edge endpoints are preserved from the backend contract, while the complete node object is retained as element data for selection. Directed edges use a target arrow to show the parent-to-dependency direction.

Node size also increases with severity, and the currently selected element receives a stronger dark border. Labels show package names and use bounded width with ellipsis to reduce overlap.

The *cose* force-directed layout is configured dynamically from the visible node count. Repulsion, ideal edge length, component spacing, overlap separation, gravity, and layout iterations are adjusted to produce a less compact result for larger graphs. Initial rendering fits the visible graph into the canvas, while bounded zoom and wheel sensitivity support inspection. Filtering recreates the graph with a layout appropriate to the smaller result, avoiding the excessive initial scale that occurs when coordinates from a larger graph are retained.

The rendered graph is already focused on smells because projection occurs in the backend. A green node therefore represents an immediate parent required to explain a smelled dependency, not an arbitrary unaffected package from the complete dependency tree.

5.4.3 Filtering and Interaction Mechanisms

The application provides two dynamic filter groups:

- Final severity: Critical, High, Medium, and Low.
- Smell type: the distinct smell names present in the loaded JSON.

Filter controls maintain separate draft and applied states. Checkbox changes do not immediately rebuild Cytoscape, but only after the user confirms the selection with Apply Filters. This staged interaction avoids repeated force-layout calculations when several options are changed in a large result.

A node remains visible when at least one of its smells matches both the selected severity set and the selected smell-type set. The filtering algorithm then retains immediate parent nodes

and the connecting edges required to interpret each matching result. Parent nodes are context, so they are not required to match a smell filter themselves. Counts above the graph display the number of nodes and edges in the currently applied view.

Selecting a node in Cytoscape updates the detail panel. Selecting the graph background clears the selection. Standard Cytoscape interaction provides panning and zooming without adding a second navigation model.

5.4.4 Smell Detail Panel and Summary View

The upper summary area presents analysis-level context before the graph:

- Application name.
- GitHub repository.
- Effective reference branch, tag, or SHA.
- Analysis date formatted in the browser's locale.
- Dependencies analysed in the complete backend graph.
- Total smells detected.
- Totals for Critical, High, Medium, and Low final ratings.
- Dynamic totals by smell type.

The dependency detail panel is aligned to the right of the graph. Before selection, it displays a short prompt. After selection, it shows the package name, version, production or development dependency type, graph depth, and highest final severity. Each associated smell is presented with its type, final rating, numerical score, detection source, and concise evidence. A parent-only node explicitly states that it was included for graph context and has no detected smells.

5.5 Execution Workflows

This section describes the two main workflows for executing the dependency smell analysis. The Local Developer workflow runs both applications locally, while the Automation-Oriented workflow is designed for non-interactive execution in a CI/CD pipeline. Both workflows use the same backend CLI command and produce the same JSON and Markdown outputs.

5.5.1 Local Developer Workflow

The primary workflow executes both applications locally but analyses a GitHub repository. The term local workflow therefore refers to where the tools run, not to a local repository input. A developer first ensures that Node.js, npm, and Python are installed and available in the system path. Dirty-Waters is installed automatically if it is not already present. The developer also sets `GITHUB_API_TOKEN` to a personal access token with repository read permissions.

Listing 3 shows the portable npm command used to execute the backend analysis from the backend directory.

```
npm run analyze -- --target owner/react-project --ref main --output reports
```

Listing 3 - Backend analysis command using the npm script

The `--ref` argument may be omitted. In that case, the backend resolves and records the repository's default branch. The first `--` separates npm arguments from the arguments forwarded to the analysis script. On Windows PowerShell, `npm.cmd` can be used instead of `npm` when required by the local npm installation or execution policy.

Alternatively, Listing 4 shows the equivalent direct Node.js command for executing the backend analysis.

```
node src/cli.js analyze --target owner/react-project --ref main --output reports
```

Listing 4 - Direct backend analysis command using Node.js

After the backend analysis is completed, Listing 5 shows the command used to start the frontend development server.

```
npm run dev
```

Listing 5 - Frontend development server command

The Vite server exposes the application locally. The developer selects `reports/analysis-results.json`, applies filters as required, and selects graph nodes to inspect package and smell details. The `analysis-report.md` can be opened independently for a compact textual review. Figure 8 illustrates the local workflow diagram.

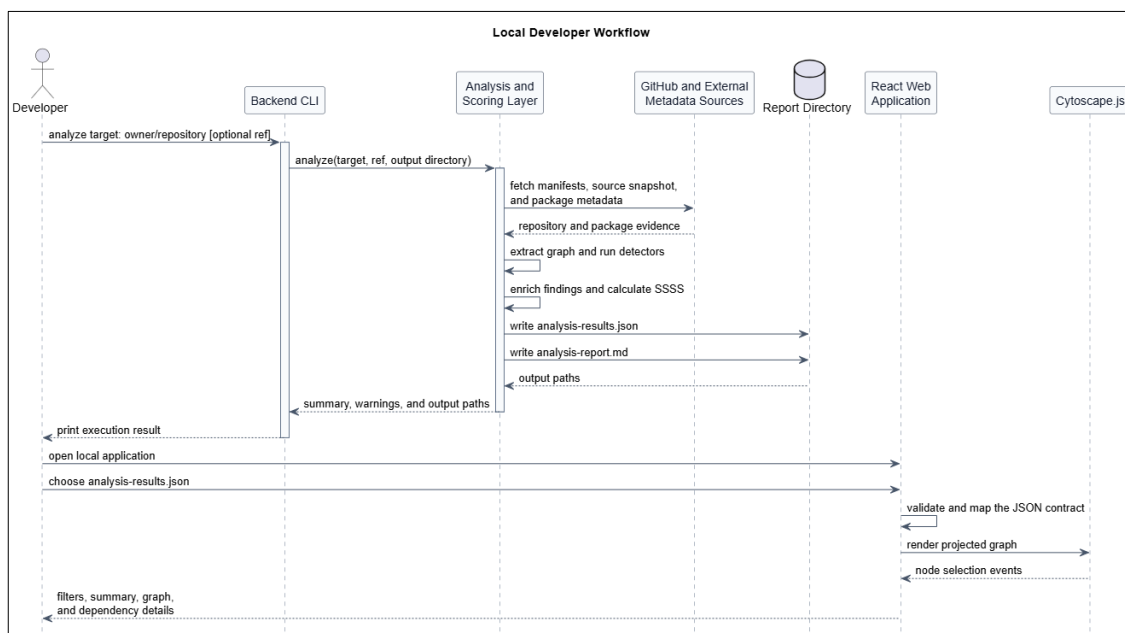


Figure 8 - Local Workflow Diagram

5.5.2 Automation-Oriented Workflow

An automation workflow can invoke the same CLI command, inject `GITHUB_API_TOKEN`, and retain `analysis-results.json` and `analysis-report.md` as build artefacts. The Markdown file supports direct inspection in common repository platforms, while the JSON file can be opened in the frontend application.

To illustrate this integration, an example GitHub Actions workflow is provided as a YAML file in the `backend/examples` directory of the project repository. This example can be adapted and placed under `.github/workflows` to automate repository analysis and save the generated reports as workflow artefacts. Figure 9 shows a successful execution of this workflow.

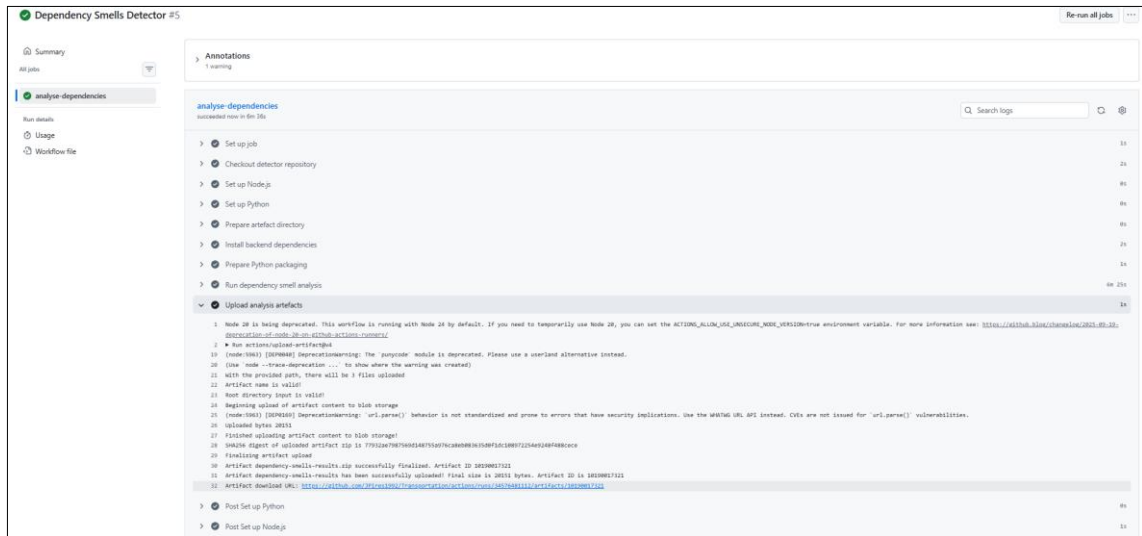


Figure 9 - Successful execution of the dependency analysis workflow in GitHub Actions

5.6 Implementation Verification

Verification combined automated backend tests, production frontend compilation, and representative manual executions. The backend test suite includes 110 tests: 109 pass, and one platform-specific test is skipped on non-Windows environments. Test coverage was also measured using the built-in coverage functionality of the Node.js test runner. Across the backend, the test suite achieves 91.32% line coverage, 77.51% branch coverage, and 92.53% function coverage. Dedicated automated tests cover critical components involved in scoring, dependency graph extraction, graph projection, and smell detection, though coverage varies by module. Table 36 summarises the coverage of selected critical backend modules.

Table 36 - Test coverage of selected critical backend modules

Module	Line coverage	Branch coverage	Function coverage
SsssScorer.js	100.00%	70.00%	100.00%
SsssPolicy.js	84.08%	82.00%	100.00%
PackageLockGraphExtractor.js	77.20%	70.97%	85.71%
GraphSmellProjection.js	89.67%	83.56%	89.47%
DirtyWatersAdapter.js	92.53%	56.67%	100.00%
CustomSmellDetector.js	96.83%	81.82%	100.00%
PackageGovernanceDetector.js	82.67%	72.41%	77.78%
PeerSpinDetector.js	76.11%	52.94%	75.00%
SourceUsageSmellDetector.js	100.00%	75.00%	100.00%

The selected modules represent the main stages of the analysis pipeline, including dependency extraction, external and internal smell detection, graph projection, and SSSS calculation. The coverage results provide additional evidence of the exercised implementation paths but should not be interpreted as evidence of the correctness of the detected findings.

The frontend is also covered by an automated test suite implemented with Vitest and React Testing Library. The suite contains 17 test cases targeting result normalisation and filtering, JSON loading and validation, application-level interactions, dependency detail inspection, and the Cytoscape.js integration boundary. Frontend coverage was measured using the V8 coverage provider. Overall, the suite achieves 93.04% statement coverage, 72.10% branch coverage, 92.20% function coverage, and 92.48% line coverage. Table 37 summarises the coverage obtained for the principal frontend modules.

Table 37 - Test coverage of principal frontend modules

Module	Statement coverage	Branch coverage	Function coverage	Line coverage
App.jsx	91.78%	61.29%	93.75%	91.42%
CytoscapeGraph.jsx	87.87%	57.14%	78.57%	86.66%
analysisResult.js	96.29%	84.50%	96.77%	95.89%

The tested modules correspond to the frontend's main responsibilities. *analysisResult.js* validates and transforms the analysis documents generated by the backend and applies graph-filtering rules. *App.jsx* coordinates JSON file loading, summary and filter presentation, and dependency-detail inspection. Finally, *CytoscapeGraph.jsx* maps the projected dependency graph to Cytoscape.js elements and handles graph node-selection events. The coverage results provide evidence that the principal frontend implementation paths are exercised by automated tests. As with the backend coverage, these measurements indicate implementation coverage rather than usability or visualisation effectiveness, which are considered separately in the prototype evaluation.

The frontend component tests isolate Cytoscape.js using test doubles, while the application-level tests replace the graph component with a controlled mock. Therefore, the automated test suite does not provide end-to-end validation of the complete workflow, from backend repository analysis to browser-based graph interaction. This integrated workflow was therefore additionally verified through representative manual executions using generated analysis artefacts.

Representative manual verification was also performed using an analysis of the JPires1992/Transportation repository [55], an npm/React project maintained by the author. This repository was selected as a controlled execution case because its source code and dependency configuration were directly available for inspection during prototype development. The analysed main branch contained 282 resolved dependencies and produced 246 smell

findings. The smell-focused graph exported for frontend visualisation contained 236 nodes and 322 edges. The analysis artefacts used for this verification are included in the examples directory of the project repository [56], supporting inspection and reproducibility of the reported execution. This repository is used in this section as a representative implementation-verification case. The broader evaluation presented in Section 5.7 considers other React/npm projects.

Table 38 summarises the verification of functional requirements. Each requirement was verified through a combination of automated tests and representative manual executions.

Table 38 - Functional requirements verification

Requirement	Verification approach	Result
FR1 - Accept a target repository	Execute the CLI with a GitHub owner/repository target, using both an explicit <code>--ref</code> and default-branch resolution. <i>ProjectInspector.test.js</i> tests also verify the rejection of unsupported local paths.	Passed
FR2 - Identify the package manager and dependency artefacts	Verify npm project context and GitHub retrieval of package.json, package-lock.json, and the supported npm-shrinkwrap.json fallback. Tests distinguish between present, conclusively missing, and unavailable artefact states.	Passed
FR3 - Extract the dependency structure	Verification of npm lockfile extraction and inspection of normalised package names, exact versions, production/development scope, depth, direct edges, and transitive edges. Tests also cover optional and peer dependencies and mixed production/development reachability.	Passed
FR4 - Detect a defined smell subset	Run focused tests for Dirty-Waters mapping, manifest constraints, lockfile absence, Knip source usage, PeerSpin conflicts, and package-governance rules. Inspect findings for type, affected package/version, source, and evidence.	Passed
FR5 - Provide a modular detector interface	Replace and register detector implementations through DetectorRegistry.js and the composition root without changing SsssScorer, the exporter, or frontend loading. Composition tests verify configurable detector lists and enablement.	Passed
FR6 - Compute SSSS score and rating	Verify that every scored finding contains normalised S, P, V, and R values, together with finalScore, finalRating, and baselineSeverity. Policy tests cover dimension thresholds and contextual enrichment.	Passed

Table 38 (continued)

Requirement	Verification approach	Result
FR7 - Export JSON for visualisation	Build and inspect analysis-results.json, including metadata, project, projected graph, public smell records, scores, warnings, and full-graph summary totals. Exporter and graph-projection tests verify the contract.	Passed
FR8 - Export a Markdown report	Build and inspect analysis-report.md, including analysis context, warnings, severity totals, smell-type totals, affected packages, scores, ratings, sources, and evidence.	Passed
FR9 - Load backend JSON in the frontend	Automated frontend tests load representative backend documents through the application's file input and verify project metadata, summary values, and dynamically generated filters. Invalid JSON and malformed backend structures are also tested to confirm that they produce an error instead of being rendered.	Passed
FR10 - Visualise and highlight the dependency graph	Automated component tests verify the conversion of projected nodes and edges into Cytoscape.js elements and the assignment of severity-specific classes for Critical, High, Medium, Low, and contextual nodes. Node-selection behaviour is also tested automatically, while the resulting graph rendering and interactive browser behaviour are additionally inspected manually.	Passed
FR11 - Inspect smell details	Automated frontend tests select affected and context-only nodes and verify that the detail panel presents package information, smell type, detection source, evidence, and the appropriate contextual message. Representative manual execution complements this verification for the complete graph interaction.	Passed
FR12 - Support non-interactive automation	Execute the analysis entirely through the CLI and verify the generation of JSON and Markdown without starting or interacting with the frontend. Optional detectors can be made mandatory to provide a failing process for incomplete mandatory analysis.	Passed

Table 39 summarises the verification of non-functional requirements. Each requirement was verified through a combination of automated tests and representative manual executions.

Table 39 - Non-functional requirements verification

Requirement	Verification approach	Result
NFR1 - Separate detection, scoring, export, and visualisation	Inspect the backend module boundaries and confirm that <i>AnalysisService.js</i> coordinates separately injected detectors, analysers, scorer, and exporters. Verify that the React application only consumes report data.	Passed
NFR2 - Make detection extensible	Register replacement and additional detector modules through <i>DetectorRegistry.js</i> . Verify that detector composition can change without modifying the SSSS formula or the frontend's dynamic smell-type handling.	Passed
NFR3 - Produce reproducible results	Repeat analyses for the same project ref and compare detected package identities, smell types, and scores when equivalent external metadata is available. Automated tests verify deterministic parsing, identity resolution, ordering, thresholds, and rejection of stale Dirty-Waters artefacts.	Passed
NFR4 - Maintain a consistent JSON structure	Verify the exporter contract and load it directly in the frontend without transformation. Confirm that invalid JSON and documents without <code>graph.nodes</code> , <code>graph.edges</code> , or <code>smells</code> arrays produce a user-facing error.	Passed
NFR5 - Provide a concise Markdown summary	Inspect the report header, warnings, summary tables, and smell detail table. Exporter tests verify project metadata, full dependency count, smell totals, score, rating, source, and evidence.	Passed
NFR6 - Remain usable up to 200 graph nodes	Load and interact with the representative Transportation analysis report, containing 236 projected nodes and 322 edges. Verify graph fitting, zooming, panning, staged filtering, node selection, and count-sensitive layout configuration through representative manual execution. Automated frontend tests additionally verify filtering logic, graph-element construction, severity classes, and node-selection event handling.	Passed
NFR7 - Execute in a typical development environment	Install and execute both applications using the documented npm commands. Run the automated backend and frontend test suites and verify successful frontend production compilation with Vite in the development environment.	Passed

Table 39 (continued)

Requirement	Verification approach	Result
NFR8 - Remain compatible with	Run the backend without interactive prompts and verify stable JSON and Markdown artefacts that can be retained by a CI/CD system. Configuration and credentials can be injected through files and environment variables.	Passed
NFR9 - Avoid unnecessary sensitive output	Inspect the explicit JSON and Markdown DTO construction and confirm that credentials, tokens, environment variables, internal scoring evidence, temporary workspaces, and raw detector paths are not intentionally exported.	Passed

5.7 Implementation Evaluation

Following the implementation verification presented in Section 5.6, this section evaluates the behaviour of the implemented artefact when applied to independent public React/npm projects. The evaluation focuses on three aspects aligned with the objectives of the dissertation: the smell findings produced by the analysis pipeline, the prioritisation behaviour of the SSSS, and the contextual information provided to support inspection of those findings. In addition, a sensitivity analysis is performed to examine the effect of the dimension weights adopted in the SSSS. The evaluation does not aim to establish detection accuracy against a complete ground-truth dataset or to measure usability through a controlled user study. Instead, it examines the behaviour and information produced by the implemented artefact on real software projects.

5.7.1 Evaluation Strategy

The evaluation adopts a multiple-case approach based on two publicly available React projects that use npm. The projects were selected before examining the prototype results according to four inclusion criteria: they had to represent recent React applications, use npm as their package manager, include both package.json and a package-lock.json, and be publicly available under a licence permitting their use as evaluation examples. Selecting the projects according to predefined technical criteria reduces the risk of choosing cases solely because they produce favourable or particularly interesting prototype results.

Both repositories were analysed using the same prototype version and the same analysis workflow on 19 September 2026. The default main branch was used in both cases. The resulting JSON reports provide the dependency information, detected smells, evidence, SSSS dimension values, final scores, and qualitative ratings used throughout the evaluation.

The evaluation is conducted in three stages. First, the overall analysis results are characterised in terms of the number and types of detected smell instances and their final severity distribution. Second, representative findings are examined to assess how the SSSS combines baseline

severity with production relevance, vulnerability exposure and persistence, and responsiveness to produce contextualised priorities. Third, the information exposed by the graph-based frontend is examined in relation to the list-oriented outputs of the underlying analysis mechanisms. A sensitivity analysis then evaluates whether moderate variations in the SSSS dimension weights materially affect the resulting rankings and rating distribution.

The complete reports generated for both evaluation cases are available in the project repository under **docs/evaluation-results [56]** .

5.7.2 Evaluation Cases and Overall Results

The evaluation was conducted on two public React/npm projects: Satoshi Dashboard [57] and joaopauloaramuni-portfolio [58]. Both projects satisfy the selection criteria defined in Section 5.7.1 and provide distinct dependency sets for examining the behaviour of the implemented artefact. Table 40 summarises the two evaluation cases and the overall results produced by the prototype.

Table 40 - Characterisation and overall results of the evaluation cases

Project	Analysed ref	Analysis date	Dependencies analysed	Smells detected	Low	Medium	High	Critical
Satoshi Dashboard	main	19 Sep. 2026	749	672	575	97	0	0
Portfolio	main	19 Sep. 2026	301	318	247	64	6	1

The Satoshi Dashboard case contains the larger dependency set, with 749 analysed dependencies and 672 detected smell instances. Its final ratings are limited to Low and Medium. In contrast, the portfolio case contains 301 analysed dependencies and 318 smell instances but includes six High findings and one Critical finding in addition to the Low and Medium results. This difference is useful for evaluating whether the SSSS distinguishes contextual conditions rather than merely reproducing the baseline severity assigned to each smell type.

Both analyses also produced a data-availability warning for the Too Many Contributors smell. Contributor metadata was unavailable for 610 package versions in the Satoshi Dashboard case and 268 package versions in the portfolio case. The detector can still identify Too Many Contributors in package versions for which contributor metadata is available. However, in both projects this represents only a relatively small subset of the analysed dependencies. Consequently, the number of detected instances of this smell should be interpreted as a lower bound rather than as a complete characterisation of its presence across the dependency graph.

The absence of a finding for package versions without contributor metadata cannot be interpreted as evidence that the smell is absent.

5.7.3 SSSS Prioritisation Behaviour

The second stage of the evaluation examined how the SSSS contextualises the baseline severity assigned to each smell type. Rather than directly translating baseline severity into the final rating, the scoring model combines it with production relevance and dependency depth (P), vulnerability exposure and persistence (V), and package responsiveness (R). Representative findings from both evaluation projects were selected to illustrate cases in which contextual information either increased or reduced the final priority. Table 41 presents these findings.

Table 41 - Representative examples of SSSS prioritisation behaviour

Project	Package	Smell	S	P	V	R	SSSS	Final rating
Satoshi Dashboard	@vercel/speed-insights@1.2.0	Install Script Execution	1.00	1.00	0.00	0.10	56.50	Medium
Portfolio	@types/babel__generator@7.27.0	Inaccessible Commit SHA/Release Tag	0.75	0.30	0.00	0.50	37.50	Low
Satoshi Dashboard	express@4.22.2	No Provenance	0.25	1.00	0.60	0.10	52.00	Medium
Portfolio	pdfjs-dist@3.11.174	No Provenance	0.25	1.00	1.00	1.00	77.50	High
Portfolio	pdfjs-dist@3.11.174	Inaccessible Commit SHA/Release Tag	0.75	1.00	1.00	1.00	92.50	Critical

The results show that the final SSSS rating can differ substantially from the baseline severity associated with a smell. For example, Install Script Execution has a Critical baseline severity in the scoring policy. However, the instance affecting *@vercel/speed-insights@1.2.0* receives a final score of 56.50 and a Medium rating. Although the dependency is directly relevant to the project (P = 1.00), no known vulnerability exposure was identified (V = 0.00), while the responsiveness dimension contributes only 0.10. The final priority is therefore lower than would be implied by the baseline severity alone.

A stronger reduction is observed for *@types/babel__generator@7.27.0*. The Inaccessible Commit SHA/Release Tag smell has a High baseline severity (S = 0.75), but the affected

dependency is classified with $P = 0.30$, no known vulnerability exposure is identified, and $R = 0.50$. These contextual factors result in an SSSS score of 37.50 and a Low final rating. This illustrates how the model can reduce the priority of a smell with a relatively high baseline severity when the remaining dimensions indicate lower contextual relevance.

The opposite behaviour is also present in the evaluation results. No Provenance has a Low baseline severity ($S = 0.25$), but different dependency contexts lead to substantially different final ratings. In the Satoshi Dashboard case, *express@4.22.2* is directly relevant to the application ($P = 1.00$) and has $V = 0.60$, resulting in a score of 52.00 and a Medium rating.

For *pdfjs-dist@3.11.174*, the same No Provenance smell also starts with $S = 0.25$, but P , V , and R all reach 1.00. As a result, the final score increases to 77.50, and the finding is rated High. Therefore, two instances of the same smell type can receive different priorities according to their dependency and vulnerability context, rather than being treated uniformly because they share the same baseline severity.

The highest-priority example observed in the evaluation is also associated with *pdfjs-dist@3.11.174*. Its Inaccessible Commit SHA/Release Tag finding has a High baseline severity ($S = 0.75$), while P , V , and R all equal 1.00. These factors increase the final score to 92.50, corresponding to a Critical rating. The comparison with the lower-priority instances demonstrates that the SSSS can both amplify and reduce baseline severity according to the contextual evidence available for each finding.

These examples indicate that the implemented SSSS behaves as intended as a contextual prioritisation mechanism: findings are not ranked solely according to smell type, and instances sharing the same baseline severity can receive different final scores and ratings. The evaluation does not establish that these priorities represent an objectively optimal remediation order. It demonstrates that the implemented scoring model incorporates the contextual dimensions defined in Chapter 4 and produces differentiated priorities when applied to real project dependencies.

5.7.4 Information Provided by the Prototype

Beyond the prioritisation behaviour examined in the previous section, the evaluation also considered the information made available when inspecting the detected findings through the frontend. The purpose of this analysis is not to establish that the graph representation is more usable than list-based tools, but to identify the additional contextual cues exposed by the implemented artefact. Table 42 summarises the main information mechanisms available in the proposed prototype and contrasts them with those typically exposed by the list-based analysis outputs used within the analysis pipeline. The comparison focuses on the information made available for inspecting and contextualising individual findings rather than on detection capability.

Table 42 - Information exposed by list-based outputs and the proposed prototype

Information	List-based analysis outputs	Proposed prototype
Detected smell and affected package	Yes	Yes
Detection evidence/source	Yes	Yes
Dependency position and depth	Limited or not directly exposed	Yes
Parent dependency context	Limited or not directly exposed	Yes
Contextual SSSS score and rating	No	Yes
Severity-based visual distinction	Limited	Yes
Interactive filtering and node inspection	No	Yes

The prototype combines findings from the analysis pipeline with their structural position in the dependency graph. When a node is inspected, the user can access package and dependency information together with the detected smell, detection source, evidence, SSSS score, and final rating. Context-only parent nodes are also retained in the projected graph, allowing a finding to be interpreted in relation to the dependency path through which the affected package is introduced. Severity and smell-type filters further allow the displayed graph to be reduced according to the investigation being performed. Figure 10 illustrates these mechanisms using the portfolio evaluation case. The displayed graph has been filtered to exclude Low-severity findings and the highly prevalent No Provenance smell, reducing the visualised result set while preserving the remaining dependency relationships required to interpret the selected findings.

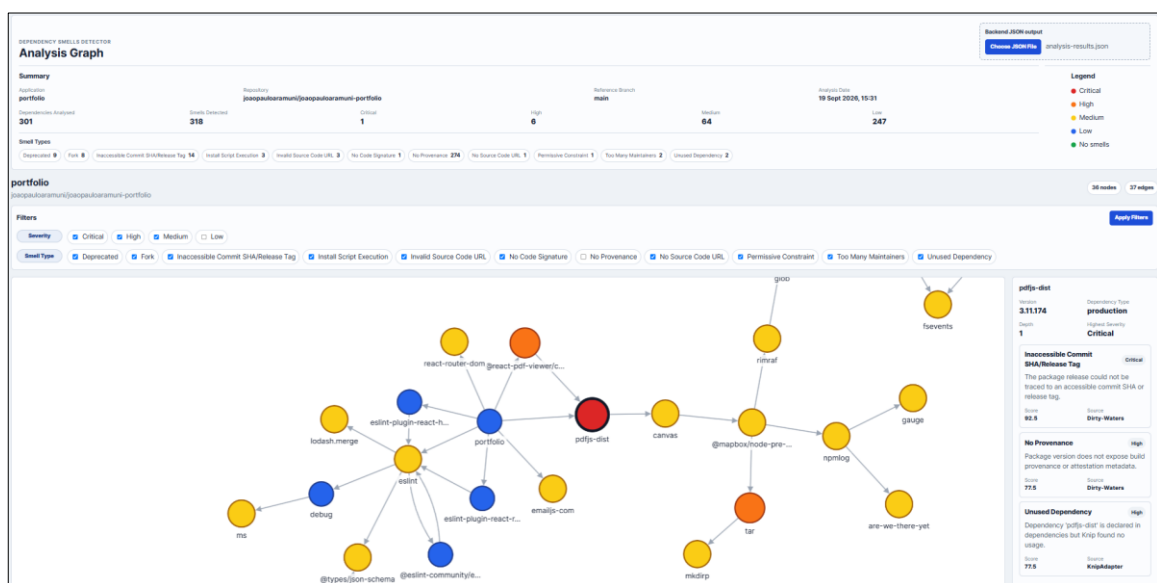


Figure 10 - Filtered frontend view of the portfolio evaluation case

As shown in Figure 10, filtering can reduce a large set of findings to a more focused dependency view while maintaining the structural context of the remaining nodes. The selected *pdfjs-dist* node also illustrates the integration of multiple information sources within the same interface: the user can inspect its dependency type and depth together with individual smell findings, their SSSS scores, final ratings, evidence, and detection sources. This allows the analysis results to be explored according to the severity or smell types relevant to a particular investigation rather than requiring all findings to be inspected simultaneously.

Compared with the underlying list-oriented analysis outputs, the main contribution of the frontend is therefore the integration of detection, prioritisation, filtering, and structural dependency context within the same representation. The evaluation shows that these information mechanisms are available and operate on the two real project analyses. However, their effect on developer efficiency or usability was not measured through a user study.

5.7.5 SSSS Sensitivity Analysis

Since the SSSS dimension weights defined in Section 4.2.2 are design choices rather than empirically validated coefficients, an illustrative sensitivity analysis was performed using three representative findings from the portfolio evaluation case. The selected findings cover different baseline severities and contextual outcomes, including cases in which contextual information increases or reduces the final rating. Each dimension weight was varied independently by -0.10, -0.05, +0.05, and +0.10. When one weight was modified, the resulting difference was distributed equally among the remaining three weights so that their total remained equal to 1. The S, P, V, and R values of the findings themselves were kept unchanged.

Table 43 - Sensitivity to the baseline severity weight (S)

S weight	<i>pdfjs-dist</i> Inaccessible Commit SHA/ Release Tag	<i>pdfjs-dist</i> No Provenance	<i>@types/babel__generator</i> Inaccessible Commit SHA/ Release Tag
0.20	95.00 Critical	85.00 High	32.67 Low
0.25	93.75 Critical	81.25 High	35.08 Low
0.30 baseline	92.50 Critical	77.50 High	37.50 Low
0.35	91.25 Critical	73.75 High	39.92 Low
0.40	90.00 Critical	70.00 High	42.33 Medium

Table 44 - Sensitivity to the production relevance weight (P)

P weight	<i>pdfjs-dist</i> Inaccessible Commit SHA/ Release Tag	<i>pdfjs-dist</i> No Provenance	<i>@types/babel__generator</i> Inaccessible Commit SHA/ Release Tag
0.15	91.67 Critical	75.00 High	38.67 Low
0.20	92.08 Critical	76.25 High	38.08 Low
0.25 baseline	92.50 Critical	77.50 High	37.50 Low
0.30	92.92 Critical	78.75 High	36.92 Low
0.35	93.33 Critical	80.00 High	36.33 Low

Table 45 - Sensitivity to the vulnerability weight (V)

V weight	<i>pdfjs-dist</i> Inaccessible Commit SHA/ Release Tag	<i>pdfjs-dist</i> No Provenance	<i>@types/babel__generator</i> Inaccessible Commit SHA/ Release Tag
0.20	91.67 Critical	75.00 High	42.67 Medium
0.25	92.08 Critical	76.25 High	40.08 Medium
0.30 baseline	92.50 Critical	77.50 High	37.50 Low
0.35	92.92 Critical	78.75 High	34.92 Low
0.40	93.33 Critical	80.00 High	32.33 Low

Table 46 - Sensitivity to the responsiveness weight (R)

R weight	<i>pdfjs-dist</i> Inaccessible Commit SHA/ Release Tag	<i>pdfjs-dist</i> No Provenance	<i>@types/babel__generator</i> Inaccessible Commit SHA/ Release Tag
0.05	91.67 Critical	75.00 High	36.00 Low
0.10	92.08 Critical	76.25 High	36.75 Low
0.15 baseline	92.50 Critical	77.50 High	37.50 Low
0.20	92.92 Critical	78.75 High	38.25 Low
0.25	93.33 Critical	80.00 High	39.00 Low

Across the tested variations, the two *pdfjs-dist* findings retain their original Critical and High ratings, although their numerical scores vary. The *@types/babel__generator* finding is more sensitive because its baseline score of 37.50 lies close to the Low/Medium threshold of 40.

Increasing the S weight to 0.40 raises it to Medium, while reducing the V weight to 0.25 or 0.20 also produces a Medium rating because this finding has $V = 0$ and the redistributed weight is assigned to dimensions with non-zero values. These examples show that moderate weight variations do not necessarily change findings that are clearly positioned within a rating range but can affect classifications close to a threshold. The analysis does not validate the selected weights as optimal. Rather, it illustrates how the adopted weighting choices influence representative prioritisation outcomes.

5.8 Implementation Limitations

Although the implemented prototype covers the main analysis, scoring, reporting, and visualisation capabilities defined in Chapter 4, several implementation limitations remain. First, the current backend supports npm projects hosted on GitHub only. Local repository analysis is intentionally excluded because part of the detection pipeline, particularly Dirty-Waters, operates against remote GitHub repositories.

Second, although the prototype is designed and evaluated in the context of React applications, the backend does not currently perform an explicit React-specific validation step. The npm/React scope is therefore established by the repositories selected for analysis rather than enforced through repository classification at runtime.

Several analyses also depend on external metadata sources and services, including GitHub, the npm Registry, npm audit, DNS, RDAP, and GitHub advisory metadata. Temporary unavailability, authentication limitations, or incomplete metadata may therefore reduce analysis coverage. The implementation distinguishes unavailable evidence from negative evidence and reports warnings rather than silently assuming that a smell is absent. Nevertheless, incomplete contextual data may affect both smell detection and the resulting SSSS dimensions. In particular, Too Many Contributors can only be evaluated for package versions for which contributor metadata is available in the npm Registry. Consequently, the absence of this metadata reduces the observable coverage of the smell without preventing the remaining governance checks from being executed.

Some detectors intentionally implement conservative approximations. PeerSpin detection identifies dependency conflict patterns without executing npm's complete installation and resolution process, while Expired Maintainer Domain relies on DNS and RDAP information to determine the observable registration status of maintainer domains. These approaches provide reproducible detection mechanisms but cannot reproduce every condition that may arise during package installation or account-level governance assessment.

The production relevance and dependency depth dimension also represents an approximation of actual production reachability. The prototype does not inspect generated production bundles or build artefacts and therefore cannot determine whether every dependency is effectively included in the deployed application. Instead, dependency declaration type, graph

depth, and source-usage evidence when available are used as practical proxies. Consequently, the resulting P value should not be interpreted as evidence of confirmed production inclusion.

From an operational perspective, the prototype requires Node.js and npm for the main application and Python for the execution of Dirty-Waters through the backend adapter. Repository analysis also relies on authenticated GitHub access through the configured GITHUB_API_TOKEN. Execution can therefore be affected by local runtime configuration, credentials, network availability, and external-service behaviour. Although these dependencies are isolated where possible through adapters and explicit failure handling, they remain operational constraints of the current implementation.

Finally, the exported dependency graph is a projection rather than the complete internal dependency graph. Only packages affected by smells and their immediate parents are exported for visualisation. This reduces the amount of information rendered by the frontend and keeps the representation focused on detected findings, but it does not provide a complete visual representation of every dependency relationship in the analysed project.

5.9 Summary

This chapter described the implementation of the prototype designed in Chapter 4. The solution was implemented as two independently executable applications: a Node.js backend responsible for repository inspection, dependency graph extraction, smell detection, contextual evidence collection, SSSS scoring, and report generation; and a React frontend responsible for interactive inspection of the resulting smell-focused dependency graph.

The backend combines reusable external analysis through Dirty-Waters with custom detectors for dependency constraints, source usage, package governance, and peer dependency conflicts. Detected findings are correlated with the internal dependency graph, enriched with vulnerability and responsiveness evidence, and scored using the SSSS before being exported as JSON and Markdown. The frontend consumes the JSON contract without repeating any analysis logic and provides severity-based visual encoding, filtering, and detailed inspection of individual smell instances.

Implementation verification showed that the functional and non-functional requirements defined in Chapter 4 are supported by the resulting artefact through automated backend and frontend tests, frontend build verification, and representative manual executions.

The prototype was subsequently evaluated on two public React/npm projects. The results showed that the SSSS can both increase and reduce the priority implied by baseline smell severity according to the contextual values of the remaining scoring dimensions. The frontend further integrates detection results, contextual prioritisation, filtering, and dependency structure within the same representation. An illustrative sensitivity analysis showed that moderate changes to the dimension weights generally preserved the ratings of findings

positioned clearly within a severity range, while findings close to a rating threshold were more sensitive to the selected weights.

Finally, the chapter identified implementation limitations related to ecosystem scope, external metadata availability, conservative detector approximations, the use of proxies for production relevance, runtime dependencies, and the projected nature of the dependency graph.

6 Conclusion

This chapter concludes the dissertation by summarising the work's main achievements and contributions in relation to the defined objectives and research questions. It also discusses the main difficulties encountered in designing and implementing the proposed solution, identifies the limitations of the resulting artefact and evaluation, and outlines directions for future work.

6.1 Achievements and Contributions

The main objective of this dissertation was to design, implement, and evaluate an approach for identifying, analysing, prioritising, and visualising software supply chain smells in React applications that use npm as their package manager. This objective was addressed through three complementary contributions: a literature-grounded smell catalogue, a contextual SSSS, and a prototype that integrates smell detection, prioritisation, reporting, and interactive dependency visualisation.

Regarding RQ1, the literature review identified and consolidated software supply chain smells relevant to npm-based React applications across dependency configuration and versioning, transparency and provenance, maintainer governance, and dependency-resolution concerns. The forward snowballing process further refined this catalogue with recent practitioner-informed evidence. The resulting catalogue was subsequently operationalised into concrete detection strategies based on project manifests, lockfiles, source usage, registry metadata, repository metadata, and external analysis tools. Smells not implemented were retained in the catalogue for traceability but excluded from the implemented subset.

Regarding RQ2, the work operationalised smell prioritisation through the proposed SSSS. Rather than assigning priority exclusively according to smell type, the model combines baseline smell severity with production relevance and dependency depth, vulnerability exposure and persistence, and package responsiveness or update strategy. The evaluation showed that this contextualisation can both increase and reduce the final priority implied by baseline severity. Findings with the same smell type or similar baseline severity can therefore receive different final scores according to the available dependency and vulnerability context. This demonstrates that the SSSS does not simply reproduce the baseline smell classification, although the evaluation does not establish that the resulting ordering is objectively optimal.

The dimension weights used by the SSSS are design choices rather than empirically validated coefficients. The sensitivity analysis therefore examined representative findings under moderate variations of each weight. Findings positioned clearly within a rating range generally retained their qualitative classification, whereas findings close to a rating threshold were more sensitive to weight changes. These results provide evidence about the influence of the selected weights without claiming that the baseline configuration is the only or optimal weighting scheme.

Regarding RQ3, the evaluation showed that the implemented frontend integrates information that is otherwise distributed across individual analysis outputs. In addition to identifying the affected package and smell, the visualisation exposes the SSSS score and rating, dependency position, parent context, detection evidence, and detection source within an interactive node-link representation. Severity and smell-type filters allow the displayed result set to be reduced according to the investigation being performed, while node inspection preserves access to detailed finding information. Compared with the underlying list-oriented outputs, the main contribution is therefore the integration of detection, contextual prioritisation, filtering, and dependency structure within the same representation.

The evaluation also confirms that contextual prioritisation does not eliminate the problem of large result sets. Real analyses may still produce a substantial number of findings, particularly when a smell is prevalent across the dependency graph. The prototype addresses this issue by allowing findings to be ordered and distinguished by contextual severity and by providing severity- and smell-based filtering, rather than by suppressing detected findings. Consequently, the contribution should be interpreted as supporting the triage and exploration of a potentially large result set rather than eliminating alert volume.

From a practical perspective, the resulting artefact can support dependency-analysis activities by bringing heterogeneous findings and contextual evidence into a common workflow that can be used interactively or incorporated into automated CI/CD execution. This is relevant to software supply chain management, where prioritisation may reduce the effort required to triage large numbers of dependency-related findings and where traceability and provenance information are increasingly important. The prototype does not provide regulatory compliance by itself, but automated dependency inspection, evidence collection, and prioritisation can contribute to the technical processes used to identify and investigate software supply chain concerns.

Taken together, these contributions show that heterogeneous software supply chain smells can be consolidated, operationalised, prioritised, and represented within a single npm-oriented analysis workflow. The resulting artefact therefore provides both a research contribution, through the structured catalogue and SSSS, and a practical contribution, through the implemented analysis and visualisation prototype. The source code and supporting documentation for the prototype are publicly available in the project repository [56].

6.2 Difficulties and Challenges

The development of the proposed solution involved several challenges arising from the integration of heterogeneous analysis sources, the characteristics of npm dependency graphs, and the operationalisation of literature-derived concepts into executable detection rules.

One of the main implementation challenges was the integration of heterogeneous analysis mechanisms. Smell detection requires information obtained from project manifests, npm lockfiles, source-code analysis, package registry metadata, repository metadata, vulnerability

advisories, and external tools. These sources use different identifiers, data structures, and failure behaviours. A common finding representation and package identity strategy were therefore required before the results could be enriched and scored consistently.

A second challenge concerned the correlation of findings with the correct dependency instance. npm dependency graphs may contain multiple installed versions of the same package, making name-only matching potentially ambiguous. The implementation therefore prioritises exact graph node identifiers and package-version identities, accepting name-only correlation only when the graph contains a unique matching node.

Another challenge involved translating literature-derived descriptions into deterministic detection rules. Several smells are conceptually well motivated but do not provide directly executable thresholds or algorithms. This required defining conservative operational rules while preserving traceability to the underlying evidence. Examples include translating the empirical thresholds identified for Too Many Maintainers and Too Many Contributors into executable detection rules, while retaining but not operationalising Overloaded Maintainer because the available evidence did not provide sufficiently validated criteria for its detection.

External metadata introduced an additional challenge because missing information cannot necessarily be interpreted as evidence that a smell or vulnerability is absent. The implementation therefore distinguishes negative evidence from unavailable or inconclusive evidence and propagates warnings or conservative scoring values where appropriate, rather than silently interpreting missing metadata as a negative result.

PeerSpin required a more specialised detection strategy because reproducing npm's complete dependency installation and node-replacement behaviour would substantially increase implementation complexity. The adopted approach therefore implements the conflict patterns described in the literature and verifies candidate package metadata through the npm Registry without executing package installation scripts.

6.3 Limitations

The implementation-specific limitations of the prototype are detailed in Section 5.8. At a broader level, the main limitations of the present work concern the coverage of the implemented smell catalogue, the validation of the SSSS, and the scope of the evaluation.

Although the literature review resulted in a broader consolidated smell catalogue, not all identified smells were operationalised in the current prototype. Hardcoded URLs and Unmaintained Packages were retained in the catalogue for traceability to the literature but excluded from the implemented subset, while Overloaded Maintainer was not operationalised because the available evidence did not provide sufficiently validated criteria for deterministic detection. Consequently, the implemented prototype should be interpreted as covering a defined subset of the catalogue rather than all software supply chain smells identified in the review.

The SSSS is grounded in the literature and in the design rationale presented in Chapter 4, but its dimensions, weights, thresholds, and scoring rules have not been independently empirically validated. The sensitivity analysis presented in Section 5.7.5 examines how moderate weight variations affect representative prioritisation outcomes, but it does not demonstrate that the selected weights are optimal. Stronger validation would require, for example, comparing SSSS-based rankings with expert judgement on a representative sample of findings or with remediation decisions observed in real software projects. These forms of validation are therefore considered directions for future work.

The evaluation presented in Section 5.7 applies the prototype to two public React/npm projects and examines overall analysis results, representative prioritisation behaviour, the information provided by the frontend, and the sensitivity of the SSSS weights. However, it does not establish detection accuracy against a complete ground-truth dataset, and the sensitivity analysis is based on representative findings rather than the complete set of possible weighting configurations and project contexts. The comparison with list-oriented analysis outputs focuses on the information and contextual mechanisms exposed by the artefact rather than on a controlled benchmark of tool effectiveness.

Finally, no user study with React developers was conducted. Consequently, although the evaluation demonstrates that filtering, contextual prioritisation, dependency structure, and detailed finding information are available through the implemented interface, their effect on developer efficiency, understanding, or usability has not been empirically measured. These limitations motivate the future work presented in the following section.

6.4 Future Work

Future work may extend the prototype beyond its current npm and GitHub scope. Support for other package managers, such as Yarn, pnpm, or Bun, would require package-manager-specific mechanisms for dependency extraction and resolution while preserving the common analysis, scoring, and output model. Support for additional repository sources and local project analysis could also broaden the applicability of the prototype.

The implemented smell catalogue may also be extended to cover smells retained in the literature-derived catalogue but excluded from the current prototype, namely Hardcoded URLs, Unmaintained Packages, and Overloaded Maintainer. Hardcoded URLs would require source-code-level URL inspection, while Unmaintained Packages would require a defensible operationalisation of package inactivity and maintenance behaviour. The implementation of Overloaded Maintainer would additionally depend on stronger empirical evidence to establish suitable detection criteria based on maintainer- and ecosystem-level information.

Further work on the SSSS should focus on independent empirical validation of its prioritisation model. This may include comparing SSSS rankings with expert assessments of representative findings or with remediation decisions observed in real projects. The scoring model could also incorporate richer vulnerability information, such as CVSS and EPSS when available, and

investigate whether the co-occurrence of multiple smells affecting the same dependency should influence priority. Production relevance could further be strengthened through analysis of generated bundles or build artefacts instead of relying primarily on dependency declaration type, graph depth, and source-usage evidence as proxies.

Finally, broader evaluation could examine the prototype across a larger and more diverse set of React/npm projects and include developers in the evaluation process. A user study could compare the proposed graph-based representation with list-oriented analysis approaches using measures such as task completion time, identification accuracy, perceived usability, and prioritisation confidence. Such evaluation would complement the artefact-based results presented in this dissertation and provide evidence regarding the effect of the proposed information and visualisation mechanisms on developer workflows.

These directions provide opportunities to extend the scope, strengthen the empirical validation, and further assess the practical applicability of the proposed approach to software supply chain smell analysis.

Bibliography

- [1] npm Inc., 'npm'. [Online]. Available: <https://www.npmjs.com/>
- [2] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, 'What are weak links in the npm supply chain?', Association for Computing Machinery (ACM), May 2022, pp. 331–340. doi: 10.1145/3510457.3513044.
- [3] R. Liu, S. Bobadilla, B. Baudry, and M. Monperrus, 'Dirty-Waters: Detecting Software Supply Chain Smells', Association for Computing Machinery (ACM), Jun. 2025, pp. 1045–1049. doi: 10.1145/3696630.3728578.
- [4] H. Tamer, D. Van Den Bongard, and F. Beck, 'Visually Analyzing the Structure and Code Quality of Component-based Web Applications', in *Proceedings - 2021 Working Conference on Software Visualization, VISSOFT 2021*, Institute of Electrical and Electronics Engineers Inc., 2021, pp. 160–164. doi: 10.1109/VISSOFT52517.2021.00031.
- [5] L. Williams *et al.*, 'Research Directions in Software Supply Chain Security', *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, May 2025, doi: 10.1145/3714464.
- [6] CISA.gov, 'Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations'. Accessed: Dec. 23, 2025. [Online]. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a>
- [7] CISA.gov, 'ED 21-01: Mitigate SolarWinds Orion Code Compromise'. Accessed: Dec. 23, 2025. [Online]. Available: <https://www.cisa.gov/news-events/directives/ed-21-01-mitigate-solarwinds-orion-code-compromise>
- [8] European Union, 'Cyber Resilience Act'. Accessed: Dec. 23, 2025. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act>
- [9] European Union, 'Cyber Resilience Act - Reporting obligations'. Accessed: Dec. 23, 2025. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/policies/cra-reporting>
- [10] OWASP Foundation, 'OWASP Dependency-Check'. [Online]. Available: <https://owasp.org/www-project-dependency-check/#>
- [11] Snyk, 'SNYK'. [Online]. Available: <https://snyk.io/>
- [12] J. vom Brocke, A. Hevner, and A. Maedche, 'Introduction to Design Science Research', 2020, pp. 1–13. doi: 10.1007/978-3-030-46781-4_1.

- [13] C. Wohlin, 'Guidelines for snowballing in systematic literature studies and a replication in software engineering', in *ACM International Conference Proceeding Series*, Association for Computing Machinery, 2014. doi: 10.1145/2601248.2601268.
- [14] M. J. Page *et al.*, 'The PRISMA 2020 statement: An updated guideline for reporting systematic reviews', Mar. 29, 2021, *BMJ Publishing Group*. doi: 10.1136/bmj.n71.
- [15] Instituto Politécnico do Porto, 'Regulamento do Código de Boas Práticas e de Conduta do Instituto Politécnico do Porto'. Accessed: Dec. 09, 2025. [Online]. Available: https://www.isep.ipp.pt/files/PPORTO_Codigoboaspraticasedeconduta.pdf
- [16] ACM, 'ACM Code of Ethics and Professional Conduct'. Accessed: Dec. 27, 2025. [Online]. Available: <https://www.acm.org/code-of-ethics/software-engineering-code>
- [17] ALLEA, 'The European Code of Conduct for Research Integrity REVISED EDITION 2023', 2023, doi: 10.26356/ECOC.
- [18] Meta, 'Describing the UI'. Accessed: Aug. 01, 2026. [Online]. Available: <https://react.dev/learn/describing-the-ui>
- [19] npm Inc., 'package.json'. Accessed: Jul. 08, 2026. [Online]. Available: <https://docs.npmjs.com/cli/v11/configuring-npm/package-json>
- [20] npm Inc., 'Package spec'. Accessed: Aug. 05, 2026. [Online]. Available: <https://docs.npmjs.com/cli/v11/using-npm/package-spec>
- [21] npm Inc., 'package-lock.json'. Accessed: Aug. 10, 2026. [Online]. Available: <https://docs.npmjs.com/cli/v11/configuring-npm/package-lock-json>
- [22] npm Inc., 'npm-package-arg'. Accessed: Aug. 21, 2026. [Online]. Available: <https://www.npmjs.com/package/npm-package-arg>
- [23] npm Inc., 'semver'. Accessed: Aug. 21, 2026. [Online]. Available: <https://www.npmjs.com/package/semver>
- [24] ISO 25000 Portal, 'ISO/IEC 25010'. Accessed: Dec. 28, 2025. [Online]. Available: <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>
- [25] MITRE, 'CWE Glossary'. Accessed: Dec. 28, 2025. [Online]. Available: <https://cwe.mitre.org/documents/glossary/index.html>
- [26] NIST, 'Guide for conducting risk assessments', Gaithersburg, MD, 2012. doi: 10.6028/NIST.SP.800-30r1.
- [27] L. Schmid, D. Gaspar, R. Liu, S. Bobadilla, B. Baudry, and M. Monperrus, 'Software Supply Chain Smells: Lightweight Analysis for Secure Dependency Management', *arXiv preprint arXiv:2603.24282*, Mar. 2026, doi: 10.48550/arXiv.2603.24282.

- [28] OWASP Foundation, 'A03:2025 Software Supply Chain Failures'. Accessed: Dec. 28, 2025. [Online]. Available: https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/
- [29] npm Inc., 'npm-audit'. Accessed: Aug. 03, 2026. [Online]. Available: <https://docs.npmjs.com/cli/v12/commands/npm-audit>
- [30] Knip, 'Unused dependencies'. Accessed: Aug. 22, 2026. [Online]. Available: <https://knip.dev/typescript/unused-dependencies>
- [31] GitHub, 'REST API endpoints for global security advisories'. Accessed: Aug. 23, 2026. [Online]. Available: <https://docs.github.com/en/rest/security-advisories/global-advisories?apiVersion=2026-03-10>
- [32] npm Inc., 'registry'. Accessed: Aug. 01, 2026. [Online]. Available: <https://docs.npmjs.com/cli/v8/using-npm/registry>
- [33] Ecma International, 'CycloneDX Bill of materials specification'. Accessed: Dec. 28, 2025. [Online]. Available: <https://ecma-international.org/publications-and-standards/standards/ecma-424/>
- [34] OWASP Foundation, 'OWASP CycloneDX (ECMA-424)'. Accessed: Dec. 28, 2025. [Online]. Available: <https://owasp.org/www-project-cyclonedx/>
- [35] NIST, 'Vulnerability Metrics'. Accessed: Dec. 28, 2025. [Online]. Available: <https://nvd.nist.gov/vuln-metrics/cvss>
- [36] Inc. FIRST.Org, 'Common Vulnerability Scoring System version 4.0 Specification Document'. Accessed: Dec. 28, 2025. [Online]. Available: <https://www.first.org/cvss/v4-0/cvss-v40-specification.pdf>
- [37] Inc. FIRST.Org, 'Exploit Prediction Scoring System (EPSS)'. Accessed: Dec. 28, 2025. [Online]. Available: <https://www.first.org/epss/>
- [38] OWASP Foundation, 'OWASP Risk Rating Methodology'. Accessed: May 11, 2026. [Online]. Available: https://owasp.org/www-community/OWASP_Risk_Rating_Methodology
- [39] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, and G. D. Bader, 'Cytoscape.js: A graph theory library for visualisation and analysis', *Bioinformatics*, vol. 32, no. 2, pp. 309–311, Jan. 2016, doi: 10.1093/bioinformatics/btv557.
- [40] A. J. Jafari, D. E. Costa, R. Abdalkareem, E. Shihab, and N. Tsantalis, 'Dependency Smells in JavaScript Projects', *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 3790–3807, Oct. 2022, doi: 10.1109/TSE.2021.3106247.

- [41] S. H. B. I. Kumar, L. R. Sampaio, A. Martin, A. Brito, and C. Fetzer, 'A Comprehensive Study on the Impact of Vulnerable Dependencies on Open-Source Software', in *Proceedings - International Symposium on Software Reliability Engineering, ISSRE*, IEEE Computer Society, 2024, pp. 96–107. doi: 10.1109/ISSRE62328.2024.00020.
- [42] M. M. A. Kabir, Y. Wang, D. Yao, and N. Meng, 'How Do Developers Follow Security-Relevant Best Practices When Using NPM Packages?', in *Proceedings - 2022 IEEE Secure Development Conference, SecDev 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 77–83. doi: 10.1109/SecDev53368.2022.00027.
- [43] A. Lemay and N. Katiyar, 'Supply Chain Risk Analysis Via SBOM Data Enrichment', in *SysCon 2025 - 19th Annual IEEE International Systems Conference, Proceedings*, Institute of Electrical and Electronics Engineers Inc., 2025. doi: 10.1109/SysCon64521.2025.11014830.
- [44] X. Wang, M. Wang, W. Shen, and R. Chang, 'Understanding and Detecting Peer Dependency Resolving Loop in npm Ecosystem', in *Proceedings - International Conference on Software Engineering*, IEEE Computer Society, 2025, pp. 129–140. doi: 10.1109/ICSE55347.2025.00054.
- [45] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, and X. Peng, 'Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem', in *Proceedings - International Conference on Software Engineering*, IEEE Computer Society, Jul. 2022, pp. 672–684. doi: 10.1145/3510003.3510142.
- [46] H. Mohayjei, A. Agaronian, E. Constantinou, N. Zannone, and A. Serebrenik, 'Investigating the Resolution of Vulnerable Dependencies with Dependabot Security Updates', in *Proceedings - 2023 IEEE/ACM 20th International Conference on Mining Software Repositories, MSR 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 234–246. doi: 10.1109/MSR59073.2023.00042.
- [47] R. Rojpaisarnkit, H. Damrongsiri, C. Treude, A. Ouni, and R. G. Kula, 'Characterising Contributions that Coincide with Vulnerability Mitigation in NPM Libraries', in *2024 IEEE/ACIS 22nd International Conference on Software Engineering Research, Management and Applications, SERA 2024 - Proceedings*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 237–242. doi: 10.1109/SERA61261.2024.10685587.
- [48] H. He, B. Vasilescu, and C. Kästner, 'Pinning Is Futile: You Need More Than Local Dependency Versioning to Defend against Supply Chain Attacks', *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 266–289, Jun. 2025, doi: 10.1145/3715728.
- [49] A. J. Jafari, D. E. Costa, A. Abdellatif, and E. Shihab, 'Dependency Practices for Vulnerability Mitigation', *arXiv preprint arXiv:2310.07847*, Oct. 2023, doi: 10.48550/arXiv.2310.07847.

- [50] J. Latendresse, S. Mujahid, D. E. Costa, and E. Shihab, 'Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM', in *ACM International Conference Proceeding Series*, Association for Computing Machinery, Sep. 2022. doi: 10.1145/3551349.3556896.
- [51] E. Wyss, D. Davidson, and L. De Carli, 'What's in a URL? An Analysis of Hardcoded URLs in npm Packages', in *SCORED 2024 - Proceedings of the 2024 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses, Co-Located with: CCS 2024*, Association for Computing Machinery, Inc, Nov. 2024, pp. 26–32. doi: 10.1145/3689944.3696168.
- [52] A. J. Jafari, 'Dependency Management Practices for the npm Software Ecosystem', Ph.D. dissertation, Concordia University, Montreal, QC, Canada, 2024.
- [53] J. Rack and C. A. Staicu, 'Jack-in-the-box: An Empirical Study of JavaScript Bundling on the Web and its Security Implications', in *CCS 2023 - Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, Association for Computing Machinery, Inc, Nov. 2023, pp. 3198–3212. doi: 10.1145/3576915.3623140.
- [54] A. Schreiber, T. Sonnekalb, and L. Von Kurnatowski, 'Towards Visual Analytics Dashboards for Provenance-driven Static Application Security Testing', in *Proceedings - 2021 IEEE Symposium on Visualization for Cyber Security, VizSec 2021*, Institute of Electrical and Electronics Engineers Inc., 2021, pp. 42–46. doi: 10.1109/VizSec53666.2021.00010.
- [55] J. P. Pires, 'Transportation'. [Online]. Available: <https://github.com/JPires1992/Transportation>
- [56] J. P. Pires, 'dependency-smells-detector'. [Online]. Available: <https://github.com/JPires1992/dependency-smells-detector>
- [57] Satoshi Dashboard, 'Satoshi Dashboard'. Accessed: Sep. 19, 2026. [Online]. Available: <https://github.com/Satoshi-Dashboard/main>
- [58] J. P. Aramuni, 'joaopauloaramuni-portfolio'. Accessed: Sep. 19, 2026. [Online]. Available: <https://github.com/joaopauloaramuni/joaopauloaramuni-portfolio>

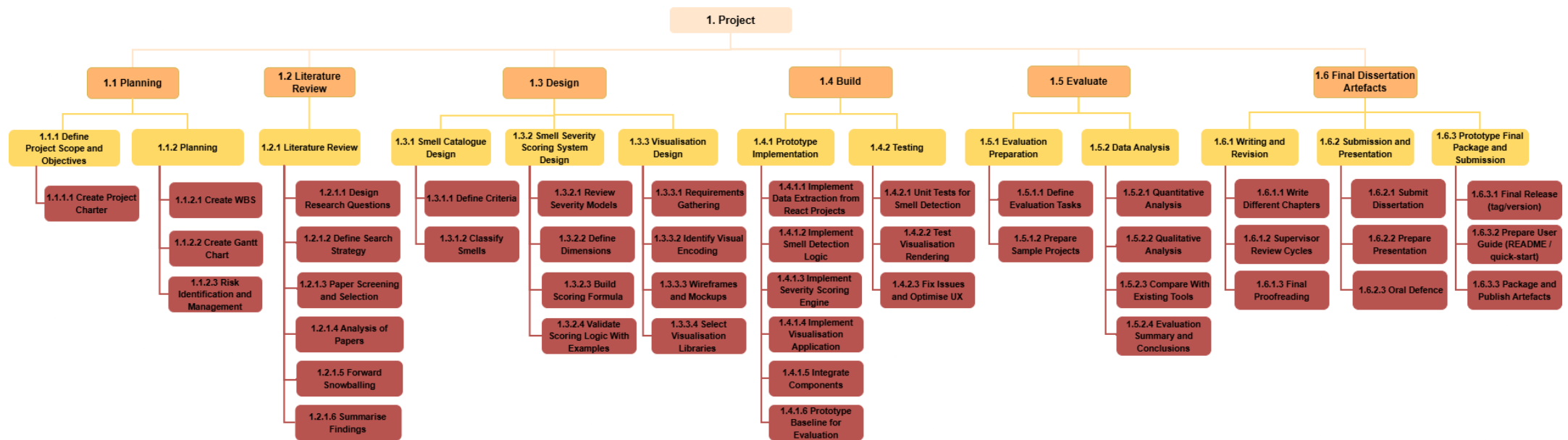
Appendix A – Project Charter

1. General Information				
Project Name:		Software Supply Chain Smells in React Applications		
Sponsor:		Isabel Azevedo		
Institution / Department		DEI.ISEP - Departamento de Engenharia Informática		
2. Project Team				
Role	Name	Department	Contact Tel	E-mail
Supervisor	Isabel Azevedo	DEI.ISEP		ifp@isep.ipp.pt
Master's Student	João Pires	-		1210624@isep.ipp.pt
3. Stakeholders				
Name			Strength	Interest
Companies that develop software in React			High	High
Developers			Low	Medium
4. Project Scope				
Problem statement / rationale				
Modern software development frequently relies on open-source dependencies, which involves significant trust in third-party code and raises concerns about software supply chain security. Software supply chain smell can be defined as a package that matches specific patterns indicating potential security issues, either currently or in the future. Software maintainability and security in component-based web applications, like those built with React, can be compromised by software supply chain smells. These smells, such as using unmaintained, stale, or bloated dependencies, can introduce potential risks that developers are often unaware of. While there has been some work on visually analysing the structure and code quality of these applications, there is a lack of a comprehensive approach that specifically targets and visualises these supply chain smells. On the other hand, developers are often overwhelmed by the volume of detected issues and lack actionable insights to triage and remediate them effectively. This makes it difficult for developers to proactively address issues, with severity-based prioritisation, related to dependency hygiene and security. Existing tools, such as the npm and third-party solutions, such as OWASP Dependency-Check and Snyk, provide partial support but do not constitute an obvious or comprehensive solution. Software Composition Analysis (SCA) tools have been developed to scan dependencies and cross-reference them with known vulnerability databases, while Software Bills of Materials (SBOMs) offer formal, machine-readable inventories of components and their hierarchical relationships. Although these techniques are valuable for security auditing, they primarily surface raw lists of vulnerabilities or versioning issues and rarely provide visual, contextual, or prioritisation mechanisms that help developers understand the structural implications of supply chain smells, such as unmaintained, stale, or bloated dependencies. As a result, current approaches fail to support developers in holistically identifying, interpreting, and managing supply chain smells within the specific context of React applications.				

Project objectives
General Objective: Design, implement, and evaluate a comprehensive approach for identifying, analysing, prioritising, and visualising software supply chain smells in React applications that use npm as their package manager. Specific Objectives: • Define and operationalise a set of software supply chain smells that affect React applications. • Propose a Smell Severity Scoring System (SSSS) to prioritise corrective actions. • Develop a visualisation tool that displays dependencies as a node-link diagram, enriched with visual cues based on the SSSS. • Evaluate the prototype on real npm/React projects in terms of smell detection, prioritisation behaviour, and the information provided, including a comparison with existing analysis tools. Literature Review A systematic literature review will be conducted following the PRISMA guidelines to identify, evaluate, and synthesise existing research on software supply chain smells, dependency management, and visualisation approaches within the npm/React ecosystems. This initial review will be complemented by the forward snowballing process to update and strengthen the evidence base by identifying recent studies that could refine, extend, or challenge the original findings. Research questions: • RQ1: What types of software supply chain smells can be systematically identified and defined in React applications? • RQ2: How can a set of software supply chain smells, specific to React applications, be operationalised in a 'Smell Severity Scoring' system to effectively prioritise remediation actions? • RQ3: What information and contextual cues can an SSSS-based dependency graph visualisation provide for identifying and prioritising software supply chain smells, compared with existing list-based analysis tools? Research Strategy The project will follow a Design Science Research framework, supported by a PRISMA-based systematic literature review to establish the theoretical foundations and justify the design of the proposed artefacts. To complement this review, the forward snowballing process is used to identify recent relevant studies for the project.
Benefits
• Improved developer awareness of software supply chain risks in React applications. • Reduction of analysis time when identifying problematic dependencies. • Better prioritisation of remediation tasks through the proposed Smell Severity Scoring system. • Potential reduction in security and maintenance incidents, due to earlier detection of dependency-related issues. • Contribution to research on software dependency visualisation and software supply chain smell detection.
Project Deliverables
• Extended Abstract • Final PREPD Document • Prototype of the Visualisation Tool • Final Dissertation Document • Source Code Repository

5. Project Schedule	
Milestones / Dates	
M1 – Extended Abstract – 15/12/2025 M2 – Final PREPD Document – 07/01/2026 M3 – Prototype Baseline for Evaluation – 31/07/2026 M4 – Prototype Final Package and Submission – 20/09/2026 M5 – Final Dissertation Document – 20/09/2026	
6. Project Costs	
Budget considerations	
Not applicable.	
7. Assumptions	8. Constraints
Access to necessary open-source tools and libraries for prototyping and visualisation.	Official ISEP dissertation timeline and deadlines.
Notes	

Appendix B – WBS



Appendix C – Backend Analysis Sequence

